CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE	**2 - 6**	
2	**DML Commands** • INSERT • UPDATE • DELETE	**7 - 11**	
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING	**12 - 15**	
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT	**16 - 21**	
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join	**22 - 29**	
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions	**30 – 37**	
7	**Subqueries and Nested Queries**	**38 - 43**	
8	**Views, Indexes, and Sequences**	**44 – 49**	
9	**Stored Procedures and Functions**	**50 – 57**	
10	**Triggers**	**58 – 64**	
11	**Cursor Implementation**	**65 – 72**	
12	**PL/SQL Exception Handling**	**73 – 82**	
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**	**83 – 89**	
14	**Mini Project**	**90-105**	
	Annexures: • **Khan Academy** • **Hacker Rank** • **Code Chef** • **Practical question**	**106 - 269**	

EX. NO:01 **Creation of a DDL operations using Movies Table**

DATE: 12.06.2026

Aim

To create a **Movies** table, insert movie records, display all records, and retrieve movies released from the year 2000 onwards.

Software Used

Oracle /MySQL/sqlite3

Description

DDL (Data Definition Language), it is used to create and modify the structure of database objects in database

DDL COMMANDS:

- Create
- Alter
- Drop
- Truncate

CREATE: Create command is used to create a new table and also use constraints.

Syntax

```
CREATE TABLE table_name (
   column1 datatype constraints,
   column2 datatype constraints,
   column3 datatype constraints
);
```

Table Structure

```
SQL> CREATE TABLE movies (
        id INTEGER PRIMARY KEY,
        name TEXT,
```

release_year INTEGER

);

Table Created

ALTER: Alter command is used to alter a table or make some changes in a table. It has 3 methods such as ADD, MODIFY, DROP

ADD: Add command is used to insert a new attribute/column with data type.

Syntax:

ALTER TABLE table_name ADD column_name datatype;

Query:

SQL> ALTER TABLE movies ADD director VARCHAR(100);

Table Altered

MODIFY: Modify command is used to modify or change the data type and to increase the data size in the table not the attribute/column.

Syntax:

ALTER TABLE table_name MODIFY column_name datatype;

Query:

SQL>ALTER TABLE movies MODIFY name VARCHAR(150);

ALTER TABLE movies ALTER COLUMN name VARCHAR(150);

DROP:Drop command is used to delete the attribute/column in a table.

Syntax:

ALTER TABLE table_name DROP COLUMN column_name;

Query:

SQL> ALTER TABLE movies DROP COLUMN director;

RENAME:Rename command is used to re-change the table name.

SQL> RENAME TABLE movies TO films;

Table renamed.

(OR)

ALTER TABLE movies RENAME TO films;

Column Rename

ALTER TABLE movies RENAME COLUMN name TO movie_name;

TRUNCATE: Truncate command is used to delete a value in the table.

Syntax: TRUNCATE TABLE tablename;

Query:

TRUNCATE TABLE movies;

DROP: Deletes the table and all its data permanently.

Syntax:DROP TABLE tablename;

Query:

DROP TABLE movies;

Explanation

- Create a table named **movies**.
- Insert movie details into the table.
- Display all movie records.
- Retrieve movies released in or after the year **2000** and sort them by release year.

Create Table

```
CREATE TABLE movies (
        id INTEGER PRIMARY KEY,
        name TEXT,
        release_year INTEGER
```

Sample Data

```
INSERT INTO movies VALUES (1, 'Avatar', 2009);
INSERT INTO movies VALUES (2, 'Titanic', 1997);
INSERT INTO movies VALUES (3, 'Star Wars: Episode IV - A New Hope', 1977);
INSERT INTO movies VALUES (4, 'Shrek 2', 2004);
INSERT INTO movies VALUES (5, 'The Lion King', 1994);
INSERT INTO movies VALUES (6, 'Disney"s Up', 2009);
```

SQL Query

Query 1: Display all movie records

```
SQL> SELECT * FROM movies;
```

Resultant Query

id	Name	release_year
1	Avatar	2009
2	Titanic	1997
3	Star Wars: Episode IV - A New Hope	1977
4	Shrek 2	2004
5	The Lion King	1994
6	Disney's Up	2009

Query 2: Display movies released from the year 2000 onwards

```
SQL> SELECT *
FROM movies
WHERE release_year >= 2000
ORDER BY release_year;
```

Resultant Query

id	name	release_year
4	Shrek 2	2004
1	Avatar	2009
6	Disney's Up	2009

RESULT:

Thus, the Movies table was created successfully, records were inserted, all movie details were displayed, and the movies released from the year 2000 onwards were retrieved successfully.

EX. NO:02 Creation of DML Operations using Students Table

DATE: 12.06.2026

Aim

To create a **Students** table, insert student records, display all records, update student details, delete a student record, and retrieve students based on their marks.

Software Used

Oracle / MySQL / sqlite3

Description

DML (Data Manipulation Language) is used to insert, modify, delete, and retrieve data stored in database tables.

DML COMMANDS:

- INSERT
- UPDATE
- DELETE
- SELECT

INSERT

INSERT command is used to add new records into a table.

Syntax

INSERT INTO table_name (column1, column2, column3)

VALUES (value1, value2, value3);

Query

SQL> INSERT INTO students VALUES (1, 'Kavin', 85);

1 row inserted.

UPDATE

UPDATE command is used to modify existing records in a table.

Syntax

UPDATE table_name

SET column_name = value

WHERE condition;

Query

SQL> UPDATE students

SET mark = 90

WHERE id = 1;

1 row updated.

DELETE

DELETE command is used to remove existing records from a table.

Syntax

DELETE FROM table_name

WHERE condition;

Query

SQL> DELETE FROM students

WHERE id = 5;

1 row deleted.

SELECT

SELECT command is used to retrieve and display records from a table.

Syntax

SELECT * FROM table_name;

Query

SQL> SELECT * FROM students;

Explanation

- Create a table named **students**.
- Insert student details into the table.
- Display all student records.
- Update the marks of a student.
- Delete a student record.
- Retrieve students who scored **80 or above** and sort them by marks.

Create Table

```
CREATE TABLE students (
   id INTEGER PRIMARY KEY,
   name TEXT,
   mark INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO students VALUES (1, 'Kavin', 85);
INSERT INTO students VALUES (2, 'Meena', 72);
INSERT INTO students VALUES (3, 'Arun', 91);
INSERT INTO students VALUES (4, 'Divya', 68);
INSERT INTO students VALUES (5, 'Rahul', 88);
INSERT INTO students VALUES (6, 'Priya', 95);
```

Records Inserted

SQL Query

Query 1: Display all student records

SQL> SELECT * FROM students;

Resultant Query

id	name	mark
1	Kavin	85
2	Meena	72
3	Arun	91
4	Divya	68
5	Rahul	88
6	Priya	95

Query 2: Update the marks of Kavin

SQL> UPDATE students

SET mark = 90

WHERE id = 1;

1 row updated.

Updated Record

id	name	mark
1	Kavin	90

Query 3: Delete Rahul's record

SQL> DELETE FROM students

WHERE id = 5;

1 row deleted.

Remaining Records

id	name	mark
1	Kavin	90
2	Meena	72
3	Arun	91
4	Divya	68
6	Priya	95

Query 4: Display students who scored 80 or above

SQL> SELECT *

FROM students

WHERE mark >= 80

ORDER BY mark;

Resultant Query

id	**name**	**mark**
1	Kavin	90
3	Arun	91
6	Priya	95

RESULT

Thus, the **Students** table was created successfully, records were inserted, student details were displayed, the student marks were updated, a student record was deleted, and the students who scored **80 or above** were retrieved successfully.

EX. NO:03 Creation of DQL Operations using Employees Table

DATE:18.06.2026

Aim

To create an Employees table, insert employee records, display all records, retrieve employees based on department and salary, sort employees by salary, and display unique departments.

Software Used

Oracle / MySQL / sqlite3

Description

DQL (Data Query Language) is used to retrieve and display data stored in database tables.

DQL COMMAND:

- SELECT

SELECT

SELECT command is used to retrieve and display records from a table.

Syntax

SELECT column1, column2, ...

FROM table_name

WHERE condition;

Query

SQL> SELECT * FROM employees;

Explanation

- Create a table named employees.
- Insert employee details into the table.
- Display all employee records.
- Retrieve employees from the IT department.
- Retrieve employees whose salary is greater than 40000.
- Sort employees by salary in descending order.
- Display unique departments.

Create Table

```
CREATE TABLE employees (
    employee_id INTEGER PRIMARY KEY,
    employee_name TEXT,
    department TEXT,
    salary INTEGER,
    city TEXT
);
```

Table Created

Sample Data

```
INSERT INTO employees VALUES (101, 'Kavin', 'IT', 45000, 'Chennai');
INSERT INTO employees VALUES (102, 'Meena', 'HR', 32000, 'Villupuram');
INSERT INTO employees VALUES (103, 'Arun', 'Finance', 38000, 'Salem');
INSERT INTO employees VALUES (104, 'Divya', 'IT', 50000, 'Trichy');
INSERT INTO employees VALUES (105, 'Rahul', 'Sales', 28000, 'Madurai');
INSERT INTO employees VALUES (106, 'Priya', 'HR', 42000, 'Chennai');
```

Records Inserted

SQL Query

Query 1: Display all employee records

```
SQL> SELECT * FROM employees;
```

Resultant Query

employee_id	employee_name	department	salary	city
101	Kavin	IT	45000	Chennai

102	Meena	HR	32000	Villupuram
103	Arun	Finance	38000	Salem
104	Divya	IT	50000	Trichy
105	Rahul	Sales	28000	Madurai
106	Priya	HR	42000	Chennai

Query 2: Display employees from IT department

SQL> SELECT employee_name, department, salary

FROM employees

WHERE department = 'IT';

Resultant Query

employee_name	department	salary
Kavin	IT	45000
Divya	IT	50000

Query 3: Display employees whose salary is greater than 40000

SQL> SELECT employee_name, salary

FROM employees

WHERE salary > 40000;

Resultant Query

employee_name	salary
Kavin	45000
Divya	50000
Priya	42000

Query 4: Display employees in descending order of salary

SQL> SELECT employee_name, salary

FROM employees

ORDER BY salary DESC;

Resultant Query

employee_name	salary
Divya	50000
Kavin	45000
Priya	42000
Arun	38000
Meena	32000
Rahul	28000

Query 5: Display unique departments

SQL> SELECT DISTINCT department

FROM employees;

Resultant Query

department
IT
HR
Finance
Sales

RESULT

Thus, the Employees table was created successfully, employee records were inserted, all employee records were displayed, employees were retrieved based on department and salary, employee records were sorted by salary, and unique departments were retrieved successfully.

EX. NO:04 Creation of Transaction Operations using

DATE:18.06.2026 Accounts Table

Aim

To create an Accounts table, insert account records, perform transaction operations using COMMIT, ROLLBACK, and SAVEPOINT, and verify the changes made to the database.

Software Used

Oracle / MySQL / sqlite3

Description

Transaction Control Language (TCL) is used to manage transactions in a database. It helps to save or undo the changes made to the database.

TRANSACTION COMMANDS:

- COMMIT
- ROLLBACK
- SAVEPOINT

COMMIT

COMMIT command is used to permanently save all the changes made during a transaction.

Syntax

COMMIT;

Query

SQL> COMMIT;

Transaction committed.

ROLLBACK

ROLLBACK command is used to undo the changes made during a transaction before the changes are committed.

Syntax

ROLLBACK;

Query

SQL> ROLLBACK;

Transaction rolled back.

SAVEPOINT

SAVEPOINT command is used to create a temporary point within a transaction. The transaction can be rolled back to that specific point.

Syntax

SAVEPOINT savepoint_name;

Query

SQL> SAVEPOINT sp1;

Savepoint created.

ROLLBACK TO SAVEPOINT

ROLLBACK TO SAVEPOINT is used to undo the changes made after a particular savepoint without undoing the entire transaction.

Syntax

ROLLBACK TO SAVEPOINT savepoint_name;

Query

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

Explanation

- Create a table named accounts.
- Insert account details into the table.
- Use COMMIT to permanently save the inserted records.
- Update an account balance and create a SAVEPOINT.
- Make another change after the savepoint.
- Use ROLLBACK TO SAVEPOINT to undo the changes made after the savepoint.
- Use ROLLBACK to undo uncommitted changes.

Create Table

```
CREATE TABLE accounts (
    account_id INTEGER PRIMARY KEY,
    account_holder TEXT,
    balance INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO accounts VALUES (101, 'Kavin', 25000);
INSERT INTO accounts VALUES (102, 'Meena', 30000);
INSERT INTO accounts VALUES (103, 'Arun', 20000);
INSERT INTO accounts VALUES (104, 'Divya', 35000);
INSERT INTO accounts VALUES (105, 'Priya', 40000);
```

Records Inserted

SQL Query

Query 1: Display all account records

SQL> SELECT * FROM accounts;

Resultant Query

account_id	account_holder	balance
101	Kavin	25000
102	Meena	30000
103	Arun	20000
104	Divya	35000
105	Priya	40000

Query 2: Commit the inserted records

SQL> COMMIT;

Transaction committed.

The inserted account records are now permanently saved.

Query 3: Update Kavin's account balance

SQL> UPDATE accounts

SET balance = 30000

WHERE account_id = 101;

1 row updated.

Updated Record

account_id	account_holder	balance
101	Kavin	30000

Query 4: Create a SAVEPOINT

SQL> SAVEPOINT sp1;

Savepoint created.

Query 5: Update Meena's account balance

SQL> UPDATE accounts

SET balance = 35000

WHERE account_id = 102;

1 row updated.

Updated Record

account_id	account_holder	balance
102	Meena	35000

Query 6: Rollback to SAVEPOINT

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

The changes made after SAVEPOINT sp1 are undone.

Resultant Query

account_id	account_holder	balance
101	Kavin	30000
102	Meena	30000

Query 7: Rollback an uncommitted change

SQL> UPDATE accounts

SET balance = 15000

WHERE account_id = 103;

1 row updated.

SQL> ROLLBACK;

Transaction rolled back.

The balance of Arun is restored to its previous value.

Resultant Query

account_id	**account_holder**	**balance**
103	**Arun**	**20000**

RESULT:

Thus, the Accounts table was created successfully, account records were inserted, and the transaction operations COMMIT, ROLLBACK, SAVEPOINT, and ROLLBACK TO SAVEPOINT were performed successfully.

EX. NO:05 **Creation of a join operations using Customers and**

DATE:22.06.2026 **Orders Table**

AIM

To create Customers and Orders tables, insert records, and retrieve related data using different types of joins such as Inner Join, Left Join, Right Join, Full Join, and Self Join.

Software Used

Oracle / MySQL / sqlite3

Description

Joins are used to combine records from two or more tables based on a related column between them.

JOIN COMMANDS:

• Inner Join

• Left Join

• Right Join

• Full Join

• Self Join

INNER JOIN

Inner Join returns only the records that have matching values in both tables.

Syntax:

SELECT columns

FROM table1

INNER JOIN table2

ON table1.column = table2.column;

LEFT JOIN

Left Join returns all records from the left table and the matching records from the right table.

Syntax:

SELECT columns

FROM table1

LEFT JOIN table2

ON table1.column = table2.column;

RIGHT JOIN

Right Join returns all records from the right table and the matching records from the left table.

Syntax:

SELECT columns

FROM table1

RIGHT JOIN table2

ON table1.column = table2.column;

FULL JOIN

Full Join returns all records from both tables. It includes matching and non-matching records.

Syntax:

SELECT columns

FROM table1

FULL JOIN table2

ON table1.column = table2.column;

SELF JOIN

Self Join is used to join a table with itself. It is useful when records in the same table are related to each other.

Syntax:

SELECT A.column, B.column

FROM table_name A

JOIN table_name B

ON A.column = B.column;

Explanation

• Create Customers and Orders tables.

• Insert customer and order details into the tables.

• Display matching customer and order records using Inner Join.

• Display all customers and their matching orders using Left Join.

• Display all orders and their matching customers using Right Join.

• Display all matching and non-matching records using Full Join.

• Use Self Join to display employees and their managers.

Create Tables

```
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    customer_name TEXT,
    city TEXT
```

```
);

CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER,
    product TEXT,
    amount INTEGER
);
```

Tables Created

Sample Data

```
INSERT INTO customers VALUES (1, 'Kavin', 'Chennai');
INSERT INTO customers VALUES (2, 'Meena', 'Villupuram');
INSERT INTO customers VALUES (3, 'Arun', 'Salem');
INSERT INTO customers VALUES (4, 'Divya', 'Trichy');
INSERT INTO customers VALUES (5, 'Priya', 'Madurai');

INSERT INTO orders VALUES (101, 1, 'Laptop', 55000);
INSERT INTO orders VALUES (102, 2, 'Mobile', 25000);
INSERT INTO orders VALUES (103, 1, 'Headphone', 3000);
INSERT INTO orders VALUES (104, 3, 'Keyboard', 2000);
INSERT INTO orders VALUES (105, 6, 'Mouse', 1000);
```

Records Inserted

SQL Query

Query 1: Inner Join

```
SQL> SELECT customers.customer_name, orders.product, orders.amount
FROM customers
INNER JOIN orders
ON customers.customer_id = orders.customer_id;
```

Resultant Query

customer_name	product	amount
Kavin	Laptop	55000
Meena	Mobile	25000
Kavin	Headphone	3000
Arun	Keyboard	2000

Query 2: Left Join

```
SQL> SELECT customers.customer_name, orders.product, orders.amount
FROM customers
LEFT JOIN orders
ON customers.customer_id = orders.customer_id;
```

Resultant Query

customer_name	**product**	**amount**
Kavin	Laptop	55000

Meena	Mobile	25000
Arun	Keyboard	2000
Divya	NULL	NULL
Priya	NULL	NULL
Kavin	Headphone	3000

Query 3: Right Join

SQL> SELECT customers.customer_name, orders.product, orders.amount

FROM customers

RIGHT JOIN orders

ON customers.customer_id = orders.customer_id;

Resultant Query

customer_name	**product**	**amount**
Kavin	Laptop	55000
Meena	Mobile	25000
Kavin	Headphone	3000
Arun	Keyboard	2000
NULL	Mouse	1000

Query 4: Full Join

SQL> SELECT customers.customer_name, orders.product, orders.amount

FROM customers

FULL JOIN orders

ON customers.customer_id = orders.customer_id;

Resultant Query

customer_name	product	amount
Kavin	Laptop	55000
Meena	Mobile	25000
Kavin	Headphone	3000
Arun	Keyboard	2000
Divya	NULL	NULL
Priya	NULL	NULL
NULL	Mouse	1000

Query 5: Self Join

Create Employee Table

```
CREATE TABLE employees (
    employee_id INTEGER PRIMARY KEY,
    employee_name TEXT,
    manager_id INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO employees VALUES (1, 'Kavin', NULL);
INSERT INTO employees VALUES (2, 'Meena', 1);
INSERT INTO employees VALUES (3, 'Arun', 1);
```

INSERT INTO employees VALUES (4, 'Divya', 2);

INSERT INTO employees VALUES (5, 'Priya', 2);

Records Inserted

SQL> SELECT A.employee_name AS Employee,

B.employee_name AS Manager

FROM employees A

LEFT JOIN employees B

ON A.manager_id = B.employee_id;

Resultant Query

Employee	Manager
Kavin	NULL
Meena	Kavin
Arun	Kavin
Divya	Meena
Priya	Meena

RESULT:

Thus, the Customers and Orders tables were created successfully, and the required records were retrieved successfully using Inner Join, Left Join, Right Join, Full Join, and Self Join operations.

EX. NO:06 Creation of a SQL functions using Product Table

DATE:29.06.2026

Aim

To create a Products table, insert product records, and retrieve data using different SQL functions such as Aggregate Functions, String Functions, and Date Functions.

Software Used

Oracle / MySQL / sqlite3

Description

SQL Functions are built-in functions used to perform calculations, manipulate text, and work with date values in a database.

SQL FUNCTION TYPES:

- Aggregate Functions
- String Functions
- Date Functions

AGGREGATE FUNCTIONS

Aggregate functions are used to perform calculations on a group of rows and return a single result.

Common aggregate functions are:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

Syntax

SELECT aggregate_function(column_name)

FROM table_name;

STRING FUNCTIONS

String functions are used to perform operations on text or character values.

Common string functions are:

- UPPER()
- LOWER()
- LENGTH()

Syntax

SELECT string_function(column_name)

FROM table_name;

DATE FUNCTIONS

Date functions are used to perform operations on date values.

Common date functions are:

- CURRENT_DATE
- YEAR()
- MONTH()
- DAY()

Syntax

SELECT date_function(date_column)

FROM table_name;

Explanation

- Create a table named products.
- Insert product details into the table.
- Display all product records.
- Calculate the total number of products using COUNT().
- Calculate the total and average price using SUM() and AVG().
- Find the highest and lowest price using MAX() and MIN().
- Display product names in uppercase and lowercase using UPPER() and LOWER().
- Find the length of product names using LENGTH().
- Retrieve product details based on the date.

Create Table

```
CREATE TABLE products (
    product_id INTEGER PRIMARY KEY,
    product_name TEXT,
    category TEXT,
    price INTEGER,
    purchase_date DATE
);
```

Table Created

Sample Data

```
INSERT INTO products VALUES (1, 'Laptop', 'Electronics', 55000, '2026-01-15');
INSERT INTO products VALUES (2, 'Mobile', 'Electronics', 25000, '2026-02-10');
INSERT INTO products VALUES (3, 'Keyboard', 'Accessories', 2000, '2026-03-05');
```

INSERT INTO products VALUES (4, 'Headphone', 'Accessories', 3000, '2026-04-12');

INSERT INTO products VALUES (5, 'Monitor', 'Electronics', 15000, '2026-05-20');

INSERT INTO products VALUES (6, 'Mouse', 'Accessories', 1000, '2026-06-18');

Records Inserted

SQL Query

Query 1: Display all product records

SQL> SELECT * FROM products;

Resultant Query

product_id	product_name	category	price	purchase_date
1	Laptop	Electronics	55000	2026-01-15
2	Mobile	Electronics	25000	2026-02-10
3	Keyboard	Accessories	2000	2026-03-05
4	Headphone	Accessories	3000	2026-04-12
5	Monitor	Electronics	15000	2026-05-20
6	Mouse	Accessories	1000	2026-06-18

AGGREGATE FUNCTIONS

Query 2: Count the total number of products

SQL> SELECT COUNT(*) AS Total_Products

FROM products;

Resultant Query

Total_Products
6

Query 3: Calculate the total price of all products

```
SQL> SELECT SUM(price) AS Total_Price
FROM products;
```

Resultant Query

Total_Price
101000

Query 4: Calculate the average price of products

```
SQL> SELECT AVG(price) AS Average_Price
FROM products;
```

Resultant Query

Average_Price
16833.33

Query 5: Find the highest and lowest product price

```
SQL> SELECT MAX(price) AS Highest_Price,
MIN(price) AS Lowest_Price
FROM products;
```

Resultant Query

Highest_Price	Lowest_Price
55000	1000

STRING FUNCTIONS

Query 6: Display product names in uppercase

SQL> SELECT product_name, UPPER(product_name) AS Upper_Name

FROM products;

Resultant Query

product_name	Upper_Name
Laptop	LAPTOP
Mobile	MOBILE
Keyboard	KEYBOARD
Headphone	HEADPHONE
Monitor	MONITOR
Mouse	MOUSE

Query 7: Display product names in lowercase

SQL> SELECT product_name, LOWER(product_name) AS Lower_Name

FROM products;

Resultant Query

product_name	Lower_Name
Laptop	laptop
Mobile	mobile

Keyboard	keyboard
Headphone	headphone
Monitor	monitor
Mouse	mouse

Query 8: Display the length of each product name

SQL> SELECT product_name, LENGTH(product_name) AS Name_Length

FROM products;

Resultant Query

product_name	Name_Length
Laptop	6
Mobile	6
Keyboard	8
Headphone	9
Monitor	7
Mouse	5

DATE FUNCTIONS

Query 9: Display the current date

SQL> SELECT CURRENT_DATE AS Current_Date;

Resultant Query

Current_Date
Current system date

Query 10: Display products purchased after March 2026

SQL> SELECT product_name, purchase_date

FROM products

WHERE purchase_date > '2026-03-31';

Resultant Query

product_name	purchase_date
Headphone	2026-04-12
Monitor	2026-05-20
Mouse	2026-06-18

RESULT:

Thus, the Products table was created successfully, product records were inserted, and the required data was retrieved successfully using Aggregate Functions, String Functions, and Date Functions.

EX. NO:07 **Creation of Subqueries and Nested Queries using**

DATE:29.06.2026 **Employees**

Aim

To create an Employees table, insert employee records, and retrieve data using Subqueries and Nested Queries.

Software Used

Oracle / MySQL / sqlite3

Description

A Subquery is a query written inside another SQL query. It is used to retrieve intermediate results and use them in the main query.

A Nested Query is a query placed inside another query to perform operations based on the result of the inner query.

SUBQUERY TYPES:

• Single Row Subquery

• Multiple Row Subquery

• Nested Subquery

SINGLE ROW SUBQUERY

A Single Row Subquery returns only one value or one row and is used with comparison operators such as =, >, <, >= and <=.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name > (SELECT column_name
                     FROM table_name
                     WHERE condition);
```

MULTIPLE ROW SUBQUERY

A Multiple Row Subquery returns more than one value or row. It is commonly used with IN, ANY, and ALL operators.

Syntax:

```
SELECT column_name

FROM table_name

WHERE column_name IN (SELECT column_name

                      FROM table_name

                      WHERE condition);
```

NESTED SUBQUERY

A Nested Subquery contains one query inside another query. The result of the inner query is used by the outer query.

Syntax:

```
SELECT column_name

FROM table_name

WHERE column_name = (SELECT column_name

                     FROM table_name

                     WHERE condition);
```

Explanation

• Create a table named employees.

• Insert employee details into the table.

• Display all employee records.

• Find employees whose salary is greater than the average salary.

• Find employees who belong to the IT or HR departments.

• Find the employee who has the highest salary.

• Retrieve employees based on the result of another query.

Create Table

```
CREATE TABLE employees (
    employee_id INTEGER PRIMARY KEY,
    employee_name TEXT,
    department TEXT,
    salary INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO employees VALUES (1, 'Kavin', 'IT', 45000);
INSERT INTO employees VALUES (2, 'Meena', 'HR', 32000);
INSERT INTO employees VALUES (3, 'Arun', 'Finance', 38000);
INSERT INTO employees VALUES (4, 'Divya', 'IT', 50000);
INSERT INTO employees VALUES (5, 'Rahul', 'Sales', 28000);
INSERT INTO employees VALUES (6, 'Priya', 'HR', 42000);
```

Records Inserted

SQL Query

Query 1: Display all employee records

SQL> SELECT * FROM employees;

Resultant Query

employee_id	employee_name	department	salary
1	Kavin	IT	45000
2	Meena	HR	32000
3	Arun	Finance	38000
4	Divya	IT	50000
5	Rahul	Sales	28000
6	Priya	HR	42000

Query 2: Display employees whose salary is greater than the average salary

SQL> SELECT employee_name, salary

FROM employees

WHERE salary > (SELECT AVG(salary)

FROM employees);

Resultant Query

employee_name	salary
Kavin	45000
Divya	50000
Priya	42000

Query 3: Display employees who belong to IT or HR department

```
SQL> SELECT employee_name, department
FROM employees
WHERE department IN (SELECT department
        FROM employees
        WHERE department IN ('IT', 'HR'));
```

Resultant Query

employee_name	department
Kavin	IT
Meena	HR
Divya	IT
Priya	HR

Query 4: Display the employee with the highest salary

```
SQL> SELECT employee_name, salary
FROM employees
WHERE salary = (SELECT MAX(salary)
        FROM employees);
```

Resultant Query

employee_name	salary
Divya	50000

Query 5: Display employees whose salary is greater than Meena's salary

```
SQL> SELECT employee_name, salary
FROM employees
WHERE salary > (SELECT salary
        FROM employees
        WHERE employee_name = 'Meena');
```

Resultant Query

employee_name	salary
Kavin	45000
Arun	38000
Divya	50000
Priya	42000

RESULT:

Thus, the Employees table was created successfully, employee records were inserted, and the required employee details were retrieved successfully using Subqueries and Nested Queries.

EX. NO:08 **Creation of views, indexes and sequences**

DATE:01.07.2026

Aim

To create and perform operations on Views, Indexes, and Sequences using SQL commands.

Software Used

Oracle SQL / SQL Command Line

Description

View:
A View is a virtual table created from one or more tables using a SELECT query. It does not store the actual data separately.

Index:
An Index is used to improve the speed of data retrieval from a table.

Sequence:
A Sequence is a database object used to generate unique numeric values automatically.

1. VIEW

Definition

A View is a virtual table based on the result of a SQL query.

Syntax

CREATE VIEW view_name AS

SELECT column1, column2

FROM table_name

WHERE condition;

Create Table

CREATE TABLE employee_details (

```
    employee_id NUMBER PRIMARY KEY,
    employee_name VARCHAR2(50),
    department VARCHAR2(30),
    salary NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO employee_details VALUES (101, 'Kavin', 'IT', 45000);
INSERT INTO employee_details VALUES (102, 'Meena', 'HR', 32000);
INSERT INTO employee_details VALUES (103, 'Arun', 'Finance', 38000);
INSERT INTO employee_details VALUES (104, 'Divya', 'IT', 50000);
INSERT INTO employee_details VALUES (105, 'Priya', 'HR', 42000);
COMMIT;
```

Records Inserted

Create View

```
CREATE VIEW it_employees AS
SELECT employee_id, employee_name, salary
FROM employee_details
WHERE department = 'IT';
```

View Created

Display View

```
SELECT * FROM it_employees;
```

Resultant Query

employee_id	employee_name	salary
101	Kavin	45000
104	Divya	50000

Explanation

The it_employees view displays only the employees who belong to the IT department.

2. INDEX

Definition

An Index is a database object that improves the speed of searching and retrieving records from a table.

Syntax

CREATE INDEX index_name

ON table_name(column_name);

Create Index

CREATE INDEX idx_employee_name

ON employee_details(employee_name);

Index Created

Use Index in Search

SELECT *

FROM employee_details

WHERE employee_name = 'Kavin';

Resultant Query

employee_id	employee_name	department	salary
101	Kavin	IT	45000

Explanation

The index idx_employee_name is created on the employee_name column. It helps improve the performance of searches based on employee names.

3. SEQUENCE

Definition

A Sequence is a database object that automatically generates a series of unique numeric values.

Syntax

CREATE SEQUENCE sequence_name

START WITH initial_value

INCREMENT BY increment_value;

Create Sequence

CREATE SEQUENCE employee_seq

START WITH 106

INCREMENT BY 1;

Sequence Created

Insert Record Using Sequence

INSERT INTO employee_details

VALUES (employee_seq.NEXTVAL, 'Rahul', 'Sales', 28000);

COMMIT;

Record Inserted

Display Sequence Value

SELECT employee_seq.CURRVAL AS current_sequence

FROM dual;

Resultant Query

current_sequence
106

Insert Another Record Using Sequence

INSERT INTO employee_details

VALUES (employee_seq.NEXTVAL, 'Anitha', 'Marketing', 35000);

COMMIT;

Record Inserted

Display All Employee Records

SELECT * FROM employee_details

ORDER BY employee_id;

Resultant Query

employee_id	employee_name	department	salary

101	Kavin	IT	45000
102	Meena	HR	32000
103	Arun	Finance	38000
104	Divya	IT	50000
105	Priya	HR	42000
106	Rahul	Sales	28000
107	Anitha	Marketing	35000

Important Commands

Drop View

DROP VIEW it_employees;

View Dropped

Drop Index

DROP INDEX idx_employee_name;

Index Dropped

Drop Sequence

DROP SEQUENCE employee_seq;

Sequence Dropped

Result

Thus, the View, Index, and Sequence were created and successfully operated using SQL commands. The View was used to display selected records, the Index was used to improve data retrieval, and the Sequence was used to generate unique numeric values automatically.

EX. NO:09 Creation of stored procedures and functions

DATE:08.07.2026

Aim

To create and execute Stored Procedures and Functions using PL/SQL.

Software Used

Oracle SQL / SQL Command Line

Description

Stored Procedure:
A Stored Procedure is a named PL/SQL program stored in the database. It is used to perform a specific task and may or may not return a value.

Function:
A Function is a named PL/SQL program that performs a specific operation and always returns a single value.

1. STORED PROCEDURE

Definition

A Stored Procedure is a stored PL/SQL block that performs a particular operation. It can accept input parameters and can modify or display data.

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name

(parameter_name IN datatype)

IS

BEGIN

   statements;

END;/
```

Create Table

```
CREATE TABLE student_marks (
   student_id NUMBER PRIMARY KEY,
   student_name VARCHAR2(50),
   mark NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO student_marks VALUES (101, 'Kavin', 85);
INSERT INTO student_marks VALUES (102, 'Meena', 72);
INSERT INTO student_marks VALUES (103, 'Arun', 91);
INSERT INTO student_marks VALUES (104, 'Divya', 68);
INSERT INTO student_marks VALUES (105, 'Priya', 95);
COMMIT;
```

Records Inserted

Display Student Records

```
SELECT * FROM student_marks;
```

Resultant Query

student_id	student_name	mark
101	Kavin	85
102	Meena	72

103	Arun	91
104	Divya	68
105	Priya	95

Create Stored Procedure

The following procedure displays the details of a student based on the student ID.

```
CREATE OR REPLACE PROCEDURE display_student (
    p_student_id IN NUMBER
)
IS
    v_name student_marks.student_name%TYPE;
    v_mark student_marks.mark%TYPE;
BEGIN
    SELECT student_name, mark
    INTO v_name, v_mark
    FROM student_marks
    WHERE student_id = p_student_id;

    DBMS_OUTPUT.PUT_LINE('Student Name: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Mark: ' || v_mark);
END;
/
```

Procedure Created

Execute Stored Procedure

SET SERVEROUTPUT ON;

EXEC display_student(103);

Resultant Query

Student Name	Mark
Arun	91

Output

Student Name: Arun

Mark: 91

Explanation

The procedure display_student accepts the student ID as an input parameter and retrieves the corresponding student name and mark from the student_marks table.

2. FUNCTION

Definition

A Function is a stored PL/SQL program that performs a specific operation and returns a single value.

Syntax

CREATE OR REPLACE FUNCTION function_name

(parameter_name IN datatype)

RETURN datatype

```
IS
BEGIN
   statements;
   RETURN value;
END;
/
```

Create Function

The following function returns the grade based on the student's mark.

```
CREATE OR REPLACE FUNCTION get_grade (
   p_mark IN NUMBER
)
RETURN VARCHAR2
IS
   v_grade VARCHAR2(5);
BEGIN
   IF p_mark >= 90 THEN
      v_grade := 'A+';
   ELSIF p_mark >= 80 THEN
      v_grade := 'A';
   ELSIF p_mark >= 70 THEN
      v_grade := 'B';
   ELSIF p_mark >= 60 THEN
      v_grade := 'C';
```

```
    ELSIF p_mark >= 50 THEN
        v_grade := 'D';
    ELSE
        v_grade := 'F';
    END IF;

    RETURN v_grade;
END;
/
```

Function Created

Execute Function

```
SELECT student_id, student_name, mark,
       get_grade(mark) AS grade
FROM student_marks;
```

Resultant Query

student_id	student_name	mark	grade
101	Kavin	85	A
102	Meena	72	B
103	Arun	91	A+
104	Divya	68	C
105	Priya	95	A+

Explanation

The function get_grade accepts the student's mark as an input and returns the corresponding grade. The function is called inside the SELECT statement for each student.

Important Commands

Display Stored Procedure

SELECT object_name, object_type

FROM user_objects

WHERE object_name = 'DISPLAY_STUDENT';

Resultant Query

object_name	object_type
DISPLAY_STUDENT	PROCEDURE

Display Stored Function

SELECT object_name, object_type

FROM user_objects

WHERE object_name = 'GET_GRADE';

Resultant Query

object_name	object_type
GET_GRADE	FUNCTION

Drop Procedure

DROP PROCEDURE display_student;

Procedure Dropped

Drop Function

DROP FUNCTION get_grade;

Function Dropped

Result

Thus, the Stored Procedure and Function were created and executed successfully using PL/SQL. The Stored Procedure was used to retrieve student details, and the Function was used to calculate the grade based on the student's mark.

EX. NO:10 Creation of Triggers method using product stock table

DATE:12.07.2026

Aim

To create and execute Triggers using PL/SQL to automatically perform an action when an INSERT, UPDATE, or DELETE operation occurs on a table.

Software Used

Oracle SQL / SQL Command Line

Description

A Trigger is a stored PL/SQL program that is automatically executed when a specified event occurs on a table or database.

Triggers can be used to automatically perform operations such as inserting, updating, or maintaining records when a particular event takes place.

1. CREATE TABLE

```
CREATE TABLE product_stock (
   product_id NUMBER PRIMARY KEY,
   product_name VARCHAR2(50),
   quantity NUMBER,
   price NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO product_stock VALUES (201, 'Laptop', 10, 55000);
INSERT INTO product_stock VALUES (202, 'Mobile', 20, 25000);
INSERT INTO product_stock VALUES (203, 'Keyboard', 15, 2000);
```

INSERT INTO product_stock VALUES (204, 'Headphone', 12, 3000);

INSERT INTO product_stock VALUES (205, 'Mouse', 25, 1000);

COMMIT;

Records Inserted

Display Product Records

SELECT * FROM product_stock;

Resultant Query

product_id	product_name	quantity	price
201	Laptop	10	55000
202	Mobile	20	25000
203	Keyboard	15	2000
204	Headphone	12	3000
205	Mouse	25	1000

2. BEFORE INSERT TRIGGER

Definition

A BEFORE INSERT Trigger is executed automatically before a new record is inserted into a table.

Syntax

CREATE OR REPLACE TRIGGER trigger_name

BEFORE INSERT ON table_name

FOR EACH ROW

BEGIN

statements;

```
END;
/
```

Create Trigger

The following trigger checks whether the quantity is negative before inserting a product record.

```
CREATE OR REPLACE TRIGGER check_product_quantity
BEFORE INSERT ON product_stock
FOR EACH ROW
BEGIN
   IF :NEW.quantity < 0 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Quantity cannot be negative');
   END IF;
END;
/
```

Trigger Created

Execute Trigger

```
INSERT INTO product_stock
VALUES (206, 'Printer', -5, 12000);
```

Output

ORA-20001: Quantity cannot be negative

The trigger prevents the insertion of an invalid quantity.

3. BEFORE UPDATE TRIGGER

Definition

A BEFORE UPDATE Trigger is executed automatically before an existing record is updated.

Create Trigger

The following trigger prevents the product price from being updated to a negative value.

```
CREATE OR REPLACE TRIGGER check_product_price
BEFORE UPDATE OF price ON product_stock
FOR EACH ROW
BEGIN
   IF :NEW.price < 0 THEN
      RAISE_APPLICATION_ERROR(-20002, 'Price cannot be negative');
   END IF;
END;
/
```

Trigger Created

Execute Trigger

```
UPDATE product_stock
SET price = -500
WHERE product_id = 203;
```

Output

ORA-20002: Price cannot be negative

The trigger prevents the invalid price from being updated.

4. AFTER INSERT TRIGGER

Definition

An AFTER INSERT Trigger is executed automatically after a new record is successfully inserted into a table.

Create Audit Table

```
CREATE TABLE product_audit (
   audit_id NUMBER,
   product_id NUMBER,
   action_type VARCHAR2(20)
);
```

Table Created

Create Trigger

```
CREATE OR REPLACE TRIGGER product_insert_audit
AFTER INSERT ON product_stock
FOR EACH ROW
BEGIN
   INSERT INTO product_audit
   VALUES (product_audit_seq.NEXTVAL, :NEW.product_id, 'INSERT');
END;
/
```

Trigger Created

Before executing the above trigger, create the required sequence:

```
CREATE SEQUENCE product_audit_seq
START WITH 1
INCREMENT BY 1;
```

Sequence Created

Insert New Product

INSERT INTO product_stock

VALUES (206, 'Printer', 8, 12000);

COMMIT;

Record Inserted

Display Audit Records

SELECT * FROM product_audit;

Resultant Query

audit_id	product_id	action_type
1	206	INSERT

5. DISPLAY FINAL PRODUCT RECORDS

SELECT * FROM product_stock

ORDER BY product_id;

Resultant Query

product_id	product_name	quantity	price
201	Laptop	10	55000
202	Mobile	20	25000
203	Keyboard	15	2000

204	Headphone	12	3000
205	Mouse	25	1000
206	Printer	8	12000

Important Commands

Display Triggers

SELECT trigger_name, status

FROM user_triggers;

Resultant Query

trigger_name	status
CHECK_PRODUCT_QUANTITY	ENABLED
CHECK_PRODUCT_PRICE	ENABLED
PRODUCT_INSERT_AUDIT	ENABLED

Drop Trigger

DROP TRIGGER check_product_quantity;

Trigger Dropped

DROP TRIGGER check_product_price;

Trigger Dropped

DROP TRIGGER product_insert_audit;

Trigger Dropped

Result

Thus, the Triggers were created and executed successfully using PL/SQL. The triggers automatically validated product quantity and price before modification and recorded product insertion details in the audit table.

EX. NO:11 Creation of cursor implementation using book table

DATE:20.07.2026

Aim

To implement and execute a Cursor using PL/SQL to retrieve and process records from a database table.

Software Used

Oracle SQL / SQL Command Line

Description

A Cursor is a pointer used to process the rows returned by a SQL query one row at a time.

Cursors are useful when each record needs to be processed individually.

There are two types of cursors:

1. Implicit Cursor
2. Explicit Cursor

In this practical, an Explicit Cursor is implemented.

1. CREATE TABLE

```
CREATE TABLE book_details (
   book_id NUMBER PRIMARY KEY,
   book_name VARCHAR2(50),
   author_name VARCHAR2(50),
   price NUMBER
);
```

Table Created

Insert Records

INSERT INTO book_details VALUES (301, 'Java Programming', 'James', 550);

INSERT INTO book_details VALUES (302, 'Database Systems', 'Korth', 650);

INSERT INTO book_details VALUES (303, 'Python Basics', 'Guido', 450);

INSERT INTO book_details VALUES (304, 'Web Development', 'Thomas', 700);

INSERT INTO book_details VALUES (305, 'Data Structures', 'Robert', 600);

COMMIT;

Records Inserted

Display Book Records

SELECT * FROM book_details;

Resultant Query

book_id	book_name	author_name	price
301	Java Programming	James	550
302	Database Systems	Korth	650
303	Python Basics	Guido	450
304	Web Development	Thomas	700
305	Data Structures	Robert	600

2. EXPLICIT CURSOR

Definition

An Explicit Cursor is a cursor that is declared and controlled by the programmer. It is used to retrieve multiple rows from a table and process them one by one.

Syntax

```
DECLARE
   CURSOR cursor_name IS
      SELECT statement;
BEGIN
   OPEN cursor_name;

   LOOP
      FETCH cursor_name INTO variables;

      EXIT WHEN cursor_name%NOTFOUND;

      statements;
   END LOOP;

   CLOSE cursor_name;
END;
/
```

Create Explicit Cursor

The following cursor retrieves each book name and price from the book_details table.

```
SET SERVEROUTPUT ON;

DECLARE
   CURSOR book_cursor IS
      SELECT book_name, price
```

```
    FROM book_details;

  v_book_name book_details.book_name%TYPE;
  v_price book_details.price%TYPE;
BEGIN
  OPEN book_cursor;

  LOOP
    FETCH book_cursor INTO v_book_name, v_price;

    EXIT WHEN book_cursor%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(
      'Book: ' || v_book_name || ' | Price: ₹' || v_price
    );
  END LOOP;

  CLOSE book_cursor;
END;
/
```

Output

Book: Java Programming | Price: ₹550

Book: Database Systems | Price: ₹650

Book: Python Basics | Price: ₹450

Book: Web Development | Price: ₹700

Book: Data Structures | Price: ₹600

3. CURSOR WITH CONDITION

The following cursor displays only the books whose price is greater than ₹500.

```
SET SERVEROUTPUT ON;

DECLARE
   CURSOR book_cursor IS
      SELECT book_id, book_name, price
      FROM book_details
      WHERE price > 500;

   v_book_id book_details.book_id%TYPE;
   v_book_name book_details.book_name%TYPE;
   v_price book_details.price%TYPE;
BEGIN
   OPEN book_cursor;

   LOOP
      FETCH book_cursor INTO v_book_id, v_book_name, v_price;

      EXIT WHEN book_cursor%NOTFOUND;
```

```
        DBMS_OUTPUT.PUT_LINE(
            'Book ID: ' || v_book_id ||
            ' | Book: ' || v_book_name ||
            ' | Price: ₹' || v_price
        );
    END LOOP;
    CLOSE book_cursor;
END;
/
```

Resultant Query

book_id	book_name	price
301	Java Programming	550
302	Database Systems	650
304	Web Development	700
305	Data Structures	600

4. CURSOR ATTRIBUTES

Cursor attributes provide information about the current status of a cursor.

Important Cursor Attributes

Cursor Attribute	Description
%FOUND	Returns TRUE if the last fetch returned a row
%NOTFOUND	Returns TRUE if the last fetch did not return a row
%ROWCOUNT	Returns the number of rows processed
%ISOPEN	Returns TRUE if the cursor is currently open

5. CURSOR USING %ROWCOUNT

The following program displays the total number of books processed by the cursor.

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR book_cursor IS
      SELECT book_name, price
      FROM book_details;

   v_book_name book_details.book_name%TYPE;
   v_price book_details.price%TYPE;
BEGIN
   OPEN book_cursor;

   LOOP
      FETCH book_cursor INTO v_book_name, v_price;

      EXIT WHEN book_cursor%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE(
         'Book: ' || v_book_name ||
         ' | Price: ₹' || v_price
      );
   END LOOP;
```

```
    DBMS_OUTPUT.PUT_LINE(
        'Total Books Processed: ' || book_cursor%ROWCOUNT
    );


    CLOSE book_cursor;
END;
/
```

Output

```
Book: Java Programming | Price: ₹550
Book: Database Systems | Price: ₹650
Book: Python Basics | Price: ₹450
Book: Web Development | Price: ₹700
Book: Data Structures | Price: ₹600
Total Books Processed: 5
```

Result

Thus, the Cursor was successfully implemented using PL/SQL. The explicit cursor was used to retrieve and process book records one by one, apply conditions, and count the number of processed records.

EX. NO:12 **Creation of PL/SQL Exception Handling using bank**

DATE:26.07.2026 **account table**

Aim

To implement Exception Handling in PL/SQL and handle runtime errors using predefined and user-defined exceptions.

Software Used

Oracle SQL / SQL Command Line

Description

An Exception is an error that occurs during the execution of a PL/SQL program.

Exception Handling is used to handle runtime errors without terminating the program abnormally.

The basic structure of exception handling is:

```
DECLARE
   declarations;
BEGIN
   statements;
EXCEPTION
   WHEN exception_name THEN
      statements;
END;
/
```

Types of Exceptions

Type	Description
Predefined Exception	Built-in exceptions provided by Oracle

User-Defined Exception	Exception created by the programmer

1. CREATE TABLE

```
CREATE TABLE bank_accounts (
   account_id NUMBER PRIMARY KEY,
   account_holder VARCHAR2(50),
   balance NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO bank_accounts VALUES (401, 'Kavin', 25000);
INSERT INTO bank_accounts VALUES (402, 'Meena', 30000);
INSERT INTO bank_accounts VALUES (403, 'Arun', 20000);
INSERT INTO bank_accounts VALUES (404, 'Divya', 35000);
INSERT INTO bank_accounts VALUES (405, 'Priya', 40000);
COMMIT;
```

Records Inserted

Display Account Records

```
SELECT * FROM bank_accounts;
```

Resultant Query

account_id	account_holder	balance
401	Kavin	25000
402	Meena	30000
403	Arun	20000
404	Divya	35000
405	Priya	40000

2. NO_DATA_FOUND EXCEPTION

Definition

NO_DATA_FOUND occurs when a SELECT INTO statement does not return any record.

Program

```
SET SERVEROUTPUT ON;
DECLARE
   v_name bank_accounts.account_holder%TYPE;
   v_balance bank_accounts.balance%TYPE;
BEGIN
   SELECT account_holder, balance
   INTO v_name, v_balance
   FROM bank_accounts
   WHERE account_id = 499;

   DBMS_OUTPUT.PUT_LINE('Account Holder: ' || v_name);
   DBMS_OUTPUT.PUT_LINE('Balance: ₹' || v_balance);
```

```
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Exception: Account not found.');
END;
/
```

Output

Exception: Account not found.

Exception Handled Successfully

3. TOO_MANY_ROWS EXCEPTION

Definition

TOO_MANY_ROWS occurs when a SELECT INTO statement returns more than one record.

Program

```
SET SERVEROUTPUT ON;
DECLARE
   v_name bank_accounts.account_holder%TYPE;
BEGIN
   SELECT account_holder
   INTO v_name
   FROM bank_accounts
   WHERE balance > 20000;

   DBMS_OUTPUT.PUT_LINE('Account Holder: ' || v_name);
```

```
EXCEPTION
   WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE(
         'Exception: More than one account found.'
      );
END;
/
```

Output

Exception: More than one account found.

Exception Handled Successfully

4. ZERO_DIVIDE EXCEPTION

Definition

ZERO_DIVIDE occurs when a number is divided by zero.

Program

```
SET SERVEROUTPUT ON;
DECLARE
   v_balance NUMBER := 25000;
   v_divisor NUMBER := 0;
   v_result NUMBER;
BEGIN
   v_result := v_balance / v_divisor;

   DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);
```

```
EXCEPTION
   WHEN ZERO_DIVIDE THEN
      DBMS_OUTPUT.PUT_LINE(
         'Exception: Cannot divide by zero.'
      );
END;
/
```

Output

Exception: Cannot divide by zero.

Exception Handled Successfully

5. USER-DEFINED EXCEPTION

Definition

A User-Defined Exception is an exception created by the programmer to handle a specific business condition.

In this example, withdrawal is allowed only when sufficient balance is available.

Program

```
SET SERVEROUTPUT ON;
DECLARE
   v_balance bank_accounts.balance%TYPE;
   v_withdraw NUMBER := 50000;
   insufficient_balance EXCEPTION;
BEGIN
```

```
    SELECT balance
    INTO v_balance
    FROM bank_accounts
    WHERE account_id = 401;

    IF v_withdraw > v_balance THEN
        RAISE insufficient_balance;
    END IF;

    UPDATE bank_accounts
    SET balance = balance - v_withdraw
    WHERE account_id = 401;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Withdrawal  Successful.');

EXCEPTION
    WHEN insufficient_balance THEN
        DBMS_OUTPUT.PUT_LINE(
            'Exception: Insufficient balance.'
        );
END;
/
```

Output

Exception: Insufficient balance.

User-Defined Exception Handled Successfully

6. EXCEPTION HANDLING WITH SUCCESSFUL TRANSACTION

The following program performs a valid withdrawal when sufficient balance is available.

```
SET SERVEROUTPUT ON;
DECLARE
   v_balance bank_accounts.balance%TYPE;
   v_withdraw NUMBER := 5000;
BEGIN
   SELECT balance
   INTO v_balance
   FROM bank_accounts
   WHERE account_id = 401;

   IF v_withdraw > v_balance THEN
      RAISE_APPLICATION_ERROR(
         -20001,
         'Insufficient balance'
      );
   END IF;

   UPDATE bank_accounts
```

```
    SET balance = balance - v_withdraw
    WHERE account_id = 401;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Withdrawal Successful.');
    DBMS_OUTPUT.PUT_LINE(
        'Withdrawn Amount: ₹' || v_withdraw
    );

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );
END;
/
```

Output

Withdrawal Successful.

Withdrawn Amount: ₹5000

Display Updated Account

```
SELECT * FROM bank_accounts
WHERE account_id = 401;
```

Resultant Query

account_id	account_holder	balance
401	Kavin	20000

Important Predefined Exceptions

Exception	Description
NO_DATA_FOUND	No record is found
TOO_MANY_ROWS	More than one record is returned
ZERO_DIVIDE	Division by zero occurs
DUP_VAL_ON_INDEX	Duplicate value is inserted into a unique/primary key column
VALUE_ERROR	Invalid value or conversion error occurs

Result

Thus, PL/SQL Exception Handling was successfully implemented. The NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, and user-defined exceptions were handled successfully using appropriate exception-handling mechanisms.

EX. NO:13 **Creation of Database Normalization using student course table**

DATE: 26.07.2026

Aim

To study and implement Database Normalization up to Boyce-Codd Normal Form (BCNF) to reduce data redundancy and improve data integrity.

Software Used

Oracle SQL / SQL Command Line

Description

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion, and update anomalies.

The major normal forms covered are:

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)
4. Boyce-Codd Normal Form (BCNF)

1. UNNORMALIZED TABLE

Consider the following Student Course table.

Unnormalized Table

The table contains repeated information. For example, the instructor name Kumar is repeated for students taking the Java course.

student_id	student_name	course_id	course_name	instructor_id	instructor_name
501	Kavin	C01	Java	I01	Kumar

502	Meena	C02	DBMS	I02	Priya
503	Arun	C01	Java	I01	Kumar
504	Divya	C03	Python	I03	Ravi

2. FIRST NORMAL FORM (1NF)

Definition

A relation is in 1NF when:

- Each column contains atomic values.
- There are no repeating groups.
- Each row is uniquely identifiable.

The above table already contains atomic values, so it can be considered as the 1NF table.

1NF Table

student_id	student_name	course_id	course_name	instructor_id	instructor_name
501	Kavin	C01	Java	I01	Kumar
502	Meena	C02	DBMS	I02	Priya
503	Arun	C01	Java	I01	Kumar
504	Divya	C03	Python	I03	Ravi

Primary Key: student_id, course_id

3. SECOND NORMAL FORM (2NF)

Definition

A relation is in 2NF when:

1. It is already in 1NF.
2. There is no partial dependency.
3. Every non-key attribute must depend on the complete primary key.

In the 1NF table, some attributes depend only on part of the composite key.

For example:

- student_name depends only on student_id.
- course_name depends only on course_id.
- instructor_id depends on course_id.

Therefore, the table is divided into separate tables.

Student Table

student_id	student_name
501	Kavin
502	Meena
503	Arun
504	Divya

Course Table

course_id	course_name	instructor_id
C01	Java	I01
C02	DBMS	I02
C03	Python	I03

Enrollment Table

student_id	course_id
501	C01

502	C02
503	C01
504	C03

The partial dependencies are removed.

Therefore, the tables are in 2NF.

4. THIRD NORMAL FORM (3NF)

Definition

A relation is in 3NF when:

1. It is already in 2NF.
2. There is no transitive dependency.
3. Non-key attributes must depend only on the primary key.

In the Course table:

course_id → instructor_id

and

instructor_id → instructor_name

Therefore, instructor_name has a transitive dependency on course_id.

To remove this dependency, the instructor information is separated into another table.

Course Table

course_id	course_name	instructor_id
C01	Java	I01

C02	DBMS	I02
C03	Python	I03

Instructor Table

instructor_id	instructor_name
I01	Kumar
I02	Priya
I03	Ravi

Enrollment Table

student_id	course_id
501	C01
502	C02
503	C01
504	C03

Therefore, the tables are in 3NF.

5. BOYCE-CODD NORMAL FORM (BCNF)

Definition

A relation is in BCNF when, for every non-trivial functional dependency:

The determinant must be a candidate key.

BCNF is a stronger version of 3NF.

Example

Consider an Instructor_Course table.

instructor_id	course_id
I01	C01
I01	C02
I02	C03
I03	C04

Assume that each instructor teaches only one course.

Functional dependency:

instructor_id → course_id

Here, instructor_id determines course_id.

If instructor_id is not a candidate key in the given relationship, the relation violates BCNF.

Therefore, the relation can be decomposed into separate tables.

Instructor Table

instructor_id	course_id
I01	C01
I02	C03
I03	C04

Course Table

course_id
C01
C02
C03

C04

After decomposition, the determinant is treated as a key in its corresponding relation.

Therefore, the relation is decomposed to satisfy BCNF.

Result

Thus, the Database Normalization process was studied and implemented up to 1NF, 2NF, 3NF, and BCNF. The normalization process reduces data redundancy, removes dependency-related anomalies, and improves database consistency and integrity.

EX. NO:14 **MINI PROJECT**

DATE:02.10.2026 **INSURANCE POLICY AND CLAIMS PROCESSING**

Aim

To develop an Insurance Policy and Claims Processing system using SQL and PL/SQL to store, manage, process, and analyze insurance claim records.

Software Used

- GitHub Codespaces
- SQLite
- Oracle Database
- SQL
- PL/SQL

Description

The Insurance Policy and Claims Processing system is developed to manage insurance policies and claims. The system stores policy details, customer details, claim dates, requested amounts, approved amounts, and claim status.

SQL queries are used to retrieve and analyze claim information, while PL/SQL procedures and functions are used to register claims, calculate claim differences, update claim status, generate policy-wise reports, and handle exceptions.

Database Tables

1. INSURANCE_POLICIES

Column Name	Data Type	Constraint
POLICY_ID	VARCHAR2(10)	PRIMARY KEY
CUSTOMER_ID	VARCHAR2(10)	NOT NULL
CUSTOMER_NAME	VARCHAR2(50)	NOT NULL
POLICY_TYPE	VARCHAR2(30)	NOT NULL

2. INSURANCE_CLAIMS

Column Name	Data Type	Constraint
CLAIM_ID	NUMBER	PRIMARY KEY
POLICY_ID	VARCHAR2(10)	FOREIGN KEY
CUSTOMER_ID	VARCHAR2(10)	NOT NULL
CLAIM_DATE	DATE	NOT NULL
POLICY_TYPE	VARCHAR2(30)	NOT NULL
REQUESTED_AMOUNT	NUMBER(12,2)	NOT NULL
APPROVED_AMOUNT	NUMBER(12,2)	—
STATUS	VARCHAR2(20)	NOT NULL

TABLE CREATION

TABLE 1

```
CREATE TABLE insurance_policies (
    policy_id VARCHAR2(10) PRIMARY KEY,
    customer_id VARCHAR2(10) NOT NULL,
    customer_name VARCHAR2(50) NOT NULL,
    policy_type VARCHAR2(30) NOT NULL
);
```

TABLE 2

```
CREATE TABLE insurance_claims (
    claim_id INTEGER PRIMARY KEY,
    policy_id TEXT NOT NULL,
    customer_id TEXT NOT NULL,
    claim_date TEXT NOT NULL,
    policy_type TEXT NOT NULL,
    requested_amount REAL NOT NULL,
```

```
    approved_amount REAL,
    status TEXT NOT NULL
);
```

Table Created

DATASET INSERTION

TABLE 1 RECORDS

```
INSERT ALL
    INTO insurance_policies VALUES ('P1001','C001','Kavin','Health')
    INTO insurance_policies VALUES ('P1002','C002','Meena','Motor')
    INTO insurance_policies VALUES ('P1003','C003','Arun','Life')
    INTO insurance_policies VALUES ('P1004','C004','Divya','Travel')
    INTO insurance_policies VALUES ('P1005','C005','Rahul','Property')
    INTO insurance_policies VALUES ('P1006','C006','Priya','Health')
    INTO insurance_policies VALUES ('P1007','C007','Karthik','Motor')
    INTO insurance_policies VALUES ('P1008','C008','Anitha','Life')
    INTO insurance_policies VALUES ('P1009','C009','Suresh','Travel')
    INTO insurance_policies VALUES ('P1010','C010','Nisha','Property')
```

TABLE 2 RECORDS

```
INSERT INTO insurance_claims VALUES (1001,'P1001','C001',DATE '2026-01-
05','Health',85000,80000,'Approved')

INSERT INTO insurance_claims VALUES (1002,'P1002','C002',DATE '2026-01-
08','Motor',125000,110000,'Approved')
```

INSERT INTO insurance_claims VALUES (1003,'P1003','C003',DATE '2026-01-12','Life',200000,NULL,'Pending')

INSERT INTO insurance_claims VALUES (1004,'P1004','C004',DATE '2026-01-15','Travel',45000,40000,'Approved')

INSERT INTO insurance_claims VALUES (1005,'P1005','C005',DATE '2026-01-18','Property',175000,NULL,'Pending')

INSERT INTO insurance_claims VALUES (1006,'P1006','C006',DATE '2026-01-22','Health',95000,90000,'Approved')

INSERT INTO insurance_claims VALUES (1007,'P1007','C007',DATE '2026-01-25','Motor',65000,60000,'Approved')

INSERT INTO insurance_claims VALUES (1008,'P1008','C008',DATE '2026-01-28','Life',300000,NULL,'Rejected')

INSERT INTO insurance_claims VALUES (1009,'P1009','C009',DATE '2026-02-02','Travel',55000,50000,'Approved')

INSERT INTO insurance_claims VALUES (1010,'P1010','C010',DATE '2026-02-05','Property',220000,200000,'Approved')

INSERT INTO insurance_claims VALUES (1011,'P1001','C001',DATE '2026-02-10','Health',120000,NULL,'Pending')

INSERT INTO insurance_claims VALUES (1012,'P1002','C002',DATE '2026-02-14','Motor',145000,130000,'Approved')

INSERT INTO insurance_claims VALUES (1013,'P1003','C003',DATE '2026-02-18','Life',180000,NULL,'Rejected')

INSERT INTO insurance_claims VALUES (1014,'P1004','C004',DATE '2026-02-20','Travel',35000,30000,'Approved')

INSERT INTO insurance_claims VALUES (1015,'P1005','C005',DATE '2026-02-23','Property',150000,NULL,'Pending')

INSERT INTO insurance_claims VALUES (1016,'P1006','C006',DATE '2026-03-01','Health',70000,65000,'Approved')

INSERT INTO insurance_claims VALUES (1017,'P1007','C007',DATE '2026-03-05','Motor',190000,175000,'Approved')

```
INSERT INTO insurance_claims VALUES (1018,'P1008','C008',DATE '2026-03-09','Life',250000,NULL,'Pending')

INSERT INTO insurance_claims VALUES (1019,'P1009','C009',DATE '2026-03-12','Travel',60000,55000,'Approved')

INSERT INTO insurance_claims VALUES (1020,'P1010','C010',DATE '2026-03-15','Property',300000,NULL,'Pending')

INSERT INTO insurance_claims VALUES (1021,'P1001','C001',DATE '2026-03-18','Health',50000,45000,'Approved')

INSERT INTO insurance_claims VALUES (1022,'P1002','C002',DATE '2026-03-22','Motor',210000,NULL,'Pending')

INSERT INTO insurance_claims VALUES (1023,'P1003','C003',DATE '2026-03-25','Life',275000,250000,'Approved')

INSERT INTO insurance_claims VALUES (1024,'P1004','C004',DATE '2026-03-28','Travel',40000,NULL,'Rejected')

INSERT INTO insurance_claims VALUES (1025,'P1005','C005',DATE '2026-03-30','Property',125000,115000,'Approved')

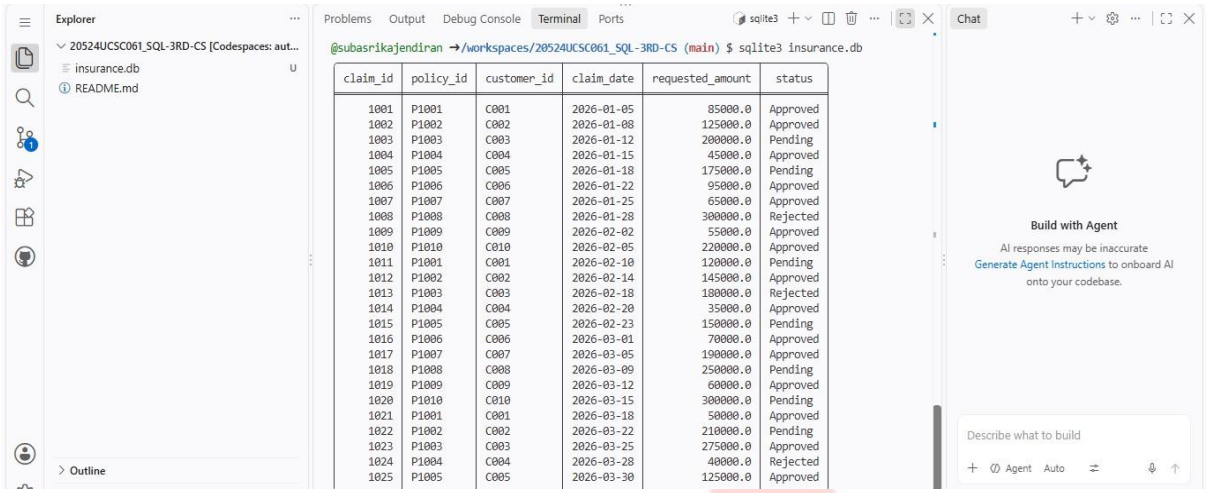
SELECT 1 FROM dual;
```

Records Inserted

SQL Query 1: Display Insurance Claim Records

```
SELECT claim_id, policy_id, customer_id, claim_date,
    requested_amount, status
FROM insurance_claims
ORDER BY claim_id;
```

Output:



claim_id	policy_id	customer_id	claim_date	requested_amount	status
1001	P1001	C001	2026-01-05	85000.0	Approved
1002	P1002	C002	2026-01-08	125000.0	Approved
1003	P1003	C003	2026-01-12	200000.0	Pending
1004	P1004	C004	2026-01-15	45000.0	Approved
1005	P1005	C005	2026-01-18	175000.0	Pending
1006	P1006	C006	2026-01-22	95000.0	Approved
1007	P1007	C007	2026-01-25	65000.0	Approved
1008	P1008	C008	2026-01-28	300000.0	Rejected
1009	P1009	C009	2026-02-02	55000.0	Approved
1010	P1010	C010	2026-02-05	220000.0	Approved
1011	P1001	C001	2026-02-10	120000.0	Pending
1012	P1002	C002	2026-02-14	145000.0	Approved
1013	P1003	C003	2026-02-18	180000.0	Rejected
1014	P1004	C004	2026-02-20	35000.0	Approved
1015	P1005	C005	2026-02-23	150000.0	Pending
1016	P1006	C006	2026-03-01	70000.0	Approved
1017	P1007	C007	2026-03-05	190000.0	Approved
1018	P1008	C008	2026-03-09	250000.0	Pending
1019	P1009	C009	2026-03-12	60000.0	Approved
1020	P1010	C010	2026-03-15	300000.0	Pending
1021	P1001	C001	2026-03-18	50000.0	Approved
1022	P1002	C002	2026-03-22	210000.0	Pending
1023	P1003	C003	2026-03-25	275000.0	Approved
1024	P1004	C004	2026-03-28	40000.0	Rejected
1025	P1005	C005	2026-03-30	125000.0	Approved

SQL Query 2: Total Claim Amount for Each Policy Type

```
SELECT policy_type,
    SUM(requested_amount) AS total_claim_amount
FROM insurance_claims
GROUP BY policy_type
ORDER BY policy_type;
```

Output

```
sqlite> SELECT policy_type,
       SUM(requested_amount) AS total_claim_amount
FROM insurance_claims
GROUP BY policy_type;
```

policy_type	total_claim_amount
Health	420000.0
Life	1205000.0
Motor	735000.0
Property	970000.0
Travel	235000.0

```
sqlite>
```

SQL Query 3: Compare Requested and Approved Amounts

```
SELECT claim_id,
    requested_amount,
    approved_amount,
```

requested_amount - approved_amount AS difference

FROM insurance_claims

WHERE status = 'Approved'

ORDER BY claim_id;

Output

```
sqlite> SELECT claim_id,
       policy_id,
       requested_amount,
       approved_amount,
       (requested_amount - approved_amount) AS difference
FROM insurance_claims
WHERE status = 'Approved';
```

claim_id	policy_id	requested_amount	approved_amount	difference
1001	P1001	85000.0	80000.0	5000.0
1002	P1002	125000.0	110000.0	15000.0
1004	P1004	45000.0	40000.0	5000.0
1006	P1006	95000.0	90000.0	5000.0
1007	P1007	65000.0	60000.0	5000.0
1009	P1009	55000.0	50000.0	5000.0
1010	P1010	220000.0	200000.0	20000.0
1012	P1002	145000.0	130000.0	15000.0
1014	P1004	35000.0	30000.0	5000.0
1016	P1006	70000.0	65000.0	5000.0
1017	P1007	190000.0	175000.0	15000.0
1019	P1009	60000.0	55000.0	5000.0
1021	P1001	50000.0	45000.0	5000.0
1023	P1003	275000.0	250000.0	25000.0
1025	P1005	125000.0	115000.0	10000.0

```
sqlite>
```

SQL Query 4: Average Claim Amount for Each Status

SELECT status,

ROUND(AVG(requested_amount),2) AS average_claim_amount

FROM insurance_claims

GROUP BY status

ORDER BY status;

Output

```
sqlite> SELECT status,
       AVG(requested_amount) AS average_claim_amount
FROM insurance_claims
GROUP BY status;
```

status	average_claim_amount
Approved	109333.33333333333
Pending	200714.28571428571
Rejected	173333.33333333334

```
sqlite>
```

SQL Query 5: Claims Exceeding ₹1,00,000

```
SELECT claim_id,
    policy_id,
    requested_amount,
    status
FROM insurance_claims
WHERE requested_amount > 100000
ORDER BY requested_amount DESC;
```

Output

```
sqlite> SELECT claim_id,
       policy_id,
       customer_id,
       requested_amount,
       status
FROM insurance_claims
WHERE requested_amount > 100000;
```

claim_id	policy_id	customer_id	requested_amount	status
1002	P1002	C002	125000.0	Approved
1003	P1003	C003	200000.0	Pending
1005	P1005	C005	175000.0	Pending
1008	P1008	C008	300000.0	Rejected
1010	P1010	C010	220000.0	Approved
1011	P1001	C001	120000.0	Pending
1012	P1002	C002	145000.0	Approved
1013	P1003	C003	180000.0	Rejected
1015	P1005	C005	150000.0	Pending
1017	P1007	C007	190000.0	Approved
1018	P1008	C008	250000.0	Pending
1020	P1010	C010	300000.0	Pending
1022	P1002	C002	210000.0	Pending
1023	P1003	C003	275000.0	Approved
1025	P1005	C005	125000.0	Approved

```
sqlite>
```

PL/SQL Procedure 1: Register Insurance Claim

```
CREATE OR REPLACE PROCEDURE register_claim (
    p_claim_id        IN NUMBER,
    p_policy_id       IN VARCHAR2,
    p_customer_id     IN VARCHAR2,
    p_claim_date      IN DATE,
    p_policy_type     IN VARCHAR2,
```

```
    p_requested_amount IN NUMBER
)
IS
    v_policy_count NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO v_policy_count
    FROM insurance_policies
    WHERE policy_id = p_policy_id;

    IF v_policy_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Invalid Policy ID');
    END IF;

    INSERT INTO insurance_claims (
        claim_id, policy_id, customer_id, claim_date,
        policy_type, requested_amount, approved_amount, status
    )
    VALUES (
        p_claim_id, p_policy_id, p_customer_id, p_claim_date,
        p_policy_type, p_requested_amount, NULL, 'Pending'
    );

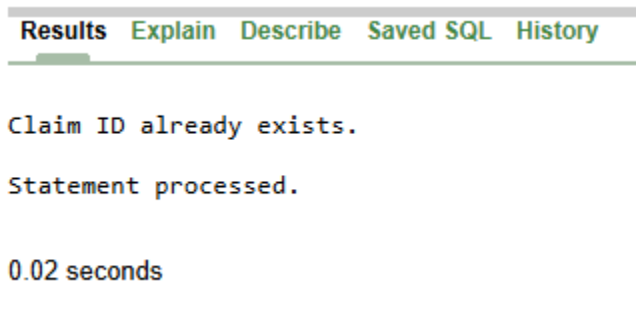
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Insurance claim registered successfully.');
```

```
EXCEPTION
   WHEN DUP_VAL_ON_INDEX THEN
      DBMS_OUTPUT.PUT_LINE('Claim ID already exists.');
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Execution

```
BEGIN
   register_claim(
      1026,
      'P1001',
      'C001',
      DATE '2026-04-01',
      'Health',
      90000
   );
END;
/
```

Output



PL/SQL Function 2: Calculate Claim Difference

```
CREATE OR REPLACE FUNCTION claim_difference (
   p_claim_id IN NUMBER
)
RETURN NUMBER
IS
   v_requested NUMBER;
   v_approved NUMBER;
BEGIN
   SELECT requested_amount, approved_amount
   INTO v_requested, v_approved
   FROM insurance_claims
   WHERE claim_id = p_claim_id;

   IF v_approved IS NULL THEN
      RETURN NULL;
   END IF;

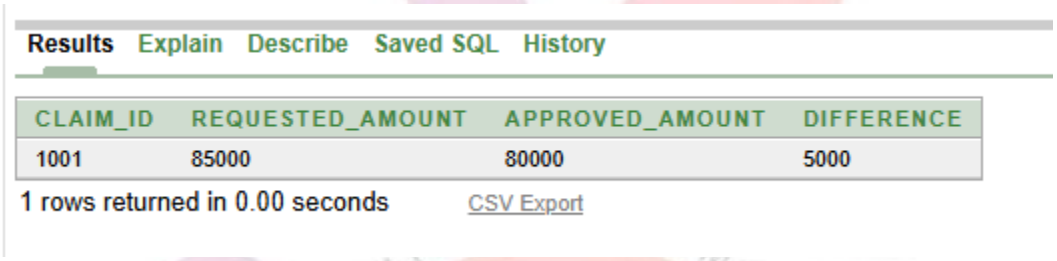
   RETURN v_requested - v_approved;

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20002, 'Claim ID not found');
END;
/
```

Execution

```
SELECT claim_id,
    requested_amount,
    approved_amount,
    claim_difference(claim_id) AS difference
FROM insurance_claims
WHERE status = 'Approved'
AND claim_id = 1001;
```

Output



Results Explain Describe Saved SQL History

CLAIM_ID	REQUESTED_AMOUNT	APPROVED_AMOUNT	DIFFERENCE
1001	85000	80000	5000

1 rows returned in 0.00 seconds CSV Export

PL/SQL Procedure 3: Update Claim Status

```
CREATE OR REPLACE PROCEDURE update_claim_report (
   p_claim_id IN NUMBER,
   p_new_status IN VARCHAR2
)
IS
   v_policy_id insurance_claims.policy_id%TYPE;
BEGIN
   SELECT policy_id
   INTO v_policy_id
   FROM insurance_claims
```

```
    WHERE claim_id = p_claim_id;

    UPDATE insurance_claims
    SET status = p_new_status
    WHERE claim_id = p_claim_id;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Claim status updated successfully.');
    DBMS_OUTPUT.PUT_LINE('Policy ID: ' || v_policy_id);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Claim ID not found.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Execution

```
BEGIN
    update_claim_report(1026, 'Approved');
END;
/
```

Output

Results Explain Describe Saved SQL History

```
Claim status updated successfully.
Policy ID: P1001

Statement processed.


0.00 seconds
```

Policy-wise Claims Report

```
SELECT policy_id,
    COUNT(*) AS total_claims,
    SUM(requested_amount) AS total_requested_amount,
    SUM(NVL(approved_amount,0)) AS total_approved_amount
FROM insurance_claims
GROUP BY policy_id
ORDER BY policy_id;
```

Output

Results Explain Describe Saved SQL History

POLICY_ID	TOTAL_CLAIMS	TOTAL_REQUESTED_AMOUNT	TOTAL_APPROVED_AMOUNT
P1001	4	345000	125000
P1002	3	480000	240000
P1003	3	655000	250000
P1004	3	120000	70000
P1005	3	450000	115000
P1006	2	165000	155000
P1007	2	255000	235000
P1008	2	550000	0
P1009	2	115000	105000
P1010	2	520000	200000

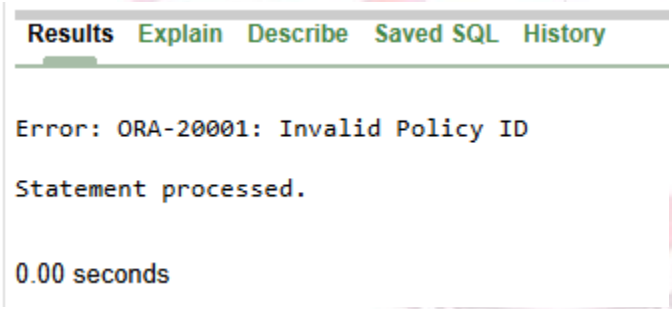
10 rows returned in 0.02 seconds CSV Export

Exception Handling

Test Case 1 – Invalid Policy ID

```
BEGIN
   register_claim(
      1027,
      'P9999',
      'C999',
      DATE '2026-04-05',
      'Health',
      75000
   );
END;
/
```

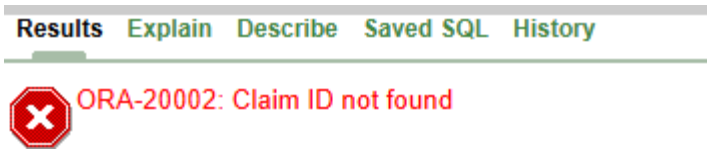
Output



Test Case 2 – Invalid Claim ID

```
SELECT claim_difference(9999) AS difference
FROM dual;
```

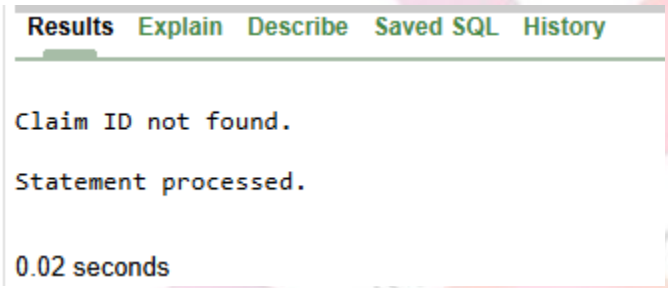
Output



Test Case 3 – Invalid Claim ID for Status Update

```
BEGIN
   update_claim_report(9999, 'Approved');
END;
/
```

Output



Dataset Insights

- The dataset contains 25 insurance claim records.
- The claims belong to Health, Motor, Life, Travel, and Property policy types.
- The claim statuses are Approved, Pending, and Rejected.
- Requested and approved amounts can be compared to analyze claim processing.
- Claims exceeding ₹1,00,000 can be identified using SQL.
- Policy-wise aggregation helps analyze the number and value of claims.

Result

Thus, the Insurance Policy and Claims Processing system was successfully implemented using SQL and PL/SQL. The required tables were created, claim records were inserted, SQL queries were executed successfully, PL/SQL procedures and functions were implemented, and exception handling was tested successfully.

ANNEXURES: SQL PRACTICE PROGRAMS

KHAN ACADEMY

1 BOOK LIST DATABASE

What are your favorite books? You can make a database table to store them in! In this first step, create a table to store your list of books. It should have columns for id, name, and rating. Now, add three of your favorite books into the table.

Program:

CREATE TABLE books(id INTEGER PRIMARY KEY, name TEXT, rating INTEGER);

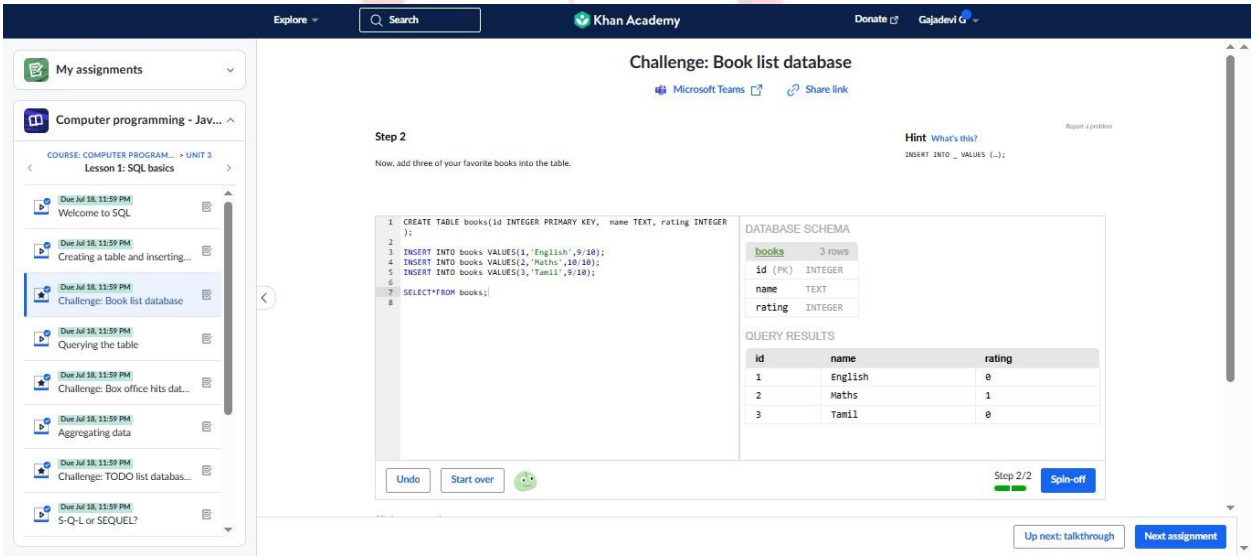
CREATE TABLE books(id INTEGER PRIMARY KEY, name TEXT, rating INTEGER);

INSERT INTO books VALUES(1,'English',9/10);

INSERT INTO books VALUES(2,'Maths',10/10);

INSERT INTO books VALUES(3,'Tamil',9/10);

SELECT*FROM books;



2 BOX OFFICE HITS DATABASE

This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies. Now, add a second query after the first, that retrieves only the movies that were released in the year 2000 or later, not before. Sort the results so that the earlier movies are listed first. You should have 2 SELECT statements after this step.

Program:

CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);

INSERT INTO movies VALUES (1, "Avatar", 2009);

INSERT INTO movies VALUES (2, "Titanic", 1997);

INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977);

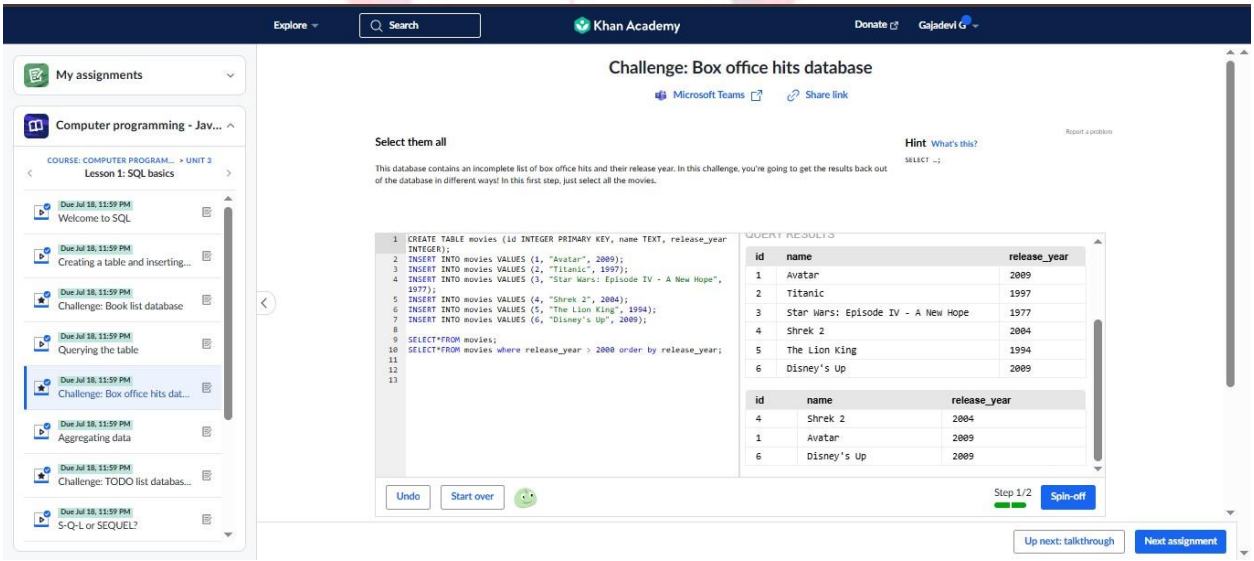
INSERT INTO movies VALUES (4, "Shrek 2", 2004);

INSERT INTO movies VALUES (5, "The Lion King", 1994);

INSERT INTO movies VALUES (6, "Disney's Up", 2009);

SELECT*FROM movies;

SELECT*FROM movies where release_year > 2000 order by release_year;



3. TODO LISTDATABASE STATS

Here's a table containing a TODO list with the number of minutes it will take to complete each item. Insert another item to your todo list with the estimated minutes it will take. Select the SUM of minutes it will take to do all of the items on your TODO list. You should only have one SELECT statement.

Program:

```
CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER);

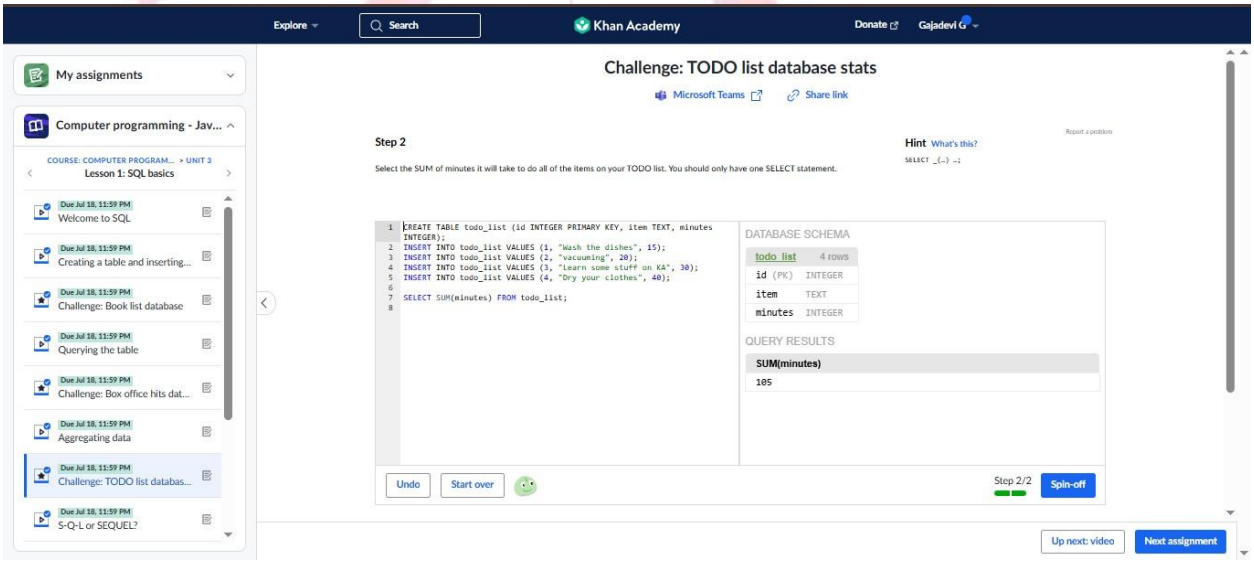
INSERT INTO todo_list VALUES (1, "Wash the dishes", 15);

INSERT INTO todo_list VALUES (2, "vacuuming", 20);

INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);

INSERT INTO todo_list VALUES (4, "Dry your clothes", 40);

SELECT SUM(minutes) FROM todo_list;
```



4. PROJECT : DESIGN A STORE DATABASE

Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.

Program:

```
CREATE TABLE sneakers (
    id INTEGER PRIMARY KEY,
    brand TEXT,
    model TEXT,
    size INTEGER,
    color TEXT,
    price REAL
);
INSERT INTO sneakers VALUES (1, "Nike", "Air Max", 9, "Black", 120);
INSERT INTO sneakers VALUES (2, "Adidas", "Ultraboost", 10, "White", 180);
INSERT INTO sneakers VALUES (3, "Puma", "RS-X", 8, "Blue", 110);
INSERT INTO sneakers VALUES (4, "New Balance", "574", 9, "Gray", 95);
INSERT INTO sneakers VALUES (5, "Converse", "Chuck Taylor", 10, "Red", 60);
INSERT INTO sneakers VALUES (6, "Vans", "Old Skool", 8, "Black", 70);
INSERT INTO sneakers VALUES (7, "Nike", "Jordan 1", 11, "Red", 200);
INSERT INTO sneakers VALUES (8, "Adidas", "NMD", 9, "Black", 140);
INSERT INTO sneakers VALUES (9, "Puma", "Suede Classic", 10, "Green", 75);
INSERT INTO sneakers VALUES (10, "New Balance", "990", 11, "Gray", 185);
INSERT INTO sneakers VALUES (11, "Converse", "Run Star Hike", 8, "White", 115);
```

```
INSERT INTO sneakers VALUES (12, "Vans", "Sk8-Hi", 9, "Blue", 80);
INSERT INTO sneakers VALUES (13, "Nike", "Air Force 1", 10, "White", 130);
INSERT INTO sneakers VALUES (14, "Adidas", "Superstar", 8, "Black", 100);
INSERT INTO sneakers VALUES (15, "Puma", "Future Rider", 9, "Yellow", 90);
SELECT *
FROM sneakers
ORDER BY price DESC;
SELECT
   AVG(price) AS average_price,
   MAX(price) AS highest_price,
   MIN(price) AS lowest_price
FROM sneakers;
```

Output

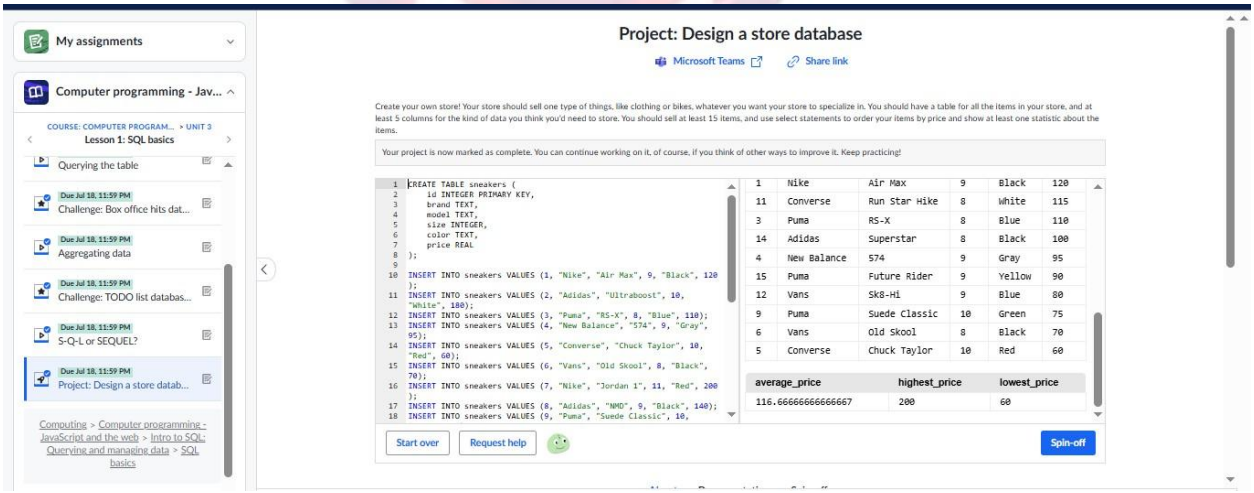
DATABASE SCHEMA

sneakers 15 rows
id (PK) INTEGER
brand TEXT
model TEXT
size INTEGER
color TEXT
price REAL

QUERY RESULTS

id	brand	model	size	color	price
7	Nike	Jordan 1	11	Red	200
10	New Balance	990	11	Gray	185
2	Adidas	Ultraboost	10	White	180
8	Adidas	NMD	9	Black	140
13	Nike	Air Force 1	10	White	130
1	Nike	Air Max	9	Black	120
11	Converse	Run Star Hike	8	White	115
3	Puma	RS-X	8	Blue	110
14	Adidas	Superstar	8	Black	100
4	New Balance	574	9	Gray	95
15	Puma	Future Rider	9	Yellow	90
12	Vans	Sk8-Hi	9	Blue	80
9	Puma	Suede Classic	10	Green	75
6	Vans	Old Skool	8	Black	70
5	Converse	Chuck Taylor	10	Red	60

average_price	highest_price	lowest_price
116.66666666666667	200	60



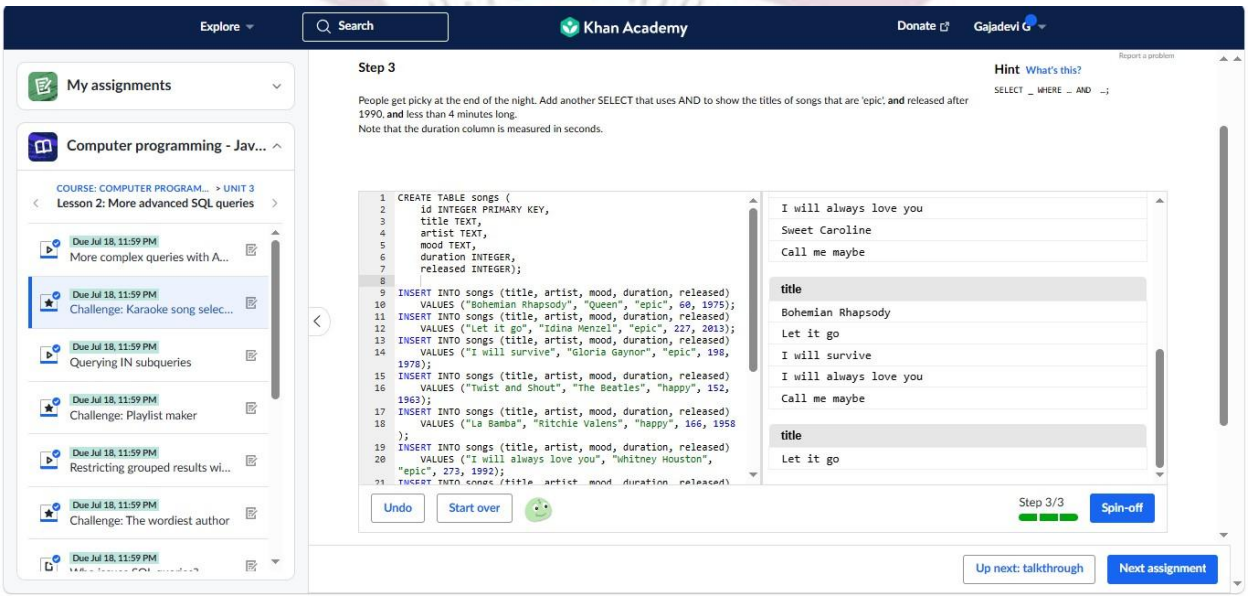
5. KARAOKE SONG SELECTOR

Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles. Maybe your friends only like singing either recent songs or truly epic songs. Add another SELECT that uses OR to show the titles of the songs that have an 'epic' mood or a release date after 1990. People get picky at the end of the night. Add another SELECT that uses AND to show the titles of songs that are 'epic', and released after 1990, and less than 4 minutes long.

Program:

```
CREATE TABLE songs (
    id INTEGER PRIMARY KEY,
    title TEXT,
    artist TEXT,
    mood TEXT,
    duration INTEGER,
    released INTEGER);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("La Bamba", "Ritchie Valens", "happy", 166, 1958);
```

```
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);
select title from songs;
select title from songs where mood = 'epic' or released > 1990;
select title from songs where mood = 'epic' and released > 1990 and duration < 240;
```



6. PLAYLIST MAKER

We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the title of all the songs by the artist named 'Queen'. Now you'll make a 'Pop' playlist. In preparation, select the name of all of the artists from the 'Pop' genre. To finish creating the 'Pop' playlist, add another query that will select the title of all the songs from the 'Pop' artists. It should use IN on a nested subquery that's based on your previous query.

Program:

```
CREATE TABLE artists (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    country TEXT,
    genre TEXT);
INSERT INTO artists (name, country, genre)
   VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
    VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
```

```
    VALUES ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Rihanna", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Guns N' Roses", "US", "Hard rock");
 INSERT INTO artists (name, country, genre)
    VALUES ("Gloria Estefan", "US", "Pop");
 INSERT INTO artists (name, country, genre)
    VALUES ("Bob Marley", "Jamaica", "Reggae");
CREATE TABLE songs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    artist TEXT,
    title TEXT);
INSERT INTO songs (artist, title)
```

```
    VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
    VALUES ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "My heart will go on");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
    VALUES ("Shania Twain", "Party for two");
INSERT INTO songs (artist, title)
    VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
    VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
    VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bohemian Rhapsody");
INSERT INTO songs (artist, title)
```

```
    VALUES ("Guns N' Roses", "Don't cry");

select title from songs where artist = 'Queen';

select name from artists where genre = 'Pop';

SELECT title

FROM songs

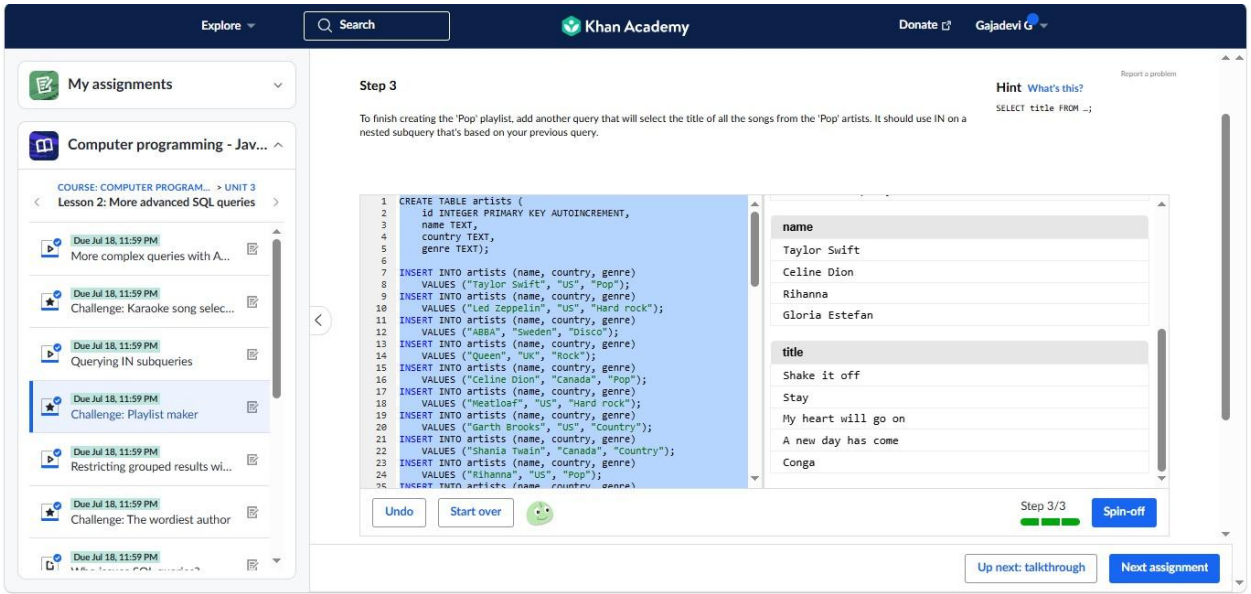
WHERE artist IN (

    SELECT name

    FROM artists

    WHERE genre = 'Pop'

);
```



7. THE WORDIEST AUTHOR

We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using GROUP BY and HAVING. Your results table should include the 'author' and their total word count as a 'total_words' column. Now select all the authors that write more than an average of 150,000 words per book. Your results table should include the 'author' and average words as an 'avg_words' column.

Program:

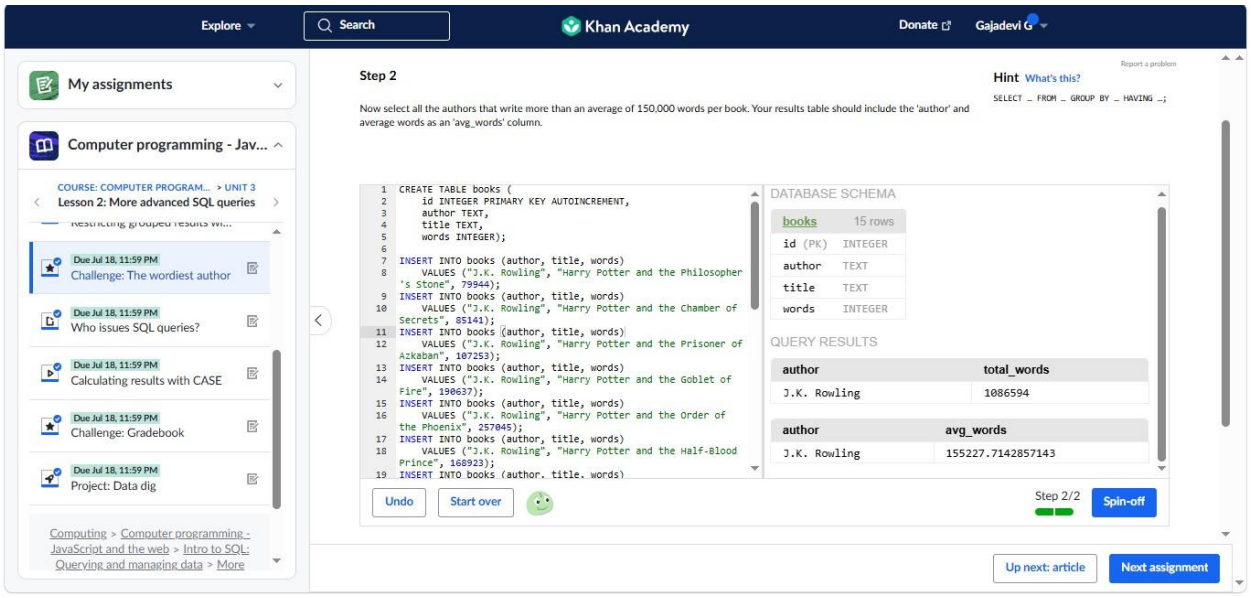
```
CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author TEXT,
    title TEXT,
    words INTEGER);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
```

```
    VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Twilight", 118501);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
```

```
    VALUES ("J.R.R. Tolkien", "Return of the King", 134462);

SELECT author,
       SUM(words) AS total_words
FROM books
GROUP BY author
HAVING SUM(words) > 1000000;

SELECT author,
       AVG(words) AS avg_words
FROM books
GROUP BY author
HAVING AVG(words) > 150000;
```



8. GRADEBOOK

We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display the name, number_grade, and percent_completed, which you can compute by multiplying and rounding the fraction_completed column. Now, this step is a little tricky. The goal is a table that shows how many students have earned which letter_grade. You can output the letter_grade by using CASE with the number_grade column, outputting 'A' for grades > 90, 'B' for grades > 80, 'C' for grades > 70, and 'F' otherwise. Then you can use COUNT with GROUP BY to show the number of students with each of those grades.

Program:

```
CREATE TABLE student_grades (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    number_grade INTEGER,
    fraction_completed REAL);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winstonia", 82, 0.9045);
```

```
SELECT name,
       number_grade,
       ROUND(fraction_completed * 100) AS percent_completed
FROM student_grades;
SELECT COUNT(*),
   CASE
      WHEN number_grade > 90 THEN 'A'
      WHEN number_grade > 80 THEN 'B'
      WHEN number_grade > 70 THEN 'C'
      ELSE 'F'
   END AS "letter_grade"
FROM student_grades
GROUP BY
   CASE
      WHEN number_grade > 90 THEN 'A'
      WHEN number_grade > 80 THEN 'B'
      WHEN number_grade > 70 THEN 'C'
      ELSE 'F'
   END;
```



9. DATA DIG

What are average, max, and min values in the data?
What about those numbers per category in the data (using HAVING)?
What ways are there to group the data values that don't exist yet (using CASE)?
What interesting ways are there to filter the data (using AND/OR)?

Program:

```
CREATE TABLE pokemon (
    id INTEGER PRIMARY KEY,
    name TEXT,
    type TEXT,
    hp INTEGER,
    attack INTEGER
);
INSERT INTO pokemon VALUES (1, "Pikachu", "Electric", 35, 55);
INSERT INTO pokemon VALUES (2, "Charizard", "Fire", 78, 84);
INSERT INTO pokemon VALUES (3, "Blastoise", "Water", 79, 83);
INSERT INTO pokemon VALUES (4, "Venusaur", "Grass", 80, 82);
INSERT INTO pokemon VALUES (5, "Jolteon", "Electric", 65, 65);
INSERT INTO pokemon VALUES (6, "Arcanine", "Fire", 90, 110);
INSERT INTO pokemon VALUES (7, "Gyarados", "Water", 95, 125);
INSERT INTO pokemon VALUES (8, "Sceptile", "Grass", 70, 85);
SELECT
    AVG(attack) AS avg_attack,
    MAX(attack) AS max_attack,
    MIN(attack) AS min_attack
FROM pokemon;
```

```
SELECT
    type,
    AVG(attack) AS avg_attack
FROM pokemon
GROUP BY type;
SELECT
    type,
    AVG(attack) AS avg_attack
FROM pokemon
GROUP BY type
HAVING AVG(attack) > 80;
SELECT
    name,
    attack,
    CASE
        WHEN attack >= 100 THEN "Very Strong"
        WHEN attack >= 80 THEN "Strong"
        ELSE "Average"
    END AS strength
FROM pokemon;
SELECT *
FROM pokemon
WHERE (type = "Fire" OR type = "Water")
AND attack > 80;
```

Output

DATABASE SCHEMA

pokemon 8 rows
id (PK) INTEGER
name TEXT
type TEXT
hp INTEGER
attack INTEGER

QUERY RESULTS

avg_attack	max_attack	min_attack
86.125	125	55

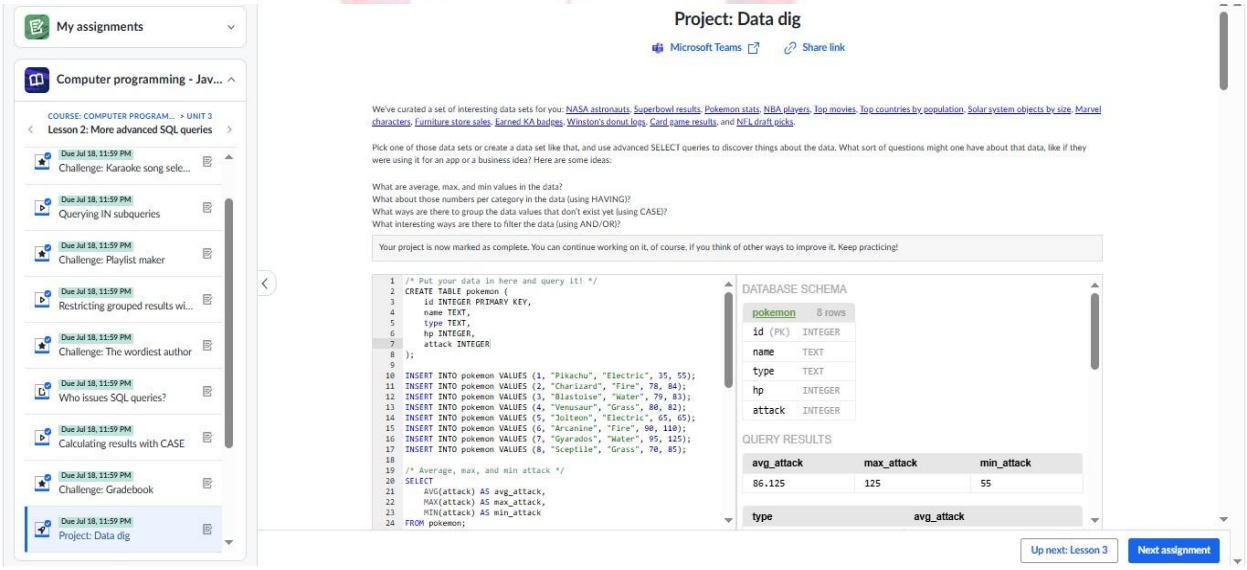
type	avg_attack
Electric	60
Fire	97
Grass	83.5
Water	104

type	avg_attack
Fire	97
Grass	83.5
Water	104

name	attack	strength
Pikachu	55	Average
Charizard	84	Strong
Blastoise	83	Strong
Venusaur	82	Strong

name	attack	strength
Jolteon	65	Average
Arcanine	110	Very Strong
Gyarados	125	Very Strong
Sceptile	85	Strong

id	name	type	hp	attack
2	Charizard	Fire	78	84
3	Blastoise	Water	79	83
6	Arcanine	Fire	90	110
7	Gyarados	Water	95	125



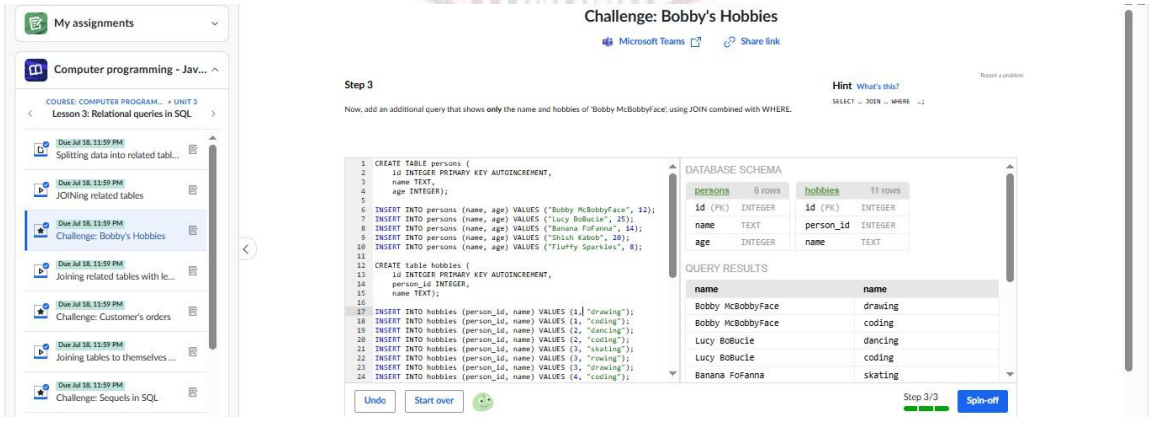
10. BOBBY'S HOBBIES

We've created a database of people and hobbies, and each row in hobbies is related to a row in persons via the person_id column. In this first step, insert one more row in persons and then one more row in hobbies that is related to the newly inserted person. Now, select the 2 tables with a join so that you can see each person's name next to their hobby. Now, add an additional query that shows only the name and hobbies of 'Bobby McBobbyFace', using JOIN combined with WHERE.

Program:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER);
INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);
CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
```

```
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
INSERT INTO persons (name, age)
VALUES ("Sam Star", 18);
INSERT INTO hobbies (person_id, name)
VALUES (6, "gaming");
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT persons.name,
    hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id
WHERE persons.name = "Bobby McBobbyFace";
```



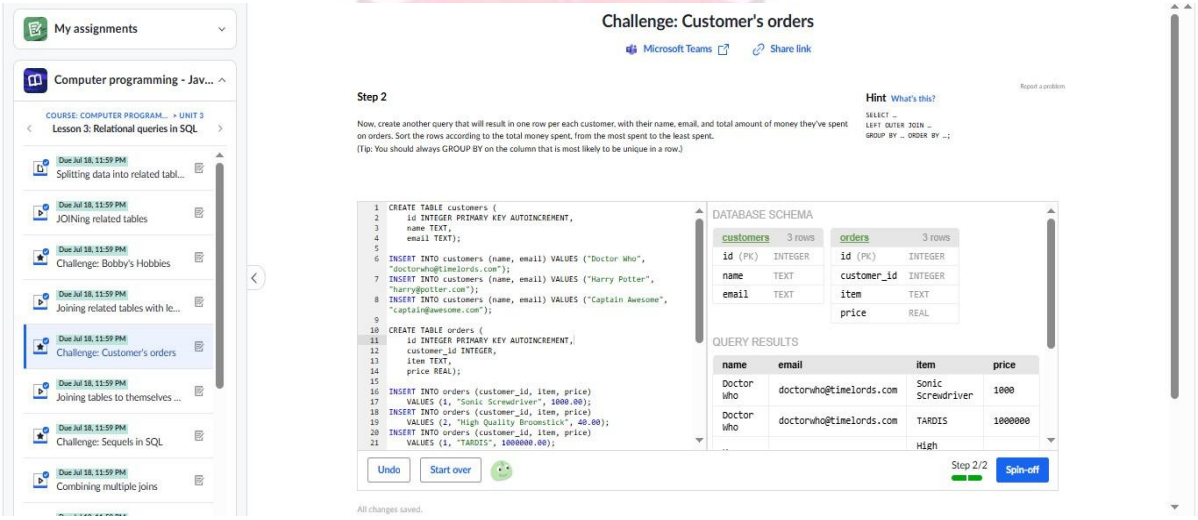
11. CUSTOMER'S ORDERS

We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY. Now, create another query that will result in one row per each customer, with their name, email, and total amount of money they've spent on orders. Sort the rows according to the total money spent, from the most spent to the least spent.

Program:

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT);
INSERT INTO customers (name, email) VALUES ("Doctor Who",
"doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome",
"captain@awesome.com");
CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER,
    item TEXT,
    price REAL);
INSERT INTO orders (customer_id, item, price)
    VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (2, "High Quality Broomstick", 40.00);
```

```
INSERT INTO orders (customer_id, item, price)
    VALUES (1, "TARDIS", 1000000.00);
SELECT customers.name,
        customers.email,
        orders.item,
        orders.price
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;
SELECT customers.name,
        customers.email,
        SUM(orders.price) AS total_spent
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id
GROUP BY customers.id
ORDER BY total_spent DESC;
```



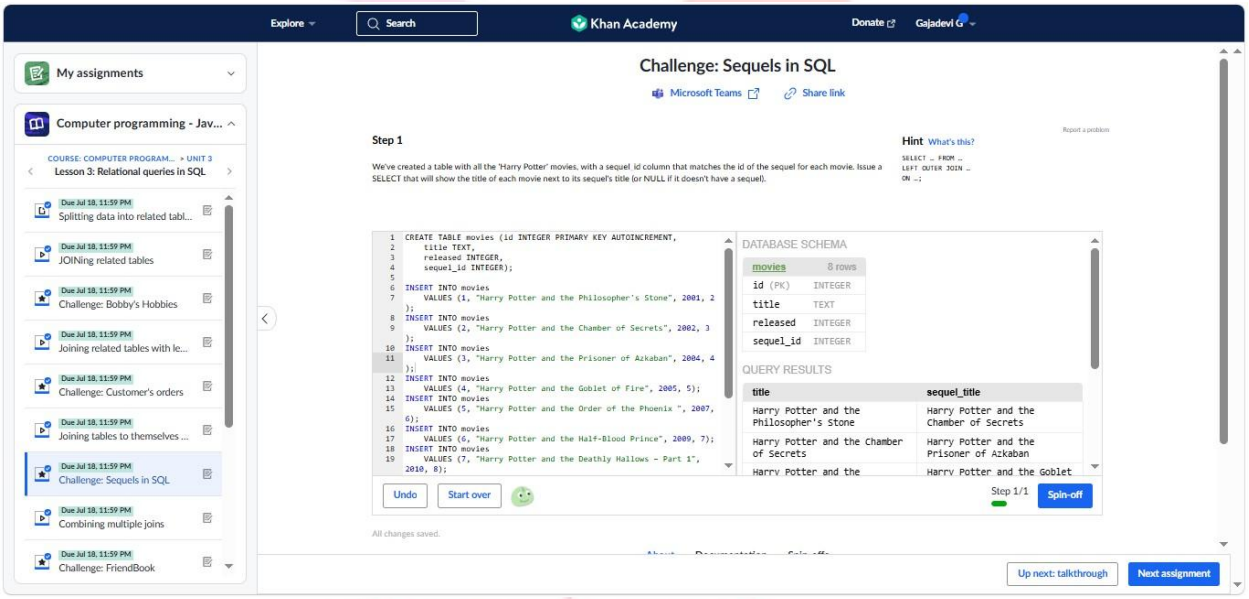
12. SEQUELS IN SQL

We've created a table with all the 'Harry Potter' movies, with a sequel_id column that matches the id of the sequel for each movie. Issue a SELECT that will show the title of each movie next to its sequel's title (or NULL if it doesn't have a sequel).

Program:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    released INTEGER,
    sequel_id INTEGER);
INSERT INTO movies
    VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2);
INSERT INTO movies
    VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);
INSERT INTO movies
    VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
    VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);
INSERT INTO movies
    VALUES (5, "Harry Potter and the Order of the Phoenix ", 2007, 6);
INSERT INTO movies
    VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7);
INSERT INTO movies
    VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8);
INSERT INTO movies
    VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);
```

```
SELECT m1.title,
       m2.title AS sequel_title
FROM movies AS m1
LEFT OUTER JOIN movies AS m2
ON m1.sequel_id = m2.id;
```



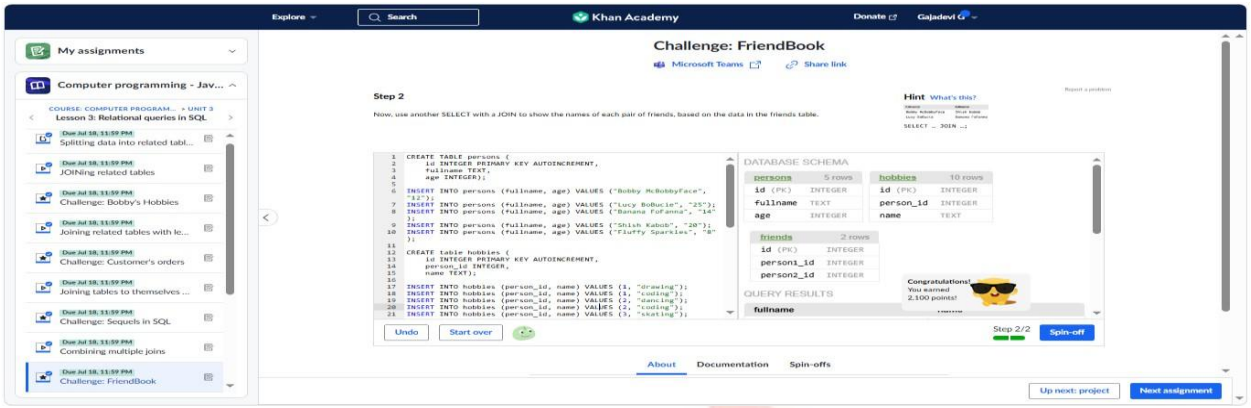
13. FRIENDBOOK

We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOIN to display a table showing people's names with their hobbies. Now, use another SELECT with a JOIN to show the names of each pair of friends, based on the data in the friends table.

Program:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    fullname TEXT,
    age INTEGER);
INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", "12");
INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", "25");
INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", "14");
INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", "20");
INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", "8");
CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
```

```
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
CREATE table friends (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   person1_id INTEGER,
   person2_id INTEGER);
INSERT INTO friends (person1_id, person2_id)
   VALUES (1, 4);
INSERT INTO friends (person1_id, person2_id)
   VALUES (2, 3);
SELECT persons.fullname,
      hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT p1.fullname AS person1,
      p2.fullname AS person2
FROM friends
JOIN persons AS p1
ON friends.person1_id = p1.id
JOIN persons AS p2
ON friends.person2_id = p2.id;
```



14.FAMOUS PEOPLE

Movie stars: What movies are they in? Are they married to each other?
Singers: What songs did they write? Where are they from?
Authors: What books did they write?
Fictional characters: How are they related to other characters? What books do they show up in?

Program:

```
CREATE TABLE authors (
    id INTEGER PRIMARY KEY,
    name TEXT,
    country TEXT
);
INSERT INTO authors VALUES (1, "J.K. Rowling", "United Kingdom");
INSERT INTO authors VALUES (2, "George Orwell", "United Kingdom");
INSERT INTO authors VALUES (3, "Mark Twain", "United States");
INSERT INTO authors VALUES (4, "Jane Austen", "United Kingdom");
CREATE TABLE books (
    id INTEGER PRIMARY KEY,
    title TEXT,
    author_id INTEGER
```

```
);
INSERT INTO books VALUES (1, "Harry Potter and the Sorcerer's Stone", 1);
INSERT INTO books VALUES (2, "Harry Potter and the Chamber of Secrets", 1);
INSERT INTO books VALUES (3, "1984", 2);
INSERT INTO books VALUES (4, "Animal Farm", 2);
INSERT INTO books VALUES (5, "The Adventures of Tom Sawyer", 3);
INSERT INTO books VALUES (6, "Pride and Prejudice", 4);
CREATE TABLE author_friends (
    id INTEGER PRIMARY KEY,
    author1_id INTEGER,
    author2_id INTEGER
);

INSERT INTO author_friends VALUES (1, 1, 2);
INSERT INTO author_friends VALUES (2, 3, 4);
SELECT authors.name AS author,
        books.title AS book
FROM authors
JOIN books
ON authors.id = books.author_id;
SELECT name,
        country
FROM authors;
SELECT a1.name AS author1,
        a2.name AS author2
```

```
FROM author_friends
JOIN authors AS a1
ON author_friends.author1_id = a1.id
JOIN authors AS a2
ON author_friends.author2_id = a2.id;
```

Output:

DATABASE SCHEMA

authors4 rows
id (PK)INTEGER
nameTEXT
countryTEXT

books6 rows
id (PK)INTEGER
titleTEXT
author_idINTEGER

author_friends2 rows
id (PK)INTEGER
author1_idINTEGER
author2_idINTEGER

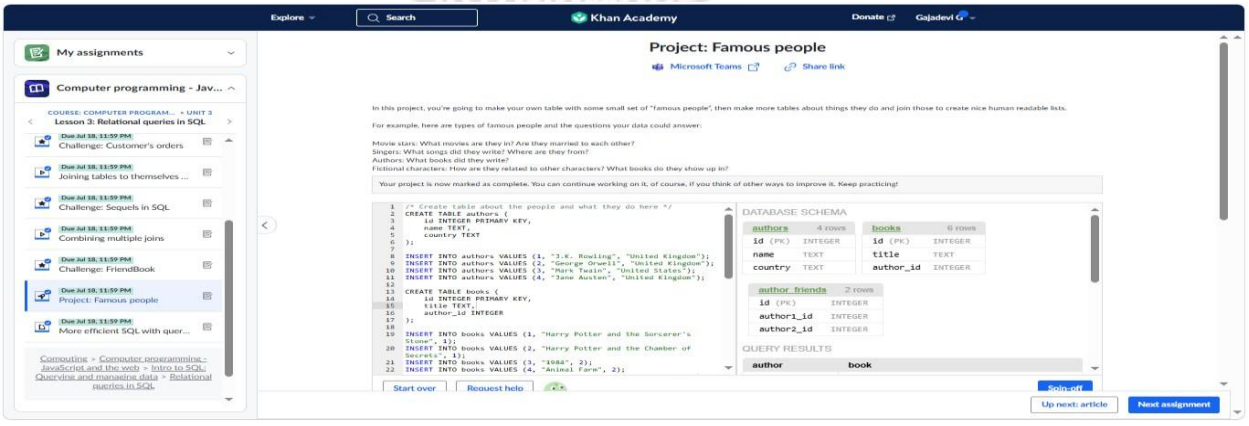
QUERY RESULTS

author	book
J.K. Rowling	Harry Potter and the Sorcerer's Stone
J.K. Rowling	Harry Potter and the Chamber of Secrets

author	book
George Orwell	1984
George Orwell	Animal Farm
Mark Twain	The Adventures of Tom Sawyer
Jane Austen	Pride and Prejudice

name	country
J.K. Rowling	United Kingdom
George Orwell	United Kingdom
Mark Twain	United States
Jane Austen	United Kingdom

author1	author2
J.K. Rowling	George Orwell
Mark Twain	Jane Austen



15. DYNAMIC DOCUMENTS

We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use UPDATE to change the author to 'Jackie Draper' for all rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected. Now you'll delete a row, being very careful not to delete all the rows. Only delete the row where the title is 'Things I'm Afraid Of'. Then re-select all the rows to make sure the table changed like you expected.

Program:

```
CREATE table documents (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    content TEXT,
    author TEXT);
INSERT INTO documents (author, title, content)
    VALUES ("Puff T.M. Dragon", "Fancy Stuff", "Ceiling wax, dragon wings, etc.");
INSERT INTO documents (author, title, content)
    VALUES ("Puff T.M. Dragon", "Living Things", "They're located in the left ear, you know.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Pirate Recipes", "Cherry pie, apple pie, blueberry pie.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Boat Supplies", "Rudder - guitar. Main mast - bed post.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Things I'm Afraid Of", "Talking to my parents, the sea, giant pirates, heights.");
SELECT * FROM documents;
UPDATE documents
SET author = "Jackie Draper"
```

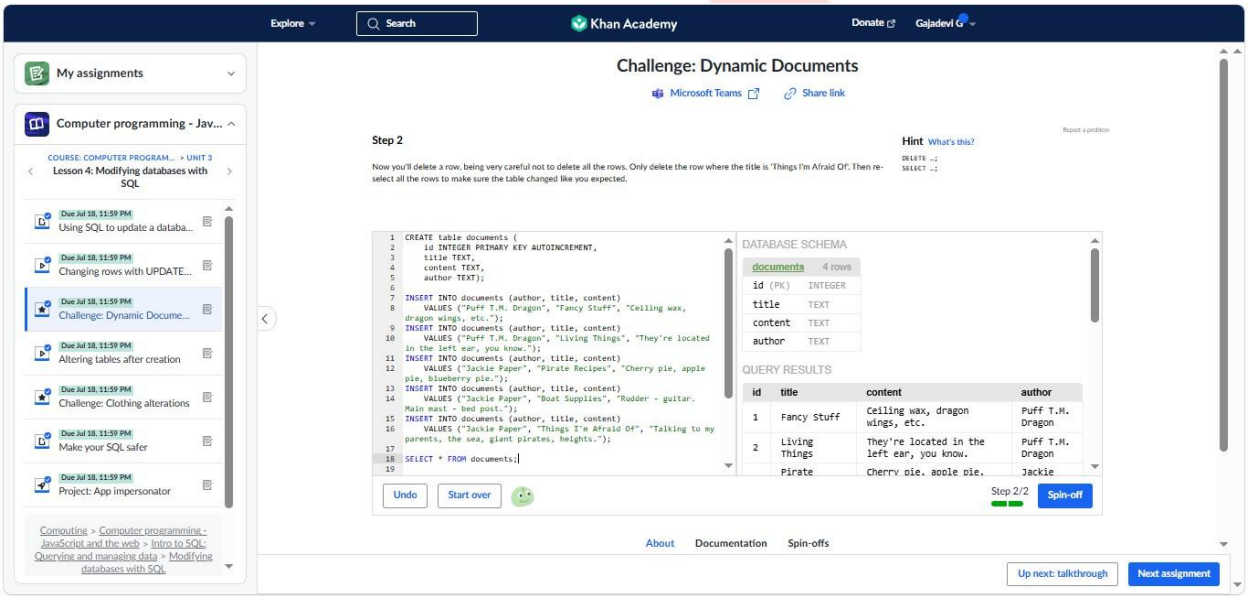
WHERE author = "Jackie Paper";

SELECT * FROM documents;

DELETE FROM documents

WHERE title = "Things I'm Afraid Of";

SELECT * FROM documents;



AUTONOMOUS

16. CLOTHING ALTERATIONS

We've created a database of clothes, and decided we need a price column. Use ALTER to add a 'price' column to the table. Then select all the columns in each row to see what your table looks like now. Now assign each item a price, using UPDATE - item 1 should be 10 dollars, item 2 should be 20 dollars, item 3 should be 30 dollars. When you're done, do another SELECT of all the rows to check that it worked as expected. Now insert a new item into the table that has all three attributes filled in, including 'price'. Do one final SELECT of all the rows to check it worked.

Program:

```
CREATE TABLE clothes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    type TEXT,
    design TEXT);
INSERT INTO clothes (type, design)
    VALUES ("dress", "pink polka dots");
INSERT INTO clothes (type, design)
    VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
    VALUES ("blazer", "black sequin");
ALTER TABLE clothes
ADD COLUMN price REAL;
SELECT * FROM clothes;
UPDATE clothes
SET price = 10
WHERE id = 1;
UPDATE clothes
SET price = 20
```

WHERE id = 2;

UPDATE clothes

SET price = 30

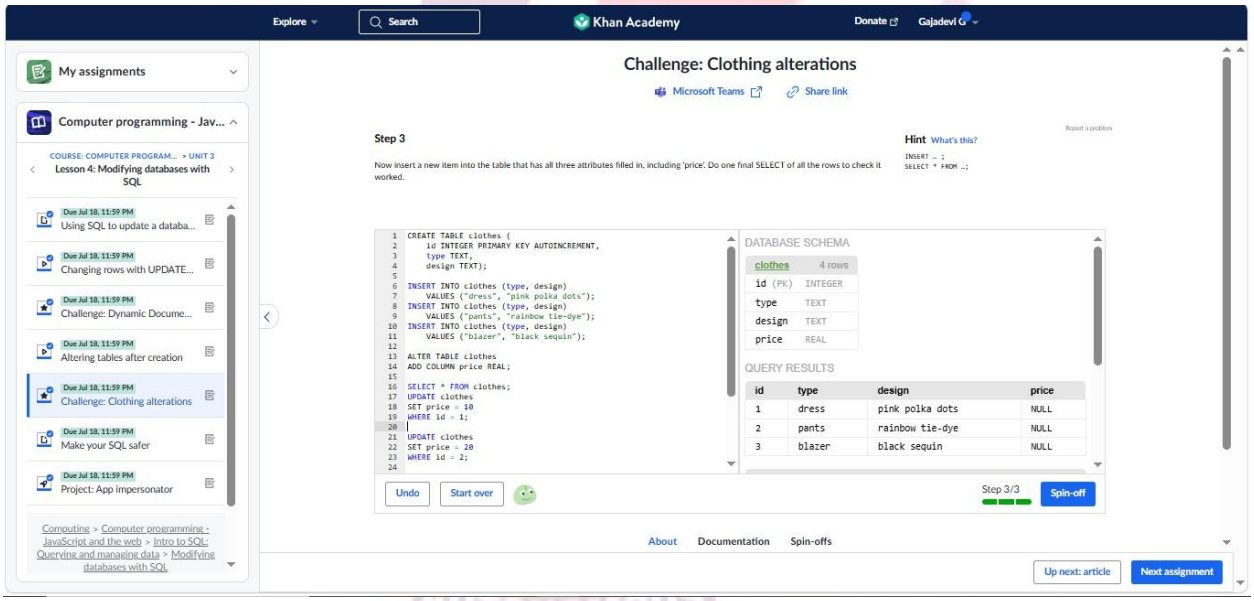
WHERE id = 3;

SELECT * FROM clothes;

INSERT INTO clothes (type, design, price)

VALUES ("shirt", "blue stripes", 15);

SELECT * FROM clothes;



17. PROJECT: APP IMPERSONATO

CREATE a table to store the data.
INSERT a few example rows in the table.
Use an UPDATE to emulate what happens when you edit data in the app.
Use a DELETE to emulate what happens when you delete data in the app.

Program:

```
CREATE TABLE posts (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   username TEXT,
   content TEXT,
   likes INTEGER
);
INSERT INTO posts (username, content, likes)
VALUES ("alex", "Just finished my homework!", 12);
INSERT INTO posts (username, content, likes)
VALUES ("sam", "Playing my favorite game tonight.", 25);
INSERT INTO posts (username, content, likes)
VALUES ("taylor", "Had a great day at the beach.", 18);
SELECT * FROM posts;
UPDATE posts
SET content = "Just finished my homework and studied SQL!"
WHERE id = 1;
SELECT * FROM posts;
DELETE FROM posts
WHERE id = 2;
SELECT * FROM posts;
```

Output:

DATABASE SCHEMA

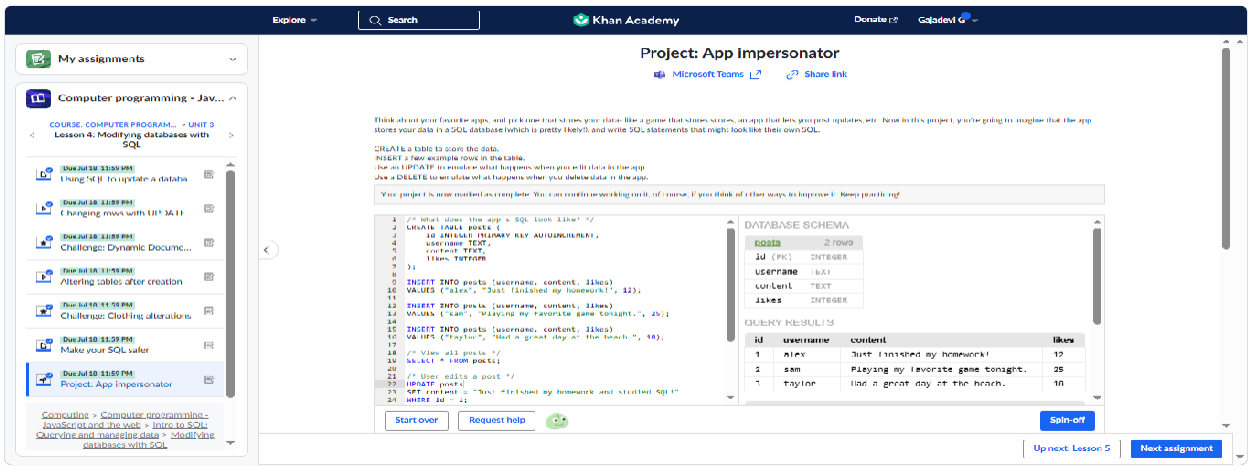
posts2 rows
id (PK)INTEGER
usernameTEXT
contentTEXT
likesINTEGER

QUERY RESULTS

id	username	content	likes
1	alex	Just finished my homework!	12
2	sam	Playing my favorite game tonight.	25
3	taylor	Had a great day at the beach.	18

id	username	content	likes
1	alex	Just finished my homework and studied SQL!	12
2	sam	Playing my favorite game tonight.	25
3	taylor	Had a great day at the beach.	18

id	username	content	likes
1	alex	Just finished my homework and studied SQL!	12
3	taylor	Had a great day at the beach.	18



Hacker rank

1. Revising the Select Query I

Query all columns for all American cities in the **CITY** table with populations larger than `100000`. The **CountryCode** for America is `USA`.

The **CITY** table is described as follows:

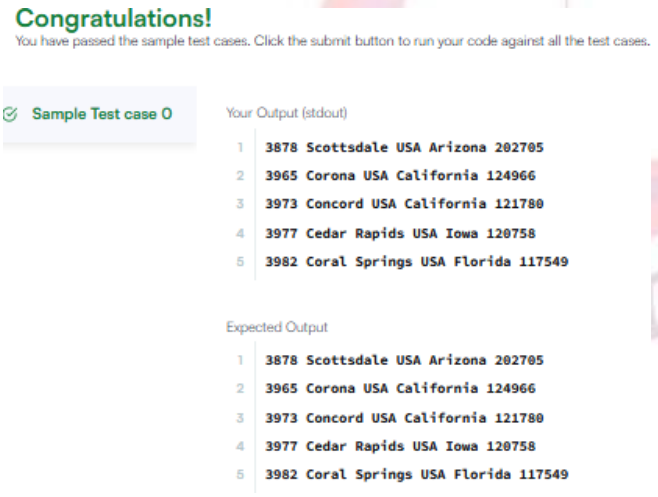
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT * FROM CITY WHERE COUNTRYCODE = 'USA' AND POPULATION > 100000;
```

Output:



2. Revising the Select Query II

Query the **NAME** field for all American cities in the **CITY** table with populations larger than `120000`. The *CountryCode* for America is `USA`.

The **CITY** table is described as follows:

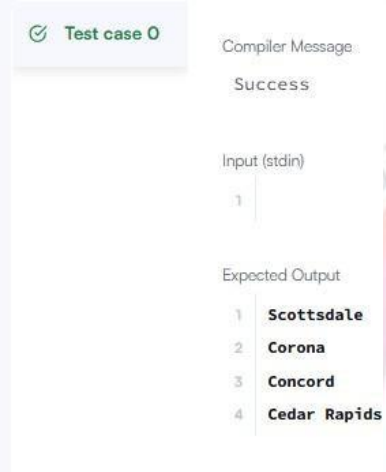
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT NAMEFROM CITY WHERE COUNTRYCODE = 'USA' AND POPULATION > 120000;
```

Output:



3. Select All

Query all columns (attributes) for every row in the **CITY** table.

The **CITY** table is described as follows:

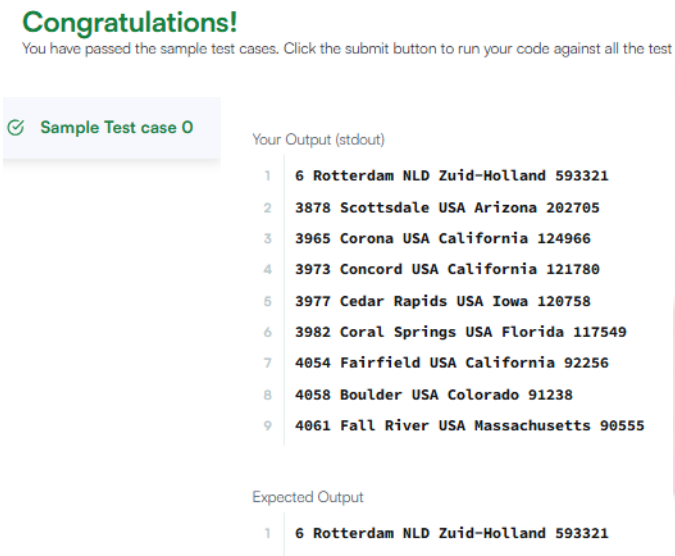
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT * FROM CITY;
```

Output:



4. Select By ID

Query all columns for a city in **CITY** with the *ID* `1661`.

The **CITY** table is described as follows:

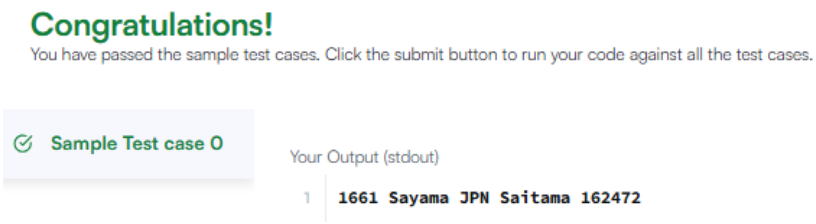
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT * FROM CITY WHERE ID = 1661;
```

Output:



5. Japanese Cities' Attributes

Query all attributes of every Japanese city in the **CITY** table. The **COUNTRYCODE** for Japan is `JPN`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT * FROM CITY WHERE COUNTRYCODE = 'JPN';
```

Output:



6. Japanese Cities' Names

Query the names of all the Japanese cities in the **CITY** table. The **COUNTRYCODE** for Japan is `JPN`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT NAME FROM CITY WHERE COUNTRYCODE = 'JPN';
```

Output:



7. Weather Observation Station 1

Query a list of **CITY** and **STATE** from the **STATION** table.

The **STATION** table is described as follows:

STATION

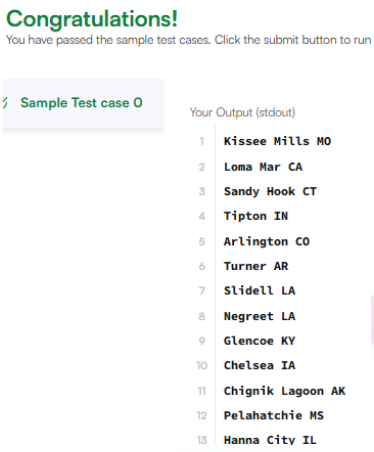
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

Input:

```
SELECT CITY, STATE FROM STATION;
```

Output:



8. Weather Observation Station 3

Query a list of **CITY** names from **STATION** for cities that have an even **ID** number. Print the results in any order, but exclude duplicates from the answer.

The **STATION** table is described as follows:

STATION

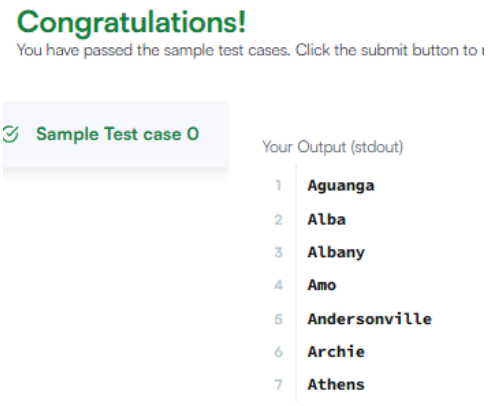
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE MOD(ID, 2) = 0 ORDER BY CITY;
```

Output:



9. Weather Observation Station 4

Find the difference between the total number of **CITY** entries in the table and the number of distinct **CITY** entries in the table.
The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

For example, if there are three records in the table with **CITY** values 'New York', 'New York', 'Bengalaru', there are 2 different city names: 'New York' and 'Bengalaru'.

Input:

```
SELECT COUNT(CITY) - COUNT (DISTINCT City) FROM STATION ;
```

Output:



10. Weather Observation Station 5

Query the two cities in **STATION** with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

Sample Input

For example, **CITY** has four entries: **DEF, ABC, PQRS** and **WXY**.

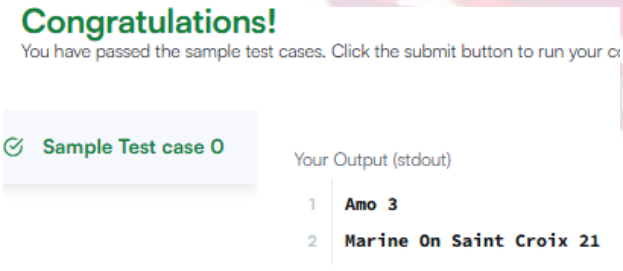
Sample Output

```
ABC 3
PQRS 4
```

Input:

```
SELECT CITY, LENGTH(CITY) FROM STATION ORDER BY LENGTH(CITY), CITY LIMIT 1;
SELECT CITY, LENGTH(CITY) FROM STATION ORDER BY LENGTH(CITY) DESC, CITY LIMIT
1;
```

Output:



11. Weather Observation Station 6

Query the list of *CITY* names starting with vowels (i.e., `a`, `e`, `i`, `o`, or `u`) from **STATION**. Your result *cannot* contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '^[aeiouAEIOU]';
```

Output:



12. Weather Observation Station 7

Query the list of *CITY* names ending with vowels (a, e, i, o, u) from **STATION**. Your result *cannot* contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

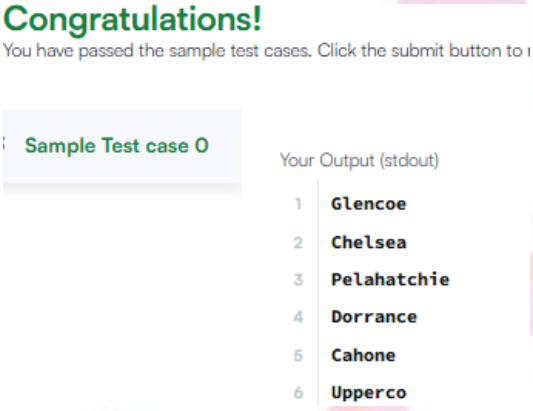
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '[aeiouAEIOU]$';
```

Output:



13. Weather Observation Station 8

Query the list of *CITY* names from **STATION** which have vowels (i.e., *a*, *e*, *i*, *o*, and *u*) as both their first *and* last characters. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP
'^[aeiouAEIOU].*[aeiouAEIOU]$';
```

Output:



14. Weather Observation Station 9

Query the list of *CITY* names from **STATION** that *do not start* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

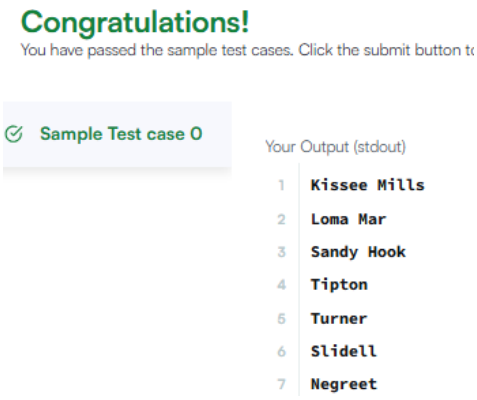
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '^[^aeiouAEIOU]';
```

Output:



15. Weather Observation Station 10

Query the list of *CITY* names from **STATION** that *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

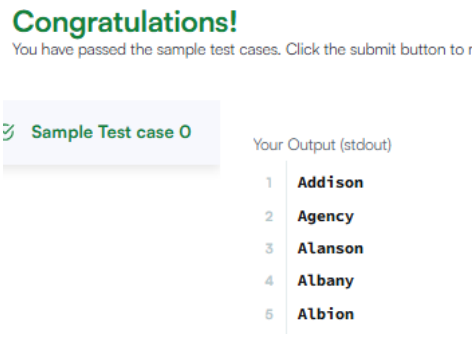
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE RIGHT(CITY, 1) NOT IN ('A', 'E', 'I',
'O', 'U') ORDER BY CITY;
```

Output:



16. Weather Observation Station 11

Query the list of *CITY* names from **STATION** that either do not start with vowels or do not end with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

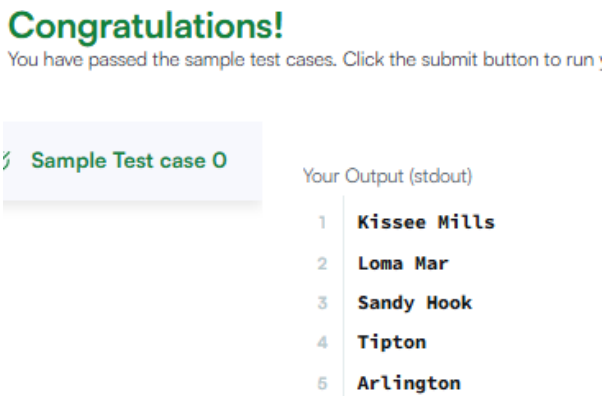
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY NOT REGEXP '^[aeiouAEIOU]' OR
CITY NOT REGEXP '[aeiouAEIOU]$';
```

Output:



17. Weather Observation Station 12

Query the list of *CITY* names from **STATION** that *do not start* with vowels and *do not end* with vowels. Your result cannot contain duplicates

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitud

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '^[^aeiouAEIOU]' AND CITY
REGEXP '[^aeiouAEIOU]$';
```

Output:



18. Higher Than 75 Marks

Query the *Name* of any student in **STUDENTS** who scored higher than *Marks*. Order your output by the *last three characters* of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending *ID*.

Input Format

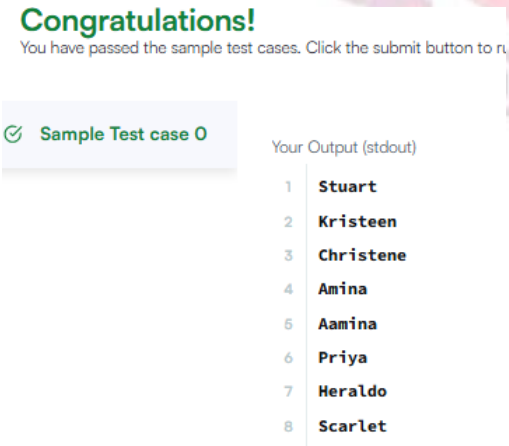
Column	*Type*
ID	*Integer*
Name	*String*
Marks	*Integer*

The **STUDENTS** table is described as follows: The *Name* column only contains uppercase (`A-Z`) and lowercase (`a-z`) letters.

Input:

```
SELECT NAME FROM STUDENTS WHERE MARKS > 75 ORDER BY RIGHT(NAME, 3), ID ASC;
```

Output:



19. Employee Names

Write a query that prints a list of employee names (i.e.: the *name* attribute) from the **Employee** table in alphabetical order.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is their monthly salary.

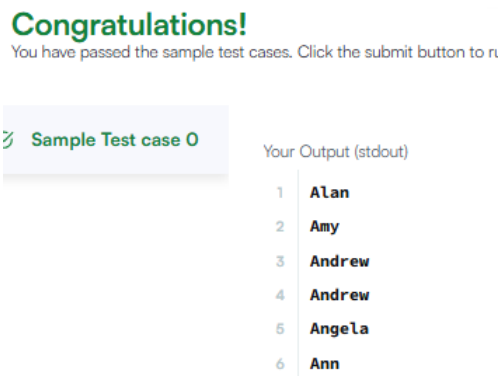
Sample Input

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Input:

```
SELECT name FROM Employee ORDER BY name;
```

Output:



20. Employee Salaries

Write a query that prints a list of employee names (i.e.: the *name* attribute) for employees in **Employee** having a salary greater than per month who have been employees for less than months. Sort your result by ascending *employee_id*.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Input:

```
SELECT name FROM Employee WHERE salary > 2000   AND months < 10 ORDER BY
employee_id ASC;
```

Output:



21. Type of Triangle

Write a query identifying the *type* of each record in the **TRIANGLES** table using its three side lengths. Output one of the following statements for each record in the table:

- **Equilateral**: It's a triangle with sides of equal length.
- **Isosceles**: It's a triangle with sides of equal length.
- **Scalene**: It's a triangle with sides of differing lengths.
- **Not A Triangle**: The given values of *A*, *B*, and *C* don't form a triangle.

Input Format

The **TRIANGLES** table is described as follows:

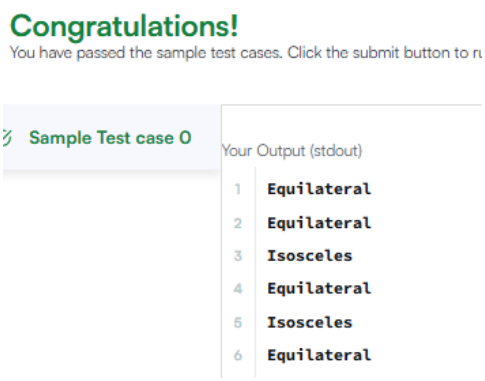
Column	*Type*
A	*Integer*
B	*Integer*
C	*Integer*

Each row in the table denotes the lengths of each of a triangle's three sides.

Input:

```
SELECT CASE  WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A Triangle
WHEN A = B AND B = C THEN 'Equilateral' WHEN A = B OR B = C OR A = C THEN
'Isosceles 'ELSE 'Scalene'END FROM TRIANGLES;
```

Output:



23. The PADS

Generate the following two result sets:

1. Query an *alphabetically ordered* list of all names in **OCCUPATIONS**, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: `AnActorName(A)`, `ADoctorName(D)`, `AProfessorName(P)`, and `ASingerName(S)`.
2. Query the number of ocurrences of each occupation in **OCCUPATIONS**. Sort the occurrences in *ascending order*, and output them in the following format:

   `There are a total of [occupation_count] [occupation]s.`

   where `[occupation_count]` is the number of occurrences of an occupation in **OCCUPATIONS** and `[occupation]` is the *lowercase* occupation name. If more than one *Occupation* has the same `[occupation_count]`, they should be ordered alphabetically.

Note: There will be at least two entries in the table for each type of occupation.

Input Format

The **OCCUPATIONS** table is described as follows:

Column	*Type*
Name	*String*
Occupation	*String*

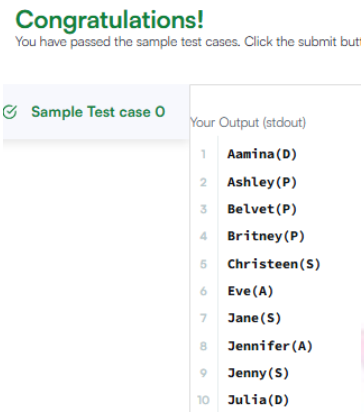
Occupation will only contain one of the following values: **Doctor**, **Professor**, **Singer** or **Actor**.

Input:

```
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')') FROM OCCUPATIONS ORDER BY
Name ASC;
SELECT CONCAT('There are a total of ', COUNT(Occupation), ' ',
LOWER(Occupation), 's.') FROM OCCUPATIONS
```

```
GROUP BY Occupation
ORDER BY COUNT(Occupation) ASC, Occupation ASC;
```

Output:



24. Occupations

Pivot the *Occupation* column in **OCCUPATIONS** so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. The output should consist of four columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Note: Print **NULL** when there are no more names corresponding to an occupation.

Input Format

The **OCCUPATIONS** table is described as follows:

Column	*Type*
Name	*String*
Occupation	*String*

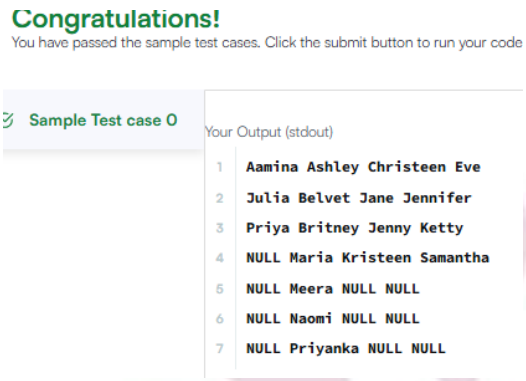
Occupation will only contain one of the following values: **Doctor**, **Professor**, **Singer** or **Actor**.

Input:

```
SELECT
    MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS Doctor,
    MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS Professor,
    MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS Singer,
    MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS Actor
FROM (
```

```
    SELECT
        Name,
        Occupation,
        ROW_NUMBER() OVER (PARTITION BY Occupation ORDER BY Name) AS rn
    FROM OCCUPATIONS
) AS sub
GROUP BY rn;
```

Output:



25. Binary Tree Nodes

You are given a table, *BST*, containing two columns: *N* and *P*, where *N* represents the value of a node in *Binary Tree*, and *P* is the parent of *N*.

Column	*Type*
N	*Integer*
P	*Integer*

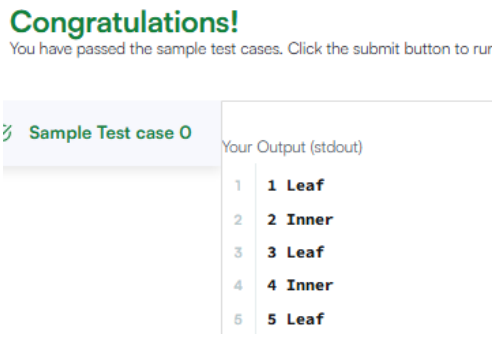
Write a query to find the node type of *Binary Tree* ordered by the value of the node. Output one of the following for each node:

- *Root*: If node is root node.
- *Leaf*: If node is leaf node.
- *Inner*: If node is neither root nor leaf node.

Input:

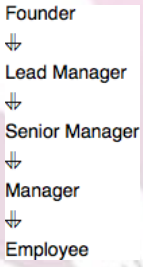
```
SELECT N, CASE WHEN P IS NULL THEN 'Root'
WHEN N IN (SELECT P FROM BST WHERE P IS NOT NULL) THEN 'Inner'
 ELSE 'Leaf' END FROM BST ORDER BY N;
```

Output:



26. New Companies

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:



Given the table schemas below, write a query to print the *company_code*, *founder* name, total number of *lead* managers, total number of *senior* managers, total number of *managers*, and total number of *employees*. Order your output by ascending *company_code*.

Note:

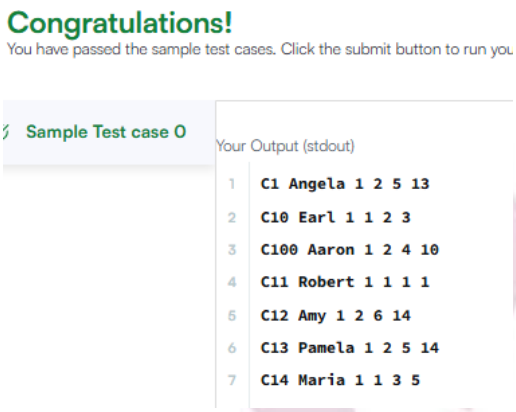
- The tables may contain duplicate records.
- The *company_code* is string, so the sorting should not be **numeric**. For example, if the *company_codes* are *C_1*, *C_2*, and *C_10*, then the ascending *company_codes* will be *C_1*, *C_10*, and *C_2*.

Input:

```
SELECT
    c.company_code,
    c.founder,
    COUNT(DISTINCT lm.lead_manager_code),
    COUNT(DISTINCT sm.senior_manager_code),
    COUNT(DISTINCT m.manager_code),
    COUNT(DISTINCT e.employee_code)
FROM Company c
```

```
LEFT JOIN Lead_Manager lm ON c.company_code = lm.company_code
LEFT JOIN Senior_Manager sm ON lm.lead_manager_code = sm.lead_manager_code
LEFT JOIN Manager m ON sm.senior_manager_code = m.senior_manager_code
LEFT JOIN Employee e ON m.manager_code = e.manager_code
GROUP BY c.company_code, c.founder
ORDER BY c.company_code ASC;
```

Output:



27. Revising Aggregations - The Count Function

Query a *count* of the number of cities in **CITY** having a *Population* larger than 100,000.

Input Format

The **CITY** table is described as follows:

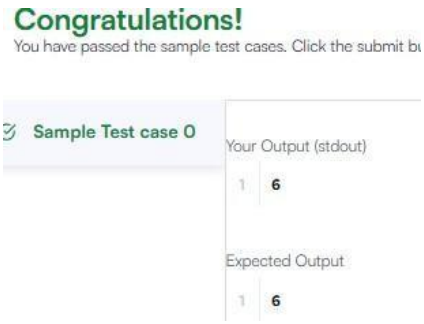
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT COUNT(*) FROM CITY WHERE POPULATION > 100000;
```

Output:



28. Revising Aggregations - Averages

Query the average population of all cities in **CITY** where *District* is **California**.

Input Format

The **CITY** table is described as follows:

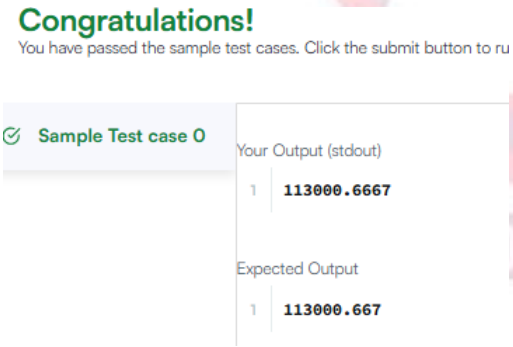
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT AVG(POPULATION) FROM CITY WHERE DISTRICT = 'California';
```

Output:



29. Average Population

Query the average population for all cities in **CITY**, rounded *down* to the nearest integer.

Input Format

The **CITY** table is described as follows:

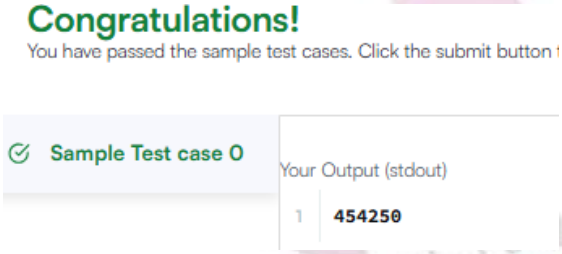
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT FLOOR(AVG(POPULATION)) FROM CITY;
```

Output:



30. Japan Population

Query the sum of the populations for all Japanese cities in **CITY**. The *COUNTRYCODE* for Japan is **JPN**.

Input Format

The **CITY** table is described as follows:

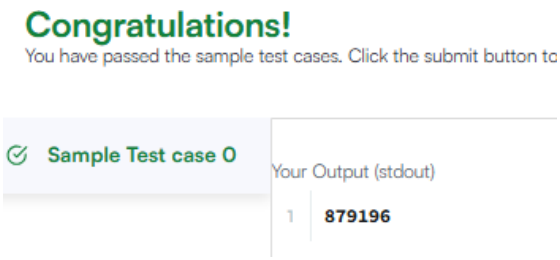
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT SUM(POPULATION) FROM CITY WHERE COUNTRYCODE = 'JPN';
```

Output:



31. Population Density Difference

Input Format

The **CITY** table is described as follows:

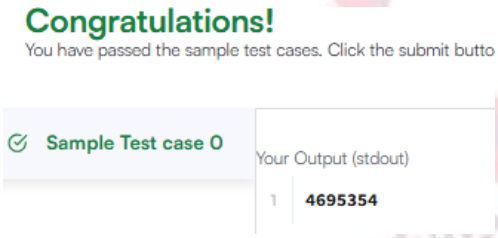
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT MAX(POPULATION) - MIN(POPULATION) FROM CITY;
```

Output:



32. The Blunder

Samantha was tasked with calculating the average monthly salaries for all employees in the **EMPLOYEES** table, but did not realize her keyboard's key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.

Input Format

The **EMPLOYEES** table is described as follows:

Column	Type
ID	Integer
Name	String
Salary	Integer

Note: *Salary* is per month.

Input:

```
SELECT CEIL(AVG(SALARY) - AVG(REPLACE(SALARY, '0', ''))) FROM EMPLOYEES;
```

Output:



33. Top Earners

We define an employee's *total earnings* to be their monthly worked, and the *maximum total earnings* to be the maximum total earnings for any employee in the **Employee** table. Write a query to find the *maximum total earnings* for all employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

Input:

```
SELECT (salary * months) AS earnings, COUNT(*) FROM Employee GROUP BY
earnings ORDER BY earnings DESC LIMIT 1;
```

Output:



34. Weather Observation Station 2

Query the following two values from the **STATION** table:

1. The sum of all values in *LAT_N* rounded to a scale of decimal places.
2. The sum of all values in *LONG_W* rounded to a scale of decimal places.

Input Format

The **STATION** table is described as follows:

STATION

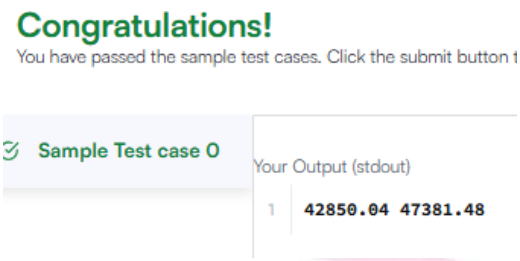
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(SUM(LAT_N), 2), ROUND(SUM(LONG_W), 2) FROM STATION;
```

Output:



35. Weather Observation Station 13

Query the sum of *Northern Latitudes* (*LAT_N*) from **STATION** having values greater than and less than . Truncate your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(SUM(LAT_N), 4) FROM STATION WHERE LAT_N > 38.7880 AND LAT_N <
137.2345;
```

Output:



35. Weather Observation Station 14

Query the greatest value of the *Northern Latitudes* (*LAT_N*) from **STATION** that is less than . Truncate your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

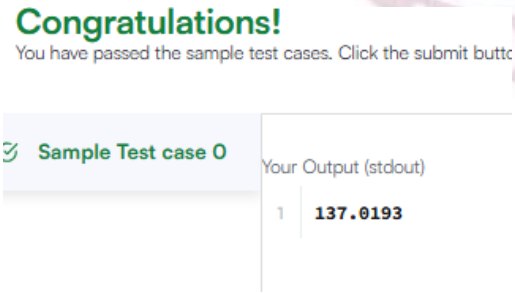
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(MAX(LAT_N), 4) FROM STATION WHERE LAT_N < 137.2345;
```

Output:



36. Weather Observation Station 15

Query the *Western Longitude* (*LONG_W*) for the largest *Northern Latitude* (*LAT_N*) in **STATION** that is less than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(LONG_W, 4) FROM STATION WHERE LAT_N = (
SELECT MAX(LAT_N) FROM STATION WHERE LAT_N < 137.2345 );
```

Output:



37. Weather Observation Station 16

Query the smallest *Northern Latitude* (*LAT_N*) from **STATION** that is greater than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(MIN(LAT_N), 4) FROM STATION WHERE LAT_N > 38.7780;
```

Output:



38. Weather Observation Station 17

Query the *Western Longitude* (*LONG_W*)where the smallest *Northern Latitude* (*LAT_N*) in **STATION** is greater than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(LONG_W, 4) FROM STATION WHERE LAT_N = (
SELECT MIN(LAT_N) FROM STATION WHERE LAT_N > 38.7780
);
```

Output:



39. Weather Observation Station 18

Consider $P_1(a, b)$ and $P_2(c, d)$ to be two points on a 2D plane.

- a happens to equal the minimum value in Northern Latitude (LAT_N in **STATION**).
- b happens to equal the minimum value in Western Longitude (LONG_W in **STATION**).
- c happens to equal the maximum value in Northern Latitude (LAT_N in **STATION**).
- d happens to equal the maximum value in Western Longitude (LONG_W in **STATION**).

Query the Manhattan Distance between points P_1 and P_2 and round it to a scale of 4 decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

Input:

```
SELECT ROUND(ABS(MAX(LAT_N) - MIN(LAT_N)) + ABS(MAX(LONG_W) - MIN(LONG_W)),4)
FROM STATION;
```

Output:



40. Weather Observation Station 19

Consider and to be two points on a 2D plane where are the respective minimum and maximum values of *Northern Latitude* (*LAT_N*) and are the respective minimum and maximum values of *Western Longitude* (*LONG_W*) in **STATION**.

Query the Euclidean Distance between points and and *format your answer* to display decimal digits.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(SQRT(POWER(MAX(LAT_N) - MIN(LAT_N), 2) +
POWER(MAX(LONG_W)MIN(LONG_W), 2)), 4) FROM STATION;
```

Output:



41. Weather Observation Station 20

A *median* is defined as a number separating the higher half of a data set from the lower half. Query the *median* of the *Northern Latitudes* (*LAT_N*) from **STATION** and round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:
```
SELECT ROUND(S.LAT_N, 4)
FROM STATION AS S
WHERE (SELECT COUNT(LAT_N) FROM STATION WHERE LAT_N < S.LAT_N) = (SELECT
COUNT(LAT_N) FROM STATION WHERE LAT_N > S.LAT_N);
```

Output:



42. Population Census

Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the *CONTINENT* is *'Asia'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

COUNTRY

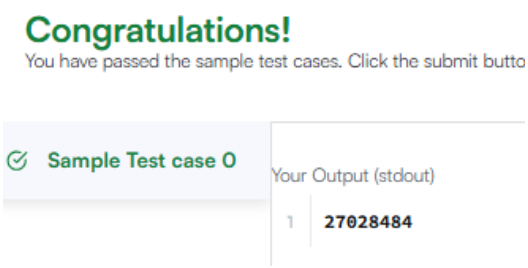
Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Input:

```
SELECT SUM(CITY.POPULATION)
 FROM CITY JOIN COUNTRY
```

```
    ON CITY.COUNTRYCODE = COUNTRY.CODE
WHERE COUNTRY.CONTINENT = 'Asia';
```

Output:



43. African Cities

Given the **CITY** and **COUNTRY** tables, query the names of all cities where the *CONTINENT* is *'Africa'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

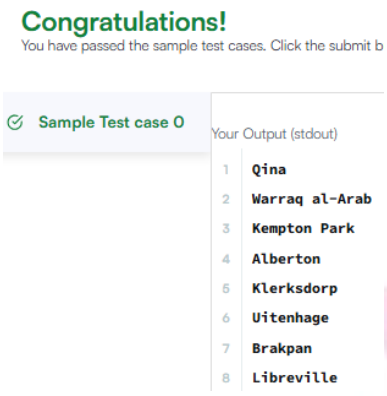
COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Input:

```
SELECT CITY.NAME FROM CITYINNER JOIN COUNTRYON CITY.CountryCode =
COUNTRY.Code WHERE COUNTRY.Continent = 'Africa';
```

Output:



44. Average Population of Each Continent

Given the **CITY** and **COUNTRY** tables, query the names of all the continents (*COUNTRY.Continent*) and their respective average city populations (*CITY.Population*) rounded *down* to the nearest integer.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

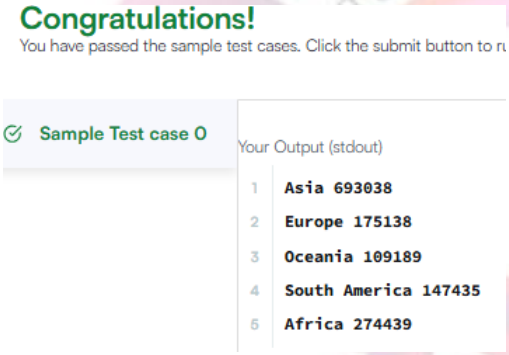
COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Input:

```
SELECT COUNTRY.Continent, FLOOR(AVG(CITY.Population))FROM CITY
INNER JOIN COUNTRY ON CITY.CountryCode = COUNTRY.Code
GROUP BY COUNTRY.Continent;
```

Output:



45. The Report

You are given two tables: *Students* and *Grades*. *Students* contains three columns *ID*, *Name* and *Marks*.

Column	Type
ID	Integer
Name	String
Marks	Integer

Grades contains the following data:

Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89
10	90	100

Ketty gives *Eve* a task to generate a report containing three columns: *Name*, *Grade* and *Mark*. *Ketty* doesn't want the NAMES of those students who received a grade lower than *8*. The report must be in descending order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their name alphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.

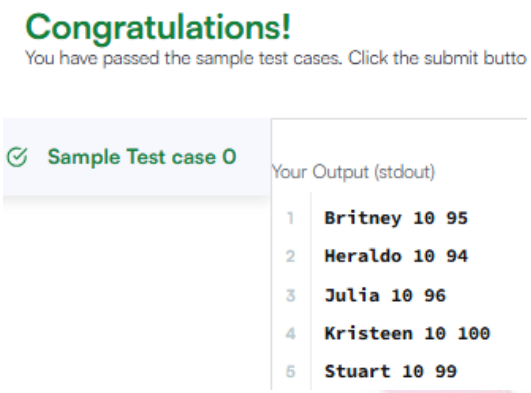
Write a query to help Eve.

Input:

```
SELECT
    CASE
        WHEN g.Grade < 8 THEN NULL
        ELSE s.Name
    END AS Name,
    g.Grade,
    s.Marks
FROM Students s
JOIN Grades g
  ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY
    g.Grade DESC,
```

```
    CASE WHEN g.Grade >= 8 THEN s.Name END ASC,
    CASE WHEN g.Grade < 8 THEN s.Marks END ASC;
```

Output:



46. Top Competitors

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective *hacker_id* and *name* of hackers who achieved full scores for *more than one* challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending *hacker_id*.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Difficulty:* The *difficult_level* is the level of difficulty of the challenge, and *score* is the maximum score that can be achieved for a challenge at that difficulty level.

Column	Type
difficulty_level	Integer
score	Integer

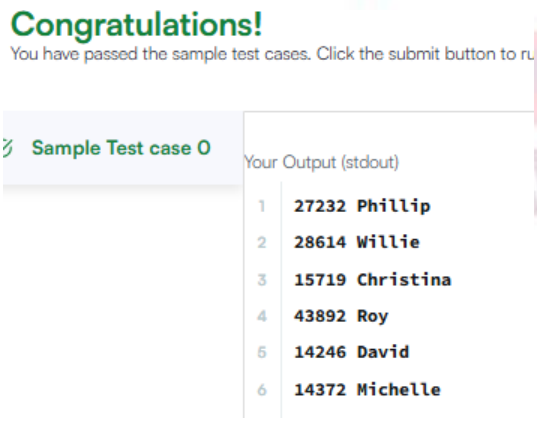
- *Challenges:* The *challenge_id* is the id of the challenge, the *hacker_id* is the id of the hacker who created the challenge, and *difficulty_level* is the level of difficulty of the challenge.

Column	Type
challenge_id	Integer
hacker_id	Integer
difficulty_level	Integer

Input:

```
SELECT
    h.hacker_id,
    h.name
FROM submissions s
INNER JOIN hackers h
    ON s.hacker_id = h.hacker_id
INNER JOIN challenges c
    ON s.challenge_id = c.challenge_id
INNER JOIN difficulty d
    ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(s.challenge_id) > 1
ORDER BY COUNT(s.challenge_id) DESC, h.hacker_id ASC;
```

Output:



47. Ollivander's Inventory

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each *non-evil* wand of high power and age. Write a query to print the *id*, *age*, *coins_needed*, and *power* of the wands that Ron's interested in, sorted in order of descending *power*. If more than one wand has same power, sort the result in order of descending *age*.

Input Format

The following tables contain data on the wands in Ollivander's inventory:

- *Wands:* The *id* is the id of the wand, *code* is the code of the wand, *coins_needed* is the total number of gold galleons needed to buy the wand, and *power* denotes the quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

- *Wands_Property:* The *code* is the code of the wand, *age* is the age of the wand, and *is_evil* denotes whether the wand is good for the dark arts. If the value of *is_evil* is *0*, it means that the wand is not evil. The mapping between *code* and *age* is one-one, meaning that if there are two pairs, and , then and .

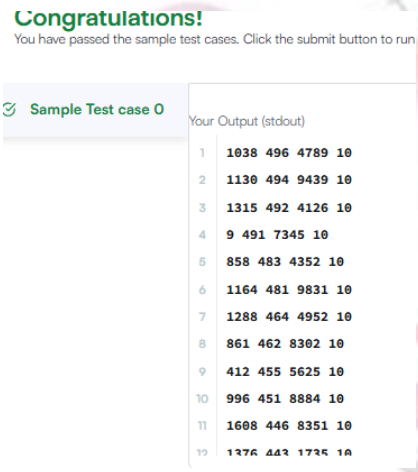
Column	Type
code	Integer
age	Integer
is_evil	Integer

Input:

```
SELECT
    w.id,
    p.age,
    w.coins_needed,
    w.power
FROM
    wands AS w
JOIN
```

```
    wands_property AS p ON w.code = p.code
WHERE
    p.is_evil = 0
    AND w.coins_needed = (
        SELECT
            MIN(w1.coins_needed)
        FROM
            wands AS w1
        JOIN
            wands_property AS p1 ON w1.code = p1.code
        WHERE
            p1.is_evil = 0
            AND w1.power = w.power
            AND p1.age = p.age
    )
ORDER BY
    w.power DESC,
    p.age DESC;
```

Output:



48. Challenges

Julia asked her students to create some coding challenges. Write a query to print the *hacker_id*, *name*, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by *hacker_id*. If more than one student created the

same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Challenges:* The *challenge_id* is the id of the challenge, and *hacker_id* is the id of the student who created the challenge.

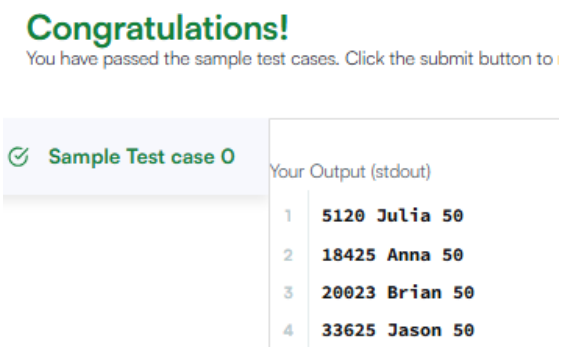
Column	Type
challenge_id	Integer
hacker_id	Integer

Input:

```
SELECT H.HACKER_ID,
       H.NAME,
       COUNT(C.CHALLENGE_ID) AS C_COUNT
FROM HACKERS H
JOIN CHALLENGES C ON C.HACKER_ID = H.HACKER_ID
GROUP BY H.HACKER_ID, H.NAME
HAVING C_COUNT =
    (SELECT COUNT(C2.CHALLENGE_ID) AS C_MAX
     FROM CHALLENGES AS C2
     GROUP BY C2.HACKER_ID
     ORDER BY C_MAX DESC LIMIT 1)
OR C_COUNT IN
    (SELECT DISTINCT C_COMPARE AS C_UNIQUE
     FROM (SELECT H2.HACKER_ID,
                  H2.NAME,
                  COUNT(CHALLENGE_ID) AS C_COMPARE
           FROM HACKERS H2
           JOIN CHALLENGES C ON C.HACKER_ID = H2.HACKER_ID
           GROUP BY H2.HACKER_ID, H2.NAME) COUNTS
     GROUP BY C_COMPARE
     HAVING COUNT(C_COMPARE) = 1)
```

```
ORDER BY C_COUNT DESC, H.HACKER_ID;
```

Output:



49. Contest Leaderboard

You did such a great job helping Julia with her last oding contest challenge that she wants you to work on this one, too.The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the *hacker_id*, *name*, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending *hacker_id*. Exclude all hackers with a total score of from your result.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

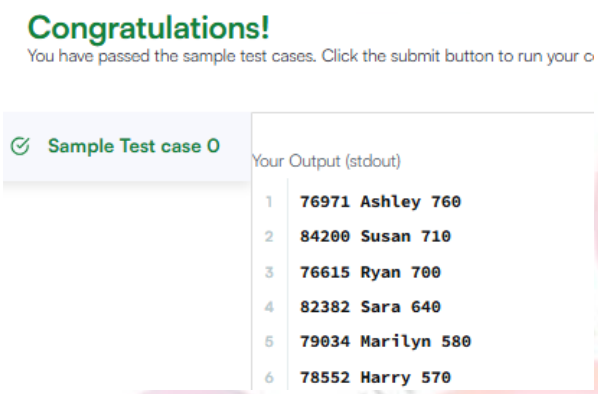
- *Submissions:* The *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, *challenge_id* is the id of the challenge for which the submission belongs to, and *score* is the score of the submission.

Column	Type
submission_id	Integer
hacker_id	Integer
challenge_id	Integer
score	Integer

Input:

```sql
SELECT X.hacker_id,
(SELECT H.NAME FROM HACKERS H
                    WHERE H.HACKER_ID = X.HACKER_ID) NAME,
SUM(X.SCORE) TOTAL_SCORE FROM
  (SELECT HACKER_ID, MAX(SCORE) SCORE FROM SUBMISSIONS S
   GROUP BY 1, S.CHALLENGE_ID) X
GROUP BY 1
HAVING TOTAL_SCORE > 0
ORDER BY TOTAL_SCORE DESC, HACKER_ID;
```

Output:



50. SQL Project Planning

You are given a table, *Projects*, containing three columns: *Task_ID*, *Start_Date* and *End_Date*. It is guaranteed that the difference between the *End_Date* and the *Start_Date* is equal to *1* day for each row in the table.

Column	*Type*
Task_ID	*Integer*
Start_Date	*Date*
End_Date	*Date*

If the *End_Date* of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

Sample Input

Task_ID	Start_Date	End_Date
1	2015-10-01	2015-10-02
2	2015-10-02	2015-10-03
3	2015-10-03	2015-10-04
4	2015-10-13	2015-10-14
5	2015-10-14	2015-10-15
6	2015-10-28	2015-10-29
7	2015-10-30	2015-10-31

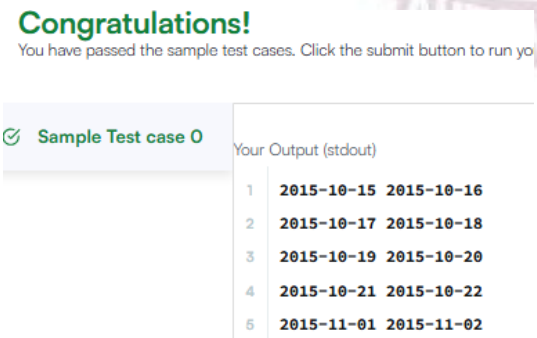
Sample Output

```
2015-10-28  2015-10-29
2015-10-30  2015-10-31
2015-10-13  2015-10-15
2015-10-01  2015-10-04
```

Input:

```
SELECT Start_Date, MIN(End_Date) AS Min_End_Date
FROM (
    SELECT Start_Date
    FROM Projects
    WHERE Start_Date NOT IN (SELECT End_Date FROM Projects)
) AS a
JOIN (
    SELECT End_Date
    FROM Projects
    WHERE End_Date NOT IN (SELECT Start_Date FROM Projects)
) AS b ON a.Start_Date < b.End_Date
GROUP BY Start_Date
ORDER BY DATEDIFF(MIN(End_Date), Start_Date) ASC, Start_Date ASC;
```

Output:



51. Placements

You are given three tables: *Students*, *Friends* and *Packages*. *Students* contains two columns: *ID* and *Name*. *Friends* contains two columns: *ID* and *Friend_ID* (*ID* of the ONLY best friend). *Packages* contains two columns: *ID* and *Salary* (offered salary in $ thousands per month).

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

Friends

Column	Type
ID	Integer
Salary	Float

Packages

Write a query to output the names of those students whose best friends got offered a higher salary than them. Names must be ordered by the salary amount offered to the best friends. It is guaranteed that no two students got same salary offer.

Sample Input

ID	Friend_ID
1	2
2	3
3	4
4	1

Friends

ID	Name
1	Ashley
2	Samantha
3	Julia
4	Scarlet

Students

ID	Salary
1	15.20
2	10.06
3	11.55
4	12.12

Packages

Sample Output

```
Samantha
Julia
Scarlet
```

Explanation

See the following table:

ID	1	2	3	4
Name	Ashley	Samantha	Julia	Scarlet
Salary	15.20	10.06	11.55	12.12
Friend ID	2	3	4	1
Friend Salary	10.06	11.55	12.12	15.20

Now,

- *Samantha's* best friend got offered a higher salary than her at 11.55
- *Julia's* best friend got offered a higher salary than her at 12.12
- *Scarlet's* best friend got offered a higher salary than her at 15.2
- *Ashley's* best friend did NOT get offered a higher salary than her

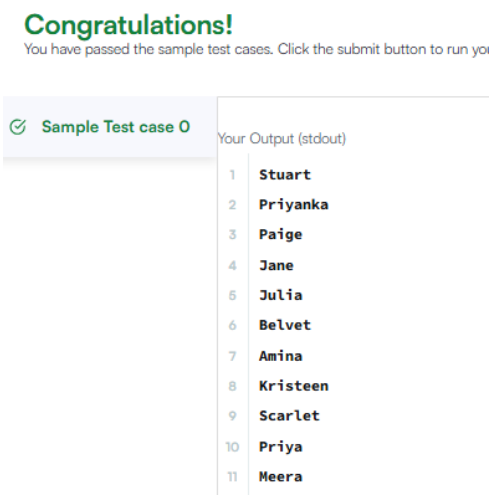
The name output, when ordered by the salary offered to their friends, will be:

- *Samantha*
- *Julia*
- *Scarlet*

Input:

```
SELECT S.NAME
FROM STUDENTS S
JOIN FRIENDS F ON S.ID = F.ID
JOIN PACKAGES P1 ON S.ID = P1.ID
JOIN PACKAGES P2 ON F.FRIEND_ID = P2.ID
WHERE P2.SALARY > P1.SALARY
ORDER BY P2.SALARY;
```

Output:



52. Symmetric Pairs

You are given a table, *Functions*, containing two columns: *X* and *Y*.

Column	*Type*
X	*Integer*
Y	*Integer*

Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be *symmetric pairs* if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such *symmetric pairs* in ascending order by the value of *X*. List the rows such that $X_1 \leq Y_1$.

Sample Input

X	*Y*
20	*20*
20	*20*
20	*21*
23	*22*
22	*23*
21	*20*

Input:

```
SELECT f1.X, f1.Y FROM Functions f1
INNER JOIN Functions f2 ON f1.X = f2.Y AND f1.Y = f2.X
GROUP BY f1.X, f1.Y
```

```
HAVING f1.X < f1.Y OR COUNT(f1.X) > 1
ORDER BY f1.X;
```

Output:



53. 15 Days of Learning SQL

Julia conducted a days of learning SQL contest. The start date of the contest was *March 01, 2016* and the end date was *March 15, 2016*.

Write a query to print total number of unique hackers who made at least submission each day (starting on the first day of the contest), and find the *hacker_id* and *name* of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest *hacker_id*. The query should print this information for each day of the contest, sorted by the date.

Input Format

The following tables hold contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Submissions:* The *submission_date* is the date of the submission, *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, and *score* is the score of the submission.

Column	Type
submission_date	Date
submission_id	Integer
hacker_id	Integer
score	Integer

Sample Input

For the following sample input, assume that the end date of the contest was *March 06, 2016*.

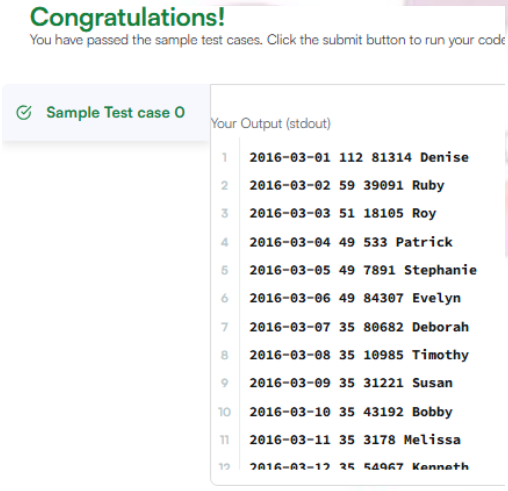
submission_date	submission_id	hacker_id	score
2016-03-01	8494	20703	0
2016-03-01	22403	53473	15
2016-03-01	23965	79722	60
2016-03-01	30173	36396	70
2016-03-02	34928	20703	0
2016-03-02	38740	15758	60
2016-03-02	42769	79722	25
2016-03-02	44364	79722	60
2016-03-03	45440	20703	0
2016-03-03	49050	36396	70
2016-03-03	50273	79722	5
2016-03-04	50344	20703	0
2016-03-04	51360	44065	90
2016-03-04	54404	53473	65
2016-03-04	61533	79722	45
2016-03-05	72852	20703	0
2016-03-05	74546	38289	0
2016-03-05	76487	62529	0
2016-03-05	82439	36396	10
2016-03-05	90006	36396	40
2016-03-06	90404	20703	0

Input:

```
SELECT SUBMISSION_DATE,
(SELECT COUNT(DISTINCT HACKER_ID)
```

```
 FROM SUBMISSIONS S2
 WHERE S2.SUBMISSION_DATE = S1.SUBMISSION_DATE AND
(SELECT COUNT(DISTINCT S3.SUBMISSION_DATE)
 FROM SUBMISSIONS S3 WHERE S3.HACKER_ID = S2.HACKER_ID AND S3.SUBMISSION_DATE
< S1.SUBMISSION_DATE) = DATEDIFF(S1.SUBMISSION_DATE , '2016-03-01')),
(SELECT HACKER_ID FROM SUBMISSIONS S2 WHERE S2.SUBMISSION_DATE =
S1.SUBMISSION_DATE
GROUP BY HACKER_ID ORDER BY COUNT(SUBMISSION_ID) DESC, HACKER_ID LIMIT 1) AS
TMP,
(SELECT NAME FROM HACKERS WHERE HACKER_ID = TMP)
FROM
(SELECT DISTINCT SUBMISSION_DATE FROM SUBMISSIONS) S1
GROUP BY SUBMISSION_DATE;
```

Output:



54. Draw The Triangle 1

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

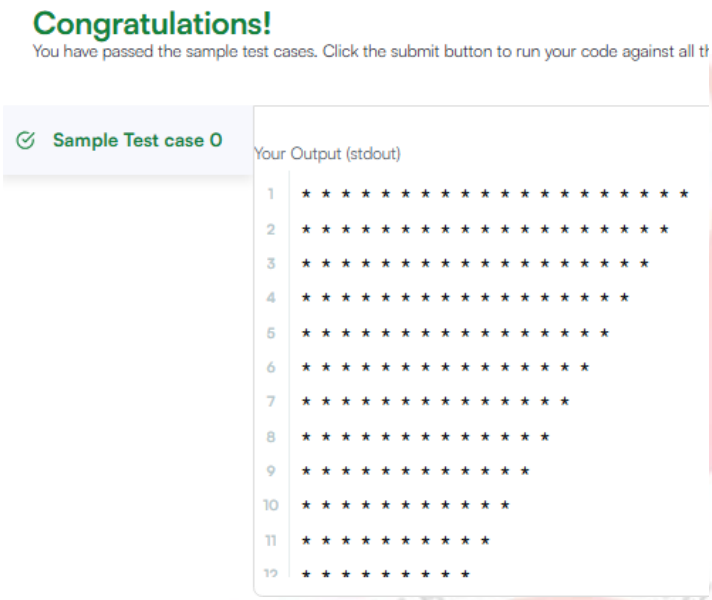
```
* * * * *
* * * *
* * *
* *
*
```

Write a query to print the pattern *P(20)*.

Input:

```
SET @TEMP := 21;
SELECT REPEAT('* ', @TEMP := @TEMP - 1)
FROM INFORMATION_SCHEMA.TABLES;
```

Output:



55. Draw The Triangle 2

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

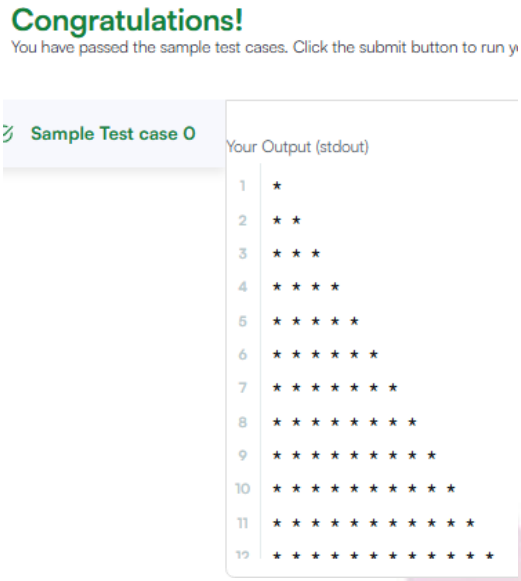
```
*
* *
* * *
* * * *
* * * * *
```

Write a query to print the pattern *P(20)*.

Input:

```
SET @number = 0;
SELECT REPEAT('* ', @number := @number + 1)
FROM information_schema.tables
WHERE @number < 20;
```

Output:



56. Print Prime Numbers

Write a query to print all *prime numbers* less than or equal to . Print your result on a single line, and use the ampersand () character as your separator (instead of a space).

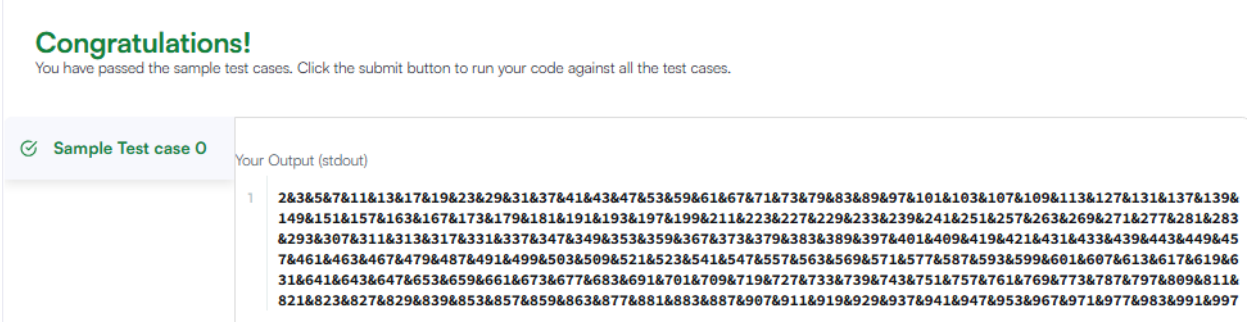
For example, the output for all prime numbers would be:

2&3&5&7

Input:

```
WITH RECURSIVE nums AS (
    SELECT 2 AS n
    UNION ALL
    SELECT n + 1 FROM nums WHERE n < 1000
)
SELECT GROUP_CONCAT(n SEPARATOR '&')
FROM nums p
WHERE NOT EXISTS (
    SELECT 1 FROM nums d WHERE d.n <= SQRT(p.n) AND d.n > 1 AND p.n % d.n = 0
);
```

Output:



57. Interviews

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the *contest_id*, *hacker_id*, *name*, and the sums of *total_submissions*, *total_accepted_submissions*, *total_views*, and *total_unique_views* for each contest sorted by *contest_id*. Exclude the contest from the result if all four sums are .

Note: A specific contest can be used to screen candidates at more than one college, but each college only holds screening contest.

Input Format

The following tables hold interview data:

- *Contests:* The *contest_id* is the id of the contest, *hacker_id* is the id of the hacker who created the contest, and *name* is the name of the hacker.

Column	Type
contest_id	Integer
hacker_id	Integer
name	String

- *Colleges:* The *college_id* is the id of the college, and *contest_id* is the id of the contest that Samantha used to screen the candidates.

Column	Type
college_id	Integer
contest_id	Integer

- *Challenges:* The *challenge_id* is the id of the challenge that belongs to one of the contests whose contest_id Samantha forgot, and *college_id* is the id of the college where the challenge was given to candidates.

Column	Type
challenge_id	Integer
college_id	Integer

- *View_Stats:* The *challenge_id* is the id of the challenge, *total_views* is the number of times the challenge was viewed by candidates, and *total_unique_views* is the number of times the challenge was viewed by unique candidates.

Column	Type
challenge_id	Integer
total_views	Integer
total_unique_views	Integer

- *Submission_Stats:* The *challenge_id* is the id of the challenge, *total_submissions* is the number of submissions for the challenge, and *total_accepted_submission* is the number of submissions that achieved full scores.

Column	Type
challenge_id	Integer
total_submissions	Integer
total_accepted_submissions	Integer

Input:

```
SELECT t1.contest_id, t1.hacker_id, t1.name,
       SUM(ts) as ts, SUM(tas) as tas, SUM(tv) as tv, SUM(tuv) as tuv
FROM CONTESTS AS t1
LEFT JOIN COLLEGES AS t2
    ON t1.contest_id = t2.contest_id
LEFT JOIN CHALLENGES AS t3
    ON t2.college_id = t3.college_id
LEFT JOIN (SELECT challenge_id,
           SUM(total_views) AS tv, SUM(total_unique_views) as tuv
           FROM View_Stats
           GROUP BY challenge_id) AS t4
    ON t3.challenge_id = t4.challenge_id
LEFT JOIN (SELECT challenge_id,
           SUM(total_submissions) AS ts, SUM(total_accepted_submissions) as
tas
           FROM Submission_Stats
           GROUP BY challenge_id) as t5
    ON t3.challenge_id = t5.challenge_id
GROUP BY t1.contest_id, t1.hacker_id, t1.name
HAVING (ts + tas + tv + tuv > 0)
ORDER BY t1.contest_id;
```

Output:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all

Sample Test case 0

Your Output (stdout)

```
845 579 Rose 1987 580 1635 566
858 1053 Angela 703 160 1002 384
883 1055 Frank 1121 319 1217 338
1793 2655 Patrick 1337 360 1216 412
2374 2765 Lisa 2733 815 3368 904
2963 2845 Kimberly 4306 1221 3603 1184
3584 2873 Bonnie 2492 652 3019 954
4044 3067 Michael 1323 449 1722 528
4249 3116 Todd 1452 376 1767 463
4269 3256 Joe 1018 372 1766 530
4483 3386 Earl 1911 572 1644 477
4541 3608 Robert 1886 516 1694 504
```



1. ALL PRODUCTS

Task:

Write a query to output `Products` table.

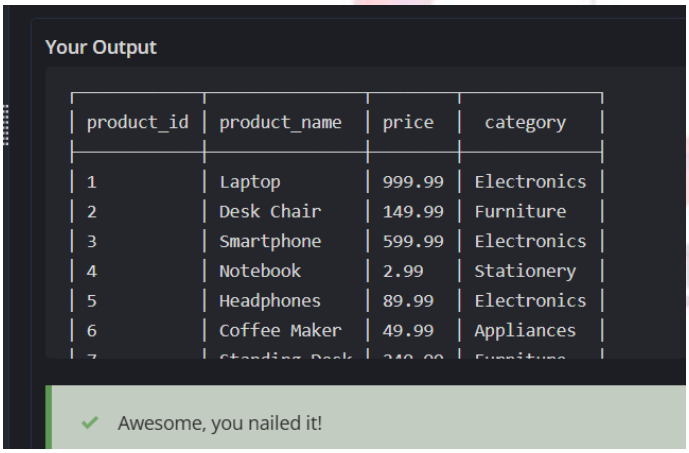
Table Name: `Products`

Product_ID	Product_Name	Price	Category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

Input:

```
SELECT * FROM Products;
```

Output:



2. HIGH PRICE OF PRODUCTS

Task:

Write a query to find all `product_name` and `category` that have a price greater than 100.00 from the **Products** table.

Table Name: `Products`

Product_ID	Product_Name	Price	Category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

Expected Columns:

- product_name
- category

Input:

```
SELECT product_name, category FROM Products WHERE price > 100.00;
```

Output:

Your Output

product_name	category
Laptop	Electronics
Desk Chair	Furniture
Smartphone	Electronics
Standing Desk	Furniture
Tablet	Electronics

3. AVERAGE SALARY

Task:

Write a query to calculate the average salary across all companies combined. Rename the column as `avg_salary`.

Table Name: `Works`

Employee_ID	Company_Name	Salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

Expected Column:

- avg_salary

Input:

```
SELECT AVG(salary) AS avg_salary FROM Works;
```

Output:

Your Output

avg_salary
62600.155

4. LOCATE PEOPLE

Task:

Write a query to retrieve the `department_name` and `location` of departments located in a city that starts with 'S'.

Table Name: `departments`

Department_ID	Department_Name	Location
1	Human Resources	New York
2	Research and Development	San Francisco
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston
6	Finance	Austin
7	IT Support	Seattle
8	Product Management	San Francisco
9	Quality Assurance	Los Angeles
10	Legal	Bostn

Expected Columns:

- department_name
- location

Input:

```
SELECT department_name, location FROM departments WHERE location LIKE 'S%';
```

Output:

Your Output

department_name	location
Research and Development	San Francisco
IT Support	Seattle
Product Management	San Francisco

5. DISTINCT COMPANIES

Task:

Write a query to select all the distinct companies (`company_name`) in the `Works` table.

Table Name: `Works`

Employee_ID	Company_Name	Salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

Expected Column:

- company name

Input:

```
SELECT DISTINCT company_name FROM Works;
```

Output:

Your Output

```
company_name
TechCorp
InnovateLtd
HealthPlus
EduWorks
GreenTech
```

6. FICTION COLLECTION SIZE

Task:

Write a query to find the total count of books whose genre is **Fiction**.

Note: Output column name should be **fiction_count**.

Table Name: `Books`

ID	Title	Author	Genre	Price	Published_Year
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	10.99	1925
2	1984	George Orwell	Dystopian	15.99	1949
3	To Kill a Mockingbird	Harper Lee	Fiction	12.99	1960
4	The Catcher in the Rye	J.D. Salinger	Fiction	14.99	1951
5	Brave New World	Aldous Huxley	Dystopian	13.99	1932
6	The Hobbit	J.R.R. Tolkien	Fantasy	9.99	1937
7	Moby Dick	Herman Melville	Fiction	18.50	1851
8	War and Peace	Leo Tolstoy	Historical	20.00	1869
9	The Picture of Dorian Gray	Oscar Wilde	Fiction	11.50	1890
10	The Alchemist	Paulo Coelho	Fiction	16.00	1988
11	Fahrenheit 451	Ray Bradbury	Dystopian	12.50	1953
12	The Chronicles of Narnia	C.S. Lewis	Fantasy	14.00	1950
13	The Handmaid's Tale	Margaret Atwood	Dystopian	15.50	1985
14	A Tale of Two Cities	Charles Dickens	Historical	13.00	1859
15	Little Women	Louisa May Alcott	Fiction	9.50	1868

Expected Column:

- fiction_count
- **Input:**

```
SELECT COUNT(*) AS fiction_count FROM Books WHERE genre = 'Fiction';
```

Output:

Your Output

```
┌─────────────┐
│ fiction_count │
├─────────────┤
│ 7           │
└─────────────┘
```

7. LIST OF MOVIES WITH RATINGS

Task:

Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

Table Name: `Cinema`

Movie_ID	Movie_Name	Description	Rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	Irish	boring	6.2
4	Ice Song	Fantasy	8.6
5	House Card	Interesting	9.1
6	The Escape	Thriller	7.8
7	Solar Flare	Sci-Fi	8.3
8	The Joker	Drama	9.0
9	Lost Dreams	Mystery	7.5
10	Galaxy War	Action	8.7

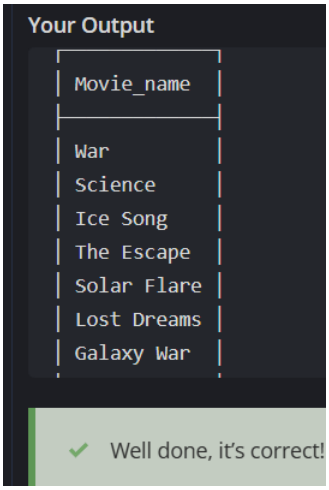
Expected Column:

- Movie_name

Input:

```
SELECT Movie_name FROM Cinema WHERE Rating > 7.0 AND Rating < 9.0;
```

Output:



8. HANDLING NULL VALUES

Task:

Write a query to retrieve `book_id`, `title`, `author` and `published_year` of the books which have **NULL** rating for their books.

Table Name: `Library`

Book_ID	Title	Author	Published_Year	Rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

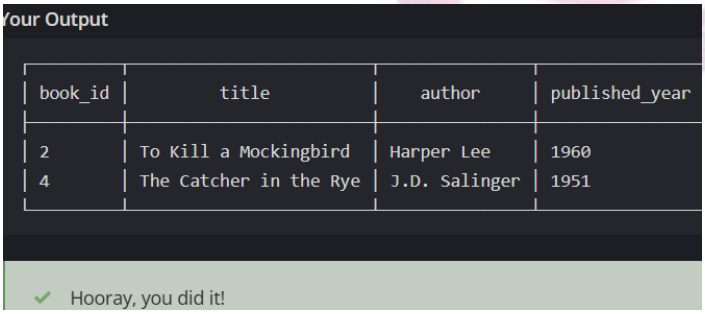
Expected Columns:

- book_id
- title
- author
- published_year

Input:

```sql
SELECT book_id, title, author, published_year FROM Library WHERE  rating IS NULL;
```

Output:



9. SALARY OF EMPLOYEES

Task:

Create a query to retrieve the `employee_name`, `company`, and `salary` for employees in the full-time category, ordered by salary in **descending** order.

Table Name: `Employees`

Employee_ID	Employee_Name	Company	Category	Department	Salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

Expected Columns:

- employee_name
- company
- salary

Input:

```
SELECT employee_name, company, salary FROM Employees WHERE category = 'Full-Time' ORDER BY salary DESC;
```

Output:

Your Output

employee_name	company	salary
Bob Brown	FinServ	90000
David Green	TechCorp	85000
John Smith	TechCorp	80000
Grace Grey	TechCorp	70000
Frank Black	HealthPlus	60000

10. DEPARTMENT OF EACH EMPLOYEE

Task:

Write a query to group the employees by their department and display the total number of employees (as `total_employees`) in each department.

Table Name: `Employees`

Employee_ID	Employee_Name	Company	Category	Department	Salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	Sales	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

Expected Column:

- department
- total_employees

Input:

```
SELECT department, COUNT(*) AS total_employees FROM Employees GROUP BY department;
```

Output:

Your Output

department	total_employees
Engineering	2
Finance	2
HR	2
IT	1
Marketing	2
Sales	1

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

11. ARTICLE VIEWS

Task:

Write a query to retrieve the `author_id`, `author_name`, and `publication_name` for authors whose articles got zero views. The result should be sorted by `author_id` in ascending order.

Return the result table sorted by id in **ascending order**.

Table Name: `Views`

Article_ID	Author_ID	Author_Name	Viewer_ID	View_Count	Publication_Name
101	1	John Doe	1	3	The Daily Times
102	2	Jane Smith	2	1	Global News
103	3	Emily Clark	4	0	Tech Monthly
104	4	Mark Lewis	4	2	Health Digest
105	5	Sara White	3	0	The Business Journal
106	1	John Doe	2	0	The Daily Times
107	2	Jane Smith	2	2	Global News

Expected Columns:

- author_id
- author_name
- publication_name

Input:

```
SELECT DISTINCT author_id, author_name, publication_name FROM Views WHERE view_count = 0
ORDER BY author_id ASC;
```

Output:

Your Output

author_id	author_name	publication_name
1	John Doe	The Daily Times
3	Emily Clark	Tech Monthly
5	Sara White	The Business Journal

12. PLAYER PERFORMANCE INSIGHTS

Task:

Write a query to find the names of the **top 3 distinct players** by highest score who have **won** matches, including their **scores**.

Table 1: `Players`

Player_ID	Player_Name	Score	Rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6

Table 2: `Matches`

Match_ID	Player1	Player2	Winner	Match_Date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

Expected Columns:

- player_name
- score

Input:

```
SELECT DISTINCT player_name, score
FROM Players JOIN Matches WHERE winner=player_name
ORDER BY score DESC
LIMIT 3;
```

Output:

Your Output

player_name	score
David	1600
Bob	1500
Charlie	1300

13. PLAYER DETAILS

Task:

Write a query to retrieve the details of the **last five matches** played, including the match ID, the names of the players who participated, the name of the winning player, match date, and the final score of the winner.

There are two tables named **`Players`** and **`Matches`**.

Table 1: `Players`

Player_ID	Player_Name	Score	Rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6
6	Frank	1450	3
7	Grace	1350	7
8	Hank	1250	9
9	Ivy	1550	8
10	Jack	1400	10

Table 2: `Matches`

Match_ID	Player_1	Player_2	Winner	Match_Date
101	Alice	Bob	Bob	2024-01-25
102	Charlie	David	David	2024-01-06
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-01
105	Charlie	Eve	Charlie	2024-01-15
106	Frank	Hank	Frank	2024-01-29
107	Grace	Ivy	Ivy	2024-01-10
108	Jack	Alice	Jack	2024-01-19
109	Bob	Charlie	Bob	2024-01-13
110	David	Eve	David	2024-01-24

Expected Output Columns:

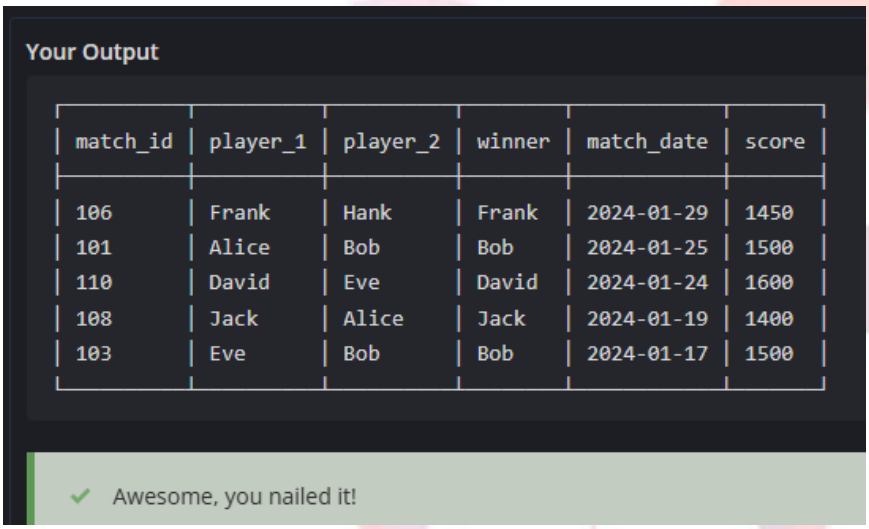
- match_id
- player_1
- player_2

- winner
- match_date
- score

Input:

```
SELECT m.match_id, m.player_1, m.player_2, m.winner, m.match_date, p.score
FROM Matches m
JOIN Players p ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

Output:



CODE CHEF BADGE



SQL PRACTICE PROGRAMS

QUESTION : 1

TABLE 1

CREATE TABLE CABHUB (VehicleID INT PRIMARY KEY,VehicleNameVARCHAR(30),Make VARCHAR(30),Colour VARCHAR(20),Capacity INT,Charges DECIMAL(10,2));

INSERT INTO CABHUB VALUES (101, 'Innova', 'Toyota', 'White', 7, 2500);

INSERT INTO CABHUB VALUES (102, 'Swift', 'Maruti', 'Red', 4, 1500);

INSERT INTO CABHUB VALUES (103, 'City', 'Honda', 'White', 5, 2000);

INSERT INTO CABHUB VALUES (104, 'Verna', 'Hyundai', 'Black', 4, 1800);

TABLE 2

CREATE TABLE CUSTOMER (CustomerID INT PRIMARY KEY,CustomerName VARCHAR(30),VehicleID INT,FOREIGN KEY (VehicleID) REFERENCES CABHUB(VehicleID));

INSERT INTO CUSTOMER VALUES (1, 'Ravi', 101);

INSERT INTO CUSTOMER VALUES (2, 'Priya', 102);

INSERT INTO CUSTOMER VALUES (3, 'Kumar', 104);

QUERY 1

SELECT VehicleName FROM CABHUB WHERE Colour = 'White';

Results Explain Describe Saved SQL History

VEHICLENAME
Innova
City

2 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT VehicleName, Capacity FROM CABHUB ORDER BY Capacity ASC;

Results Explain Describe Saved SQL History

VEHICLENAME	CAPACITY
Swift	4
Verna	4
City	5
Innova	7

4 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT MAX(Charges) FROM CABHUB;

Results Explain Describe Saved SQL History

MAX(CHARGES)
2500

1 rows returned in 0.00 seconds CSV Export

QUERY 4

SELECT CUSTOMER.CustomerName, CABHUB.VehicleName FROM CUSTOMER JOIN CABHUB ON CUSTOMER.VehicleID = CABHUB.VehicleID;

Results Explain Describe Saved SQL History

CUSTOMERNAME	VEHICLENAME
Ravi	Innova
Priya	Swift
Kumar	Verna

3 rows returned in 0.01 seconds CSV Export

QUERY 5

SELECT COUNT(DISTINCT Make) FROM CABHUB;

Results Explain Describe Saved SQL History

COUNT(DISTINCTMAKE)
4

1 rows returned in 0.00 seconds CSV Export

QUERY 6

SELECT MAX(Charges), MIN(Charges) FROM CABHUB;

Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
2500	1500

1 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT COUNT(*), Make FROM CABHUB GROUP BY Make;

Results Explain Describe Saved SQL History

COUNT(*)	MAKE
1	Hyundai
1	Toyota
1	Maruti
1	Honda

4 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT VehicleName FROM CABHUB WHERE Capacity = 4;

Results Explain Describe Saved SQL History

VEHICLENAME
Swift
Verna

2 rows returned in 0.00 seconds CSV Export

QUESTION-2

TABLE 1

CREATE TABLE CO2_EMISSION (VehicleID INT PRIMARY KEY,Company VARCHAR(30),Model VARCHAR(30),FuelType VARCHAR(20),EngineSize DECIMAL(3,1),CO2Emission INT);

INSERTION

INSERT INTO CO2_EMISSION VALUES(101, 'Toyota', 'Innova', 'Diesel', 2.4, 180);

INSERT INTO CO2_EMISSION VALUES(102, 'Honda', 'City', 'Petrol', 1.5, 120);

INSERT INTO CO2_EMISSION VALUES(103, 'Hyundai', 'Creta', 'Diesel', 1.6, 170);

INSERT INTO CO2_EMISSION VALUES(104, 'Maruti', 'Swift', 'Petrol', 1.2, 110);

INSERT INTO CO2_EMISSION VALUES(105, 'Kia', 'Seltos', 'Petrol', 1.5, 145);

INSERT INTO CO2_EMISSION VALUES(106, 'Tata', 'Nexon', 'Diesel', 1.5, 155);

INSERT INTO CO2_EMISSION VALUES(107, 'Mahindra', 'XUV', 'Diesel', 2.0, 190);

QUERY 1

SELECT Company, Model FROM CO2_EMISSION WHERE EngineSize > 1.5;

Results Explain Describe Saved SQL History

COMPANY	MODEL	CO2EMISSION
Maruti	Swift	110
Honda	City	120
Kia	Seltos	145
Tata	Nexon	155
Hyundai	Creta	170
Toyota	Innova	180
Mahindra	XUV	190

7 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT Company, Model, CO2Emission FROM CO2_EMISSION ORDER BY CO2Emission ASC;

Results Explain Describe Saved SQL History

COMPANY	MODEL
Toyota	Innova
Hyundai	Creta
Mahindra	XUV

3 rows returned in 0.02 seconds CSV Export

QUERY 3

SELECT *FROM CO2_EMISSION WHERE CO2Emission > 150;

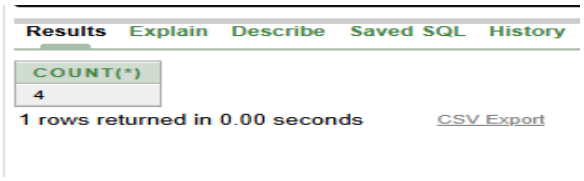
Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Innova	Diesel	2.4	180
103	Hyundai	Creta	Diesel	1.6	170
106	Tata	Nexon	Diesel	1.5	155
107	Mahindra	XUV	Diesel	2	190

4 rows returned in 0.02 seconds CSV Export

QUERY 4

SELECT COUNT(*)FROM CO2_EMISSION WHERE FuelType = 'Diesel';

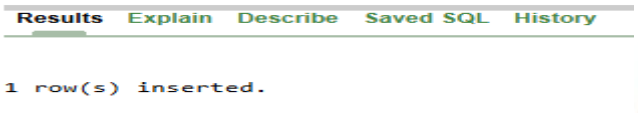

Results Explain Describe Saved SQL History

COUNT(*)
4

1 rows returned in 0.00 seconds CSV Export

QUERY 5

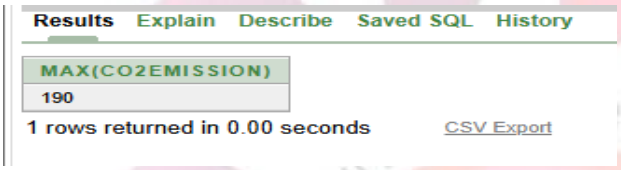
INSERT INTO CO2_EMISSION VALUES (108, 'MG', 'Hector', 'Petrol', 1.5, 160);


Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

QUERY 6

SELECT MAX(CO2Emission)FROM CO2_EMISSION;

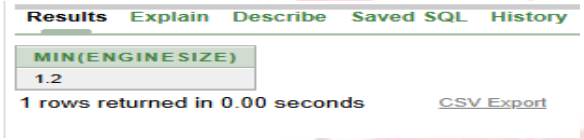

Results Explain Describe Saved SQL History

MAX(CO2EMISSION)
190

1 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT MIN(EngineSize)FROM CO2_EMISSION;

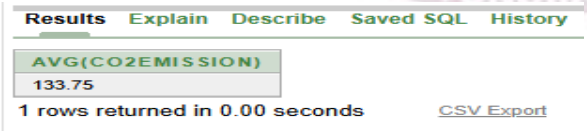

Results Explain Describe Saved SQL History

MIN(ENGINESIZE)
1.2

1 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT AVG(CO2Emission)FROM CO2_EMISSION WHERE FuelType='Petrol';

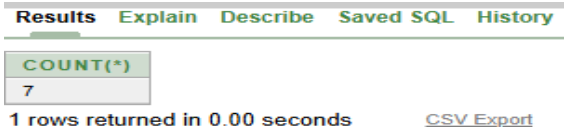

Results Explain Describe Saved SQL History

AVG(CO2EMISSION)
133.75

1 rows returned in 0.00 seconds CSV Export

QUERY 9

SELECT COUNT(*)FROM CO2_EMISSION WHERE EngineSize>=1.5;


Results Explain Describe Saved SQL History

COUNT(*)
7

1 rows returned in 0.00 seconds CSV Export

QUESTION-3

TABLE 1

CREATE TABLE APPLICANT (NO NUMBER,NAME VARCHAR2(30),FEE NUMBER,GENDER CHAR(1),C_ID VARCHAR2(5),JOINYEAR NUMBER);

INSERT INTO APPLICANT VALUES(1012, 'Amandeep', 30000, 'M', 'A01', 2012);

INSERT INTO APPLICANT VALUES(1102, 'Avisha', 25000, 'F', 'A02', 2009);

INSERT INTO APPLICANT VALUES(1103, 'Ekant', 30000, 'M', 'A02', 2011);

INSERT INTO APPLICANT VALUES(1049, 'Arun', 30000, 'M', 'A03', 2009);

INSERT INTO APPLICANT VALUES(1025, 'Amber', 40000, 'M', 'A02', 2011);

INSERT INTO APPLICANT VALUES(1106, 'Ela', 40000, 'F', 'A05', 2010);

INSERT INTO APPLICANT VALUES(1017, 'Nikita', 35000, 'F', 'A03', 2012);

INSERT INTO APPLICANT VALUES(1108, 'Arleena', 30000, 'F', 'A03', 2012);

INSERT INTO APPLICANT VALUES(2109, 'Shakti', 35000, 'M', 'A04', 2011);

INSERT INTO APPLICANT VALUES(1101, 'Kirat', 25000, 'M', 'A01', 2012);

TABLE 2

CREATE TABLE COURSE (C_ID VARCHAR2(5),COURSE VARCHAR2(30));

INSERT INTO COURSE VALUES('A01', 'Fashion Design');

INSERT INTO COURSE VALUES('A02', 'Networking');

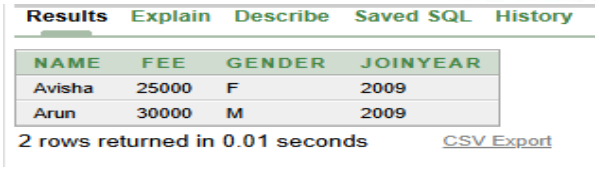
INSERT INTO COURSE VALUES('A03', 'Hotel Management');

INSERT INTO COURSE VALUES('A04', 'Event Management');

INSERT INTO COURSE VALUES('A05', 'Office Management');

QUERY 1

SELECT NAME, FEE, GENDER, JOINYEAR FROM APPLICANT WHERE JOINYEAR < 2010;


Results Explain Describe Saved SQL History

NAME	FEE	GENDER	JOINYEAR
Avisha	25000	F	2009
Arun	30000	M	2009

2 rows returned in 0.01 seconds CSV Export

QUERY 2

SELECT NAME FROM APPLICANT WHERE FEE > 30000;

Results Explain Describe Saved SQL History

NAME
Amber
Ela
Nikita
Shakti

4 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT NAME FROM APPLICANT ORDER BY JOINYEAR ASC;

Results Explain Describe Saved SQL History

NAME
Arun
Avisha
Ela
Ekant
Amber
Shakti
Nikita
Arleena
Amandeep
Kirat

10 rows returned in 0.00 seconds CSV Export

QUERY 4

SELECT JOINYEAR, COUNT(*)FROM APPLICANT GROUP BY JOINYEAR;

Results Explain Describe Saved SQL History

JOINYEAR	COUNT(*)
2009	2
2010	1
2011	3
2012	4

4 rows returned in 0.00 seconds CSV Export

QUERY 5

SELECT APPLICANT.NAME, COURSE.COURSE FROM APPLICANT JOIN COURSE ON APPLICANT.C_ID = COURSE.C_ID;

Results Explain Describe Saved SQL History

NAME	COURSE
Amandeep	Fashion Design
Arun	Hotel Management
Ela	Office Management
Nikita	Hotel Management
Arleena	Hotel Management
Shakti	Event Management
Kirat	Fashion Design

7 rows returned in 0.00 seconds CSV Export

QUERY 6

SELECT NAME, JOINYEAR FROM APPLICANT WHERE GENDER = 'F' AND C_ID = 'A02';

Results Explain Describe Saved SQL History

NAME	JOINYEAR
Avisha	2009

1 rows returned in 0.02 seconds CSV Export

QUERY 7

SELECT MIN(JOINYEAR) FROM APPLICANT WHERE GENDER = 'M';

Results Explain Describe Saved SQL History

MIN(JOINYEAR)
2009

1 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT AVG(FEE)FROM APPLICANT WHERE C_ID = 'A01' OR C_ID = 'A05';

Results Explain Describe Saved SQL History

AVG(FEE)
31666.666666666666666666666666666666666667

1 rows returned in 0.02 seconds CSV Export

QUERY 9

SELECT SUM(FEE), C_ID FROM APPLICANT GROUP BY C_ID HAVING COUNT(*) = 2;

Results Explain Describe Saved SQL History

SUM(FEE)	C_ID
55000	A01

1 rows returned in 0.00 seconds CSV Export

QUESTION-4

TABLE 1

CREATE TABLE PRODUCTS (PID NUMBER,SNAME VARCHAR2(30),QTY NUMBER,PRICE NUMBER,COMPANY VARCHAR2(30),SUPCODE VARCHAR2(5));

INSERT INTO PRODUCTS VALUES(101, 'DIGITAL CAMERA14X', 120, 12000, 'RENIX', 'S01');

INSERT INTO PRODUCTS VALUES(102, 'DIGITAL PAD 1li', 100, 22000, 'DIGI POP', 'S02');

INSERT INTO PRODUCTS VALUES(104, 'PEN DRIVE 16 GB', 500, 1100, 'STOREKING', 'S01');

INSERT INTO PRODUCTS VALUES(106, 'LED SCREEN', 70, 28000, 'DISEXPERTS', 'S02');

INSERT INTO PRODUCTS VALUES(105, 'CAR GPS SYSSYTEM', 60, 12000, 'MOVEON', 'S03');

TABLE 2

CREATE TABLE SUPPLIERS (SUPCODE VARCHAR2(5),SNAME VARCHAR2(30),CITY VARCHAR2(20));

INSERT INTO SUPPLIERS VALUES('S01', 'GET ALL INC', 'KOLKATA');

INSERT INTO SUPPLIERS VALUES('S03', 'EASY MARKET CORP', 'DELHI');

INSERT INTO SUPPLIERS VALUES('S02', 'DIGI BUSY GROUP', 'CHENNAI');

QUERY 1

SELECT *FROM PRODUCTS ORDER BY SNAME ASC;

Results Explain Describe Saved SQL History

PID	SNAME	QTY	PRICE	COMPANY	SUPCODE
105	CAR GPS SYSSYTEM	60	12000	MOVEON	S03
101	DIGITAL CAMERA14X	120	12000	RENIX	S01
102	DIGITAL PAD 1li	100	22000	DIGI POP	S02
106	LED SCREEN	70	28000	DISEXPERTS	S02
104	PEN DRIVE 16 GB	500	1100	STOREKING	S01

5 rows returned in 0.01 seconds CSV Export

QUERY 2

SELECT SNAME, PRICE FROM PRODUCTS WHERE PRICE BETWEEN 10000 AND 15000;

Results Explain Describe Saved SQL History

SNAME	PRICE
DIGITAL CAMERA14X	12000
CAR GPS SYSSYTEM	12000

2 rows returned in 0.02 seconds CSV Export

QUERY 3

SELECT SUPCODE, COUNT(*)FROM PRODUCTS GROUP BY SUPCODE;

Results Explain Describe Saved SQL History

SUPCODE	COUNT(*)
S01	2
S02	2
S03	1

3 rows returned in 0.01 seconds CSV Export

QUERY 4

SELECT PRICE, SNAME, QTY FROM PRODUCTS WHERE QTY > 100;

Results Explain Describe Saved SQL History

PRICE	SNAME	QTY
12000	DIGITAL CAMERA14X	120
1100	PEN DRIVE 16 GB	500

2 rows returned in 0.00 seconds CSV Export

QUERY 5

SELECT SNAME FROM SUPPLIERS WHERE CITY = 'DELHI' OR CITY = 'CHENNAI';

Results Explain Describe Saved SQL History

SNAME
EASY MARKET CORP
DIGI BUSY GROUP

2 rows returned in 0.00 seconds CSV Export

QUERY 6

SELECT COMPANY, SNAME FROM PRODUCTS ORDER BY COMPANY DESC;

Results Explain Describe Saved SQL History

COMPANY	SNAME
STOREKING	PEN DRIVE 16 GB
RENIX	DIGITAL CAMERA14X
MOVEON	CAR GPS SYSSYTEM
DISEXPERTS	LED SCREEN
DIGI POP	DIGITAL PAD 1li

5 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT DISTINCT SUPCODE FROM PRODUCTS;

Results Explain Describe Saved SQL History

SUPCODE
S01
S02
S03

3 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT MAX(PRICE), MIN(PRICE) FROM PRODUCTS;

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
28000	1100

1 rows returned in 0.00 seconds CSV Export

QUERY 9

SELECT PRICE * QTY AS AMOUNT FROM PRODUCTS WHERE PID = 104;

Results Explain Describe Saved SQL History

AMOUNT
550000

1 rows returned in 0.00 seconds CSV Export

QUERY 10

SELECT P.SNAME, S.SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P.SUPCODE = S.SUPCODE AND QTY > 100;

Results Explain Describe Saved SQL History

SNAME	SNAME
PEN DRIVE 16 GB	GET ALL INC
DIGITAL CAMERA14X	GET ALL INC

2 rows returned in 0.02 seconds CSV Export

TRUTH-TOIL-TRIUMPH AUTONOMOUS

QUESTION-5

TABLE 1

CREATE TABLE SHOPPE (ID VARCHAR2(10),SName VARCHAR2(30),Area VARCHAR2(20));

INSERT INTO SHOPPE VALUES('S0001', 'ABC Computeronics', 'CP');

INSERT INTO SHOPPE VALUES('S0002', 'All Infotech Media', 'GK II');

INSERT INTO SHOPPE VALUES('S0003', 'Tech Shoppe', 'CP');

INSERT INTO SHOPPE VALUES('S0004', 'Greeks Techno Soft', 'Nehru Place');

INSERT INTO SHOPPE VALUES('S0005', 'Hitech Tech Store', 'Nehru Place');

TABLE 2

CREATE TABLE ACCESSORIES (No VARCHAR2(10),Name VARCHAR2(30),Price NUMBER,ID VARCHAR2(10));

INSERT INTO ACCESSORIES VALUES('A01', 'Mother Board', 12000, 'S01');

INSERT INTO ACCESSORIES VALUES('A02', 'Hard Disk', 5000, 'S01');

INSERT INTO ACCESSORIES VALUES('A03', 'Keyboard', 500, 'S02');

INSERT INTO ACCESSORIES VALUES('A04', 'Mouse', 300, 'S01');

INSERT INTO ACCESSORIES VALUES('A05', 'Mother Board', 13000, 'S02');

INSERT INTO ACCESSORIES VALUES('A06', 'Keyboard', 400, 'S03');

INSERT INTO ACCESSORIES VALUES('A07', 'LCD', 6000, 'S04');

INSERT INTO ACCESSORIES VALUES('T08', 'LCD', 5500, 'S05');

INSERT INTO ACCESSORIES VALUES('T09', 'Mouse', 350, 'S05');

INSERT INTO ACCESSORIES VALUES('T10', 'Hard Disk', 4500, 'S03');

QUERY 1

SELECT Name, Price FROM ACCESSORIES ORDER BY Price ASC;

Results Explain Describe Saved SQL History

NAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT ID, SName FROM SHOPPE WHERE Area = 'Nehru Place';

Results Explain Describe Saved SQL History

ID	SNAME
S0004	Greeks Techno Soft
S0005	Hitech Tech Store

2 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT Name, MIN(Price), MAX(Price)FROM ACCESSORIES GROUP BY Name;

Results Explain Describe Saved SQL History

NAME	MIN(PRICE)	MAX(PRICE)
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.00 seconds CSV Export

QUERY 4

SELECT A.Name, A.Price, S.SName FROM ACCESSORIES A, SHOPPE S WHERE A.ID = SUBSTR(S.ID,1,1) || SUBSTR(S.ID,-2);

Results Explain Describe Saved SQL History

NAME	PRICE	SNAME
Mother Board	12000	ABC Computeronics
Hard Disk	5000	ABC Computeronics
Keyboard	500	All Infotech Media
Mouse	300	ABC Computeronics
Mother Board	13000	All Infotech Media
Keyboard	400	Tech Shoppe
LCD	6000	Greeks Techno Soft
LCD	5500	Hitech Tech Store
Mouse	350	Hitech Tech Store
Hard Disk	4500	Tech Shoppe

More than 10 rows available. Increase rows selector to view more rows.

10 rows returned in 0.00 seconds CSV Export

QUERY 5

SELECT DISTINCT Name FROM ACCESSORIES WHERE Price >= 500;

Results Explain Describe Saved SQL History

NAME
Mother Board
LCD
Keyboard
Hard Disk

4 rows returned in 0.00 seconds CSV Export

QUERY 6

SELECT Area, COUNT(*)FROM SHOPPE GROUP BY Area;

Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT COUNT(DISTINCT Area) FROM SHOPPE;

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT Name, Price * 0.05 AS DISCOUNT FROM ACCESSORIES WHERE ID IN ('S02', 'S03');

Results Explain Describe Saved SQL History

NAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225
Hard Disk	225

5 rows returned in 0.00 seconds CSV Export

QUESTION-6

TABLE 1

```
CREATE TABLE HOSPITAL (No NUMBER,Name VARCHAR2(30),Age
NUMBER,Department VARCHAR2(30),DateOfAdm DATE,Charges NUMBER,Gender
CHAR(1));

INSERT INTO HOSPITAL VALUES(1, 'Arpit', 62, 'Surgery',

TO_DATE('21/01/1998','DD/MM/YYYY'), 300, 'M');

INSERT INTO HOSPITAL VALUES(2, 'Zarina', 22, 'ENT',
TO_DATE('12/12/1997','DD/MM/YYYY'), 250, 'F');

INSERT INTO HOSPITAL VALUES(3, 'Kareem', 32, 'Orthopedic',
TO_DATE('19/02/1998','DD/MM/YYYY'), 200, 'M');

INSERT INTO HOSPITAL VALUES(4, 'Arun', 12, 'Surgery',
TO_DATE('11/01/1998','DD/MM/YYYY'), 300, 'M');

INSERT INTO HOSPITAL VALUES(5, 'Zubin', 30, 'ENT',
TO_DATE('24/02/1998','DD/MM/YYYY'), 250, 'M');

INSERT INTO HOSPITAL VALUES(6, 'Ketaki', 16, 'ENT',
TO_DATE('12/01/1998','DD/MM/YYYY'), 250, 'M');

INSERT INTO HOSPITAL VALUES(7, 'Ankita', 29, 'Cardiology',
TO_DATE('20/02/1998','DD/MM/YYYY'), 800, 'F');

INSERT INTO HOSPITAL VALUES(8, 'Zareen', 45, 'Gynecology',
TO_DATE('22/02/1998','DD/MM/YYYY'), 300, 'F');

INSERT INTO HOSPITAL VALUES(9, 'Kush', 19, 'Cardiology',
TO_DATE('13/01/1998','DD/MM/YYYY'), 800, 'M');

INSERT INTO HOSPITAL VALUES(10, 'Shilpa', 23, 'Nuclear Medicine',
TO_DATE('21/02/1998','DD/MM/YYYY'), 400, 'F');
```

QUERY 1

```
SELECT *FROM HOSPITAL WHERE Department = 'Cardiology';
```

Results | Explain | Describe | Saved SQL | History

NO	NAME	AGE	DEPARTMENT	DATEOFADM	CHARGES	GENDER
7	Ankita	29	Cardiology	20-FEB-98	800	F
9	Kush	19	Cardiology	13-JAN-98	800	M

2 rows returned in 0.01 seconds CSV Export

QUERY 2

```
SELECT Name FROM HOSPITAL WHERE Gender = 'F' AND Department = 'ENT';
```

Results | Explain | Describe | Saved SQL | History

NAME
Zarina

1 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT Name, DateOfAdm FROM HOSPITAL ORDER BY DateOfAdm ASC;

Results Explain Describe Saved SQL History

NAME	DATEOFADM
Zarina	12-DEC-97
Arun	11-JAN-98
Ketaki	12-JAN-98
Kush	13-JAN-98
Arpit	21-JAN-98
Kareem	19-FEB-98
Ankita	20-FEB-98
Shilpa	21-FEB-98
Zareen	22-FEB-98
Zubin	24-FEB-98

10 rows returned in 0.00 seconds CSV Export

QUERY 4

SELECT Name, Charges, Age FROM HOSPITAL WHERE Gender = 'F';

Results Explain Describe Saved SQL History

NAME	CHARGES	AGE
Zarina	250	22
Ankita	800	29
Zareen	300	45
Shilpa	400	23

4 rows returned in 0.00 seconds CSV Export

QUERY 5

SELECT COUNT(*)FROM HOSPITAL WHERE Age < 30;

Results Explain Describe Saved SQL History

COUNT(*)
6

1 rows returned in 0.00 seconds CSV Export

QUERY 6

INSERT INTO HOSPITAL VALUES
(11,'Aftab',24,'Surgery',TO_DATE('25/02/1998','DD/MM/YYYY'),300, 'M');

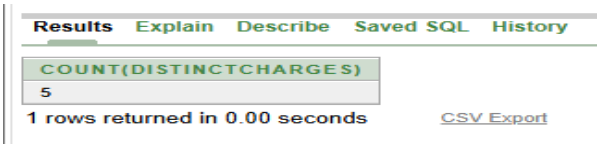
Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

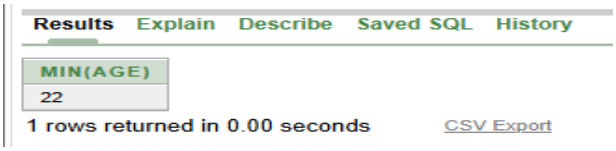
QUERY 7

SELECT COUNT(DISTINCT Charges)FROM HOSPITAL;



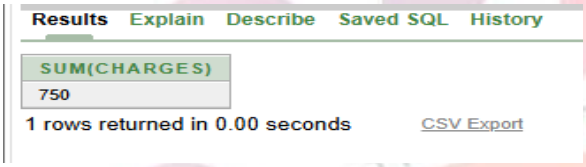
QUERY 8

SELECT MIN(Age) FROM HOSPITAL WHERE Gender = 'F';



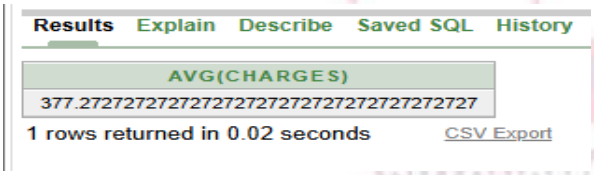
QUERY 9

SELECT SUM(Charges) FROM HOSPITAL WHERE Department = 'ENT';



QUERY 10

SELECT AVG(Charges) FROM HOSPITAL WHERE DateOfAdm < TO_DATE('12/08/1998','DD/MM/YYYY');



QUESTION-7

TABLE 1

CREATE TABLE STORE (SNo VARCHAR2(5),SName VARCHAR2(30),Area VARCHAR2(20));

INSERT INTO STORE VALUES('S01', 'ABC Computronics', 'GK II');

INSERT INTO STORE VALUES('S02', 'All InfoTech Media', 'CP');

INSERT INTO STORE VALUES('S03', 'Tech Shoppe', 'Nehru Place');

INSERT INTO STORE VALUES('S04', 'Geeks Techno Soft', 'Nehru Place');

INSERT INTO STORE VALUES('S05', 'Hitech Tech Store', 'CP');

TABLE 2

CREATE TABLE ITEM (INo VARCHAR2(5),IName VARCHAR2(30),Price NUMBER,SNo VARCHAR2(5));

INSERT INTO ITEM VALUES('T01', 'Mother Board', 12000, 'S01');

INSERT INTO ITEM VALUES('T02', 'Hard Disk', 5000, 'S01');

INSERT INTO ITEM VALUES('T03', 'Keyboard', 500, 'S02');

INSERT INTO ITEM VALUES('T04', 'Mouse', 300, 'S01');

INSERT INTO ITEM VALUES('T05', 'Mother Board', 13000, 'S02');

INSERT INTO ITEM VALUES('T06', 'Keyboard', 400, 'S03');

INSERT INTO ITEM VALUES('T07', 'LCD', 6000, 'S04');

INSERT INTO ITEM VALUES('T08', 'LCD', 5500, 'S05');

INSERT INTO ITEM VALUES('T09', 'Mouse', 350, 'S05');

INSERT INTO ITEM VALUES('T10', 'Hard Disk', 4500, 'S03');

QUERY 1

SELECT IName, Price FROM ITEM ORDER BY Price ASC;

Results Explain Describe Saved SQL History

INAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT SNo, SName FROM STORE WHERE Area = 'CP';

Results Explain Describe Saved SQL History

SNO	SNAME
S02	All InfoTech Media
S05	Hitech Tech Store

2 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT IName, MIN(Price), MAX(Price) FROM ITEM GROUP BY IName;

Results Explain Describe Saved SQL History

INAME	MIN(PRICE)	MAX(PRICE)
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.00 seconds CSV Export

QUERY 4

SELECT ITEM.IName, ITEM.Price, STORE.SName FROM ITEM, STORE WHERE ITEM.SNo = STORE.SNo;

Results Explain Describe Saved SQL History

INAME	PRICE	SNAME
Mother Board	12000	ABC Computronics
Hard Disk	5000	ABC Computronics
Keyboard	500	All InfoTech Media
Mouse	300	ABC Computronics
Mother Board	13000	All InfoTech Media
Keyboard	400	Tech Shoppe
LCD	6000	Geeks Techno Soft
LCD	5500	Hitech Tech Store
Mouse	350	Hitech Tech Store
Hard Disk	4500	Tech Shoppe

10 rows returned in 0.00 seconds CSV Export

QUERY 5

SELECT DISTINCT IName FROM ITEM WHERE Price >= 5000;

Results Explain Describe Saved SQL History

INAME
Mother Board
LCD
Hard Disk

3 rows returned in 0.00 seconds CSV Export

QUERY 6

SELECT Area, COUNT(*) FROM STORE GROUP BY Area;

Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT COUNT(DISTINCT Area) FROM STORE;

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT IName, Price * 0.05 AS DISCOUNT FROM ITEM WHERE SNo IN ('S02', 'S03');

Results Explain Describe Saved SQL History

INAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.02 seconds CSV Export

QUESTION-8

CREATE TABLE TEACHER (No NUMBER,Name VARCHAR2(30),Age NUMBER,Department VARCHAR2(30),DateOfJoin DATE,Salary NUMBER,Gender CHAR(1));

INSERT INTO TEACHER VALUES(1, 'Jugal', 34, 'Computer', TO_DATE('10/01/1997','DD/MM/YYYY'), 12000, 'M');

INSERT INTO TEACHER VALUES(2, 'Sharmila', 31, 'History', TO_DATE('24/03/1998','DD/MM/YYYY'), 20000, 'F');

INSERT INTO TEACHER VALUES(3, 'Sandeep', 32, 'Maths', TO_DATE('12/12/1996','DD/MM/YYYY'), 30000, 'M');

INSERT INTO TEACHER VALUES(4, 'Sangeeta', 35, 'History', TO_DATE('01/07/1997','DD/MM/YYYY'), 40000, 'F');

INSERT INTO TEACHER VALUES(5, 'Rakesh', 42, 'Maths', TO_DATE('05/09/1997','DD/MM/YYYY'), 25000, 'M');

INSERT INTO TEACHER VALUES(6, 'Shyam', 50, 'History', TO_DATE('27/06/1998','DD/MM/YYYY'), 30000, 'M');

INSERT INTO TEACHER VALUES(7, 'Shiv Om', 44, 'Computer', TO_DATE('25/02/1997','DD/MM/YYYY'), 21000, 'M');

INSERT INTO TEACHER VALUES(8, 'Shalakha', 33, 'Maths', TO_DATE('31/07/1997','DD/MM/YYYY'), 20000, 'F');

QUERY 1

SELECT *FROM TEACHER WHERE Department = 'History';

Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATEOFJOIN	SALARY	GENDER
2	Sharmila	31	History	24-MAR-98	20000	F
4	Sangeeta	35	History	01-JUL-97	40000	F
6	Shyam	50	History	27-JUN-98	30000	M

3 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT Name FROM TEACHER WHERE Gender = 'F' AND Department = 'Hindi';

Results Explain Describe Saved SQL History

no data found

QUERY 3

SELECT Name, DateOfJoin FROM TEACHER ORDER BY DateOfJoin ASC;

Results Explain Describe Saved SQL History

NAME	DATEOFJOIN
Sandeep	12-DEC-96
Jugal	10-JAN-97
Shiv Om	25-FEB-97
Sangeeta	01-JUL-97
Shalakha	31-JUL-97
Rakesh	05-SEP-97
Sharmila	24-MAR-98
Shyam	27-JUN-98

8 rows returned in 0.01 seconds CSV Export

QUERY 4

SELECT Name, Salary, Age FROM TEACHER WHERE Gender = 'M';

Results Explain Describe Saved SQL History

NAME	SALARY	AGE
Jugal	12000	34
Sandeep	30000	32
Rakesh	25000	42
Shyam	30000	50
Shiv Om	21000	44

5 rows returned in 0.00 seconds CSV Export

QUERY 5

SELECT COUNT(*) FROM TEACHER WHERE Age > 23;

Results Explain Describe Saved SQL History

COUNT(*)
8

1 rows returned in 0.00 seconds CSV Export

QUERY 6

INSERT INTO TEACHER VALUES(9, 'Raja', 26, 'Computer',TO_DATE('13/05/1995','DD/MM/YYYY'),2300, 'M');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

QUERY 7

SELECT COUNT(DISTINCT Department) FROM TEACHER;

Results Explain Describe Saved SQL History

COUNT(DISTINCTDEPARTMENT)
3

1 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT MAX(Age) FROM TEACHER WHERE Gender = 'F';

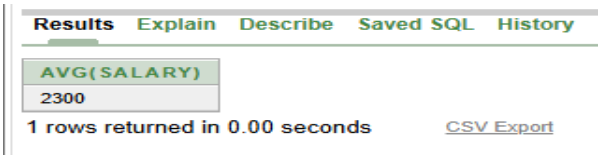
Results Explain Describe Saved SQL History

MAX(AGE)
35

1 rows returned in 0.00 seconds CSV Export

QUERY 9

SELECT AVG(Salary)FROM TEACHER WHERE DateOfJoin <
TO_DATE('12/07/1996','DD/MM/YYYY');



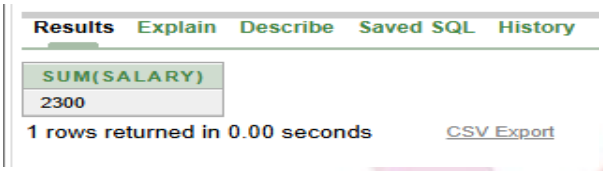
Results Explain Describe Saved SQL History

AVG(SALARY)
2300

1 rows returned in 0.00 seconds CSV Export

QUERY 10

SELECT SUM(Salary)FROM TEACHER WHERE DateOfJoin <
TO_DATE('12/07/1996','DD/MM/YYYY');



Results Explain Describe Saved SQL History

SUM(SALARY)
2300

1 rows returned in 0.00 seconds CSV Export

QUESTION-9

TABLE 1

CREATE TABLE BOOK (Book_Id VARCHAR2(10),Book_Name VARCHAR2(30),Author_Name VARCHAR2(30),Publishers VARCHAR2(30),Price NUMBER,Type VARCHAR2(20),Qty NUMBER);

INSERT INTO BOOK VALUES('F0001', 'The Tears', 'William Hopkins', 'First Publ.', 750, 'Fiction', 10);

INSERT INTO BOOK VALUES('F0002', 'Thunderbolts', 'Anna Roberts', 'First Publ.', 700, 'Fiction', 5);

INSERT INTO BOOK VALUES('T0001', 'My First C++', 'Briian & Brooke', 'EPB', 250, 'Text', 10);

INSERT INTO BOOK VALUES('T0002', 'C++ Brainworks', 'A. W. Rossaine', 'TDH', 325, 'Text', 5);

INSERT INTO BOOK VALUES('C0001', 'Fast Cook', 'LataKapoor', 'EPB', 350, 'Cookery', 8);

TABLE 2

CREATE TABLE ISSUE (Book_Id VARCHAR2(10),Quantity_Issued NUMBER);

INSERT INTO ISSUED VALUES('F0001', 3);

INSERT INTO ISSUED VALUES('T0001', 1);

INSERT INTO ISSUED VALUES('C0001', 5);

QUERY 1

SELECT Book_Name, Author_Name, Price FROM BOOK WHERE Publishers = 'EPB';

Results Explain Describe Saved SQL History

BOOK_NAME	AUTHOR_NAME	PRICE
My First C++	Briian & Brooke	250
Fast Cook	LataKapoor	350

2 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT Book_Name FROM BOOK WHERE Type = 'Fiction';

Results Explain Describe Saved SQL History

BOOK_NAME
The Tears
Thunderbolts

2 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT Book_Name, Price FROM BOOK ORDER BY Price DESC;

Results Explain Describe Saved SQL History

BOOK_NAME	PRICE
The Tears	750
Thunderbolts	700
Fast Cook	350
C++ Brainworks	325
My First C++	250

5 rows returned in 0.00 seconds CSV Export

QUERY 4

UPDATE BOOKS SET Price = Price + 50 WHERE Publishers = 'First Publ.';

Results Explain Describe Saved SQL History

```
2 row(s) updated.

0.00 seconds
```

QUERY 5

SELECT BOOKS.Book_Id, BOOKS.Book_Name, ISSUED.Quantity_Issued FROM BOOKS, ISSUED WHERE BOOKS.Book_Id = ISSUED.Book_Id;

Results Explain Describe Saved SQL History

BOOK_ID	BOOK_NAME	QUANTITY_ISSUED
F0001	The Tears	3
T0001	My First C++	1
C0001	Fast Cook	5
F0001	The Tears	3
T0001	My First C++	1
C0001	Fast Cook	5

6 rows returned in 0.00 seconds CSV Export

QUERY 6

SELECT COUNT(DISTINCT Publishers)FROM BOOKS;

Results Explain Describe Saved SQL History

COUNT(DISTINCTPUBLISHERS)
3

1 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT SUM(Price)FROM BOOKS WHERE Qty > 5;

Results Explain Describe Saved SQL History

SUM(PRICE)
1400

1 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT BOOK_NAME, AUTHOR_NAME FROM BOOKS WHERE Price < 500;

Results Explain Describe Saved SQL History

BOOK_NAME	AUTHOR_NAME
My First C++	Brian & Brooke
C++ Brainworks	A.W.Rossaine
Fast Cook	LataKapoor

3 rows returned in 0.00 seconds CSV Export

QUERY 9

SELECT COUNT(*)FROM BOOKS;

Results Explain Describe Saved SQL History

COUNT(*)
5

1 rows returned in 0.00 seconds CSV Export

QUESTION-10

TABLE 1

CREATE TABLE CARDEN (Ccode NUMBER,CarName VARCHAR2(20),Make VARCHAR2(20),Color VARCHAR2(15),Capacity NUMBER,Charges NUMBER);

INSERT INTO CARDEN VALUES(501, 'A-star', 'Suzuki', 'RED', 3, 14);

INSERT INTO CARDEN VALUES(503, 'Indigo', 'Tata', 'SILVER', 3, 12);

INSERT INTO CARDEN VALUES(502, 'Innova', 'Toyota', 'WHITE', 7, 15);

INSERT INTO CARDEN VALUES(509, 'SX4', 'Suzuki', 'SILVER', 4, 14);

INSERT INTO CARDEN VALUES(510, 'C-Class', 'Mercedes', 'RED', 4, 35);

TABLE 2

CREATE TABLE CUSTOMERS (CCode NUMBER,CName VARCHAR2(30),Ccode_Car NUMBER);

INSERT INTO CUSTOMERS VALUES(1001, 'HamantSahu', 501);

INSERT INTO CUSTOMERS VALUES(1002, 'Raj Lal', 509);

INSERT INTO CUSTOMERS VALUES(1003, 'Feroja Shah', 503);

INSERT INTO CUSTOMERS VALUES(1004, 'Ketan Dhal', 502);

QUERY 1

SELECT CarName FROM CARDEN WHERE Color = 'SILVER';

Results Explain Describe Saved SQL History

CARNAME
Indigo
SX4

2 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT CarName, Make, Capacity FROM CARDEN ORDER BY Capacity DESC;

Results Explain Describe Saved SQL History

CARNAME	MAKE	CAPACITY
Innova	Toyota	7
C-Class	Mercedes	4
SX4	Suzuki	4
Indigo	Tata	3
A-star	Suzuki	3

5 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT MAX(Charges) FROM CARDEN;

Results Explain Describe Saved SQL History

MAX(CHARGES)
35

1 rows returned in 0.00 seconds CSV Export

QUERY 4

SELECT CUSTOMERS.CName, CARDEN.CarName FROM CUSTOMERS, CARDEN WHERE CUSTOMERS.Ccode_Car = CARDEN.Ccode;

Results Explain Describe Saved SQL History

CNAME	CARNAME
HamantSahu	A-star
Feroja Shah	Indigo
Ketan Dhal	Innova
Raj Lal	SX4

4 rows returned in 0.01 seconds CSV Export

QUERY 5

SELECT COUNT(DISTINCT Make)FROM CARDEN;

Results Explain Describe Saved SQL History

COUNT(DISTINCTMAKE)
4

1 rows returned in 0.00 seconds CSV Export

QUERRY 6

SELECT MAX(Charges), MIN(Charges)FROM CARDEN;

Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
35	12

1 rows returned in 0.02 seconds CSV Export

QUERY 7

SELECT COUNT(*), Make FROM CARDEN GROUP BY Make;

Results Explain Describe Saved SQL History

COUNT(*)	MAKE
2	Suzuki
1	Mercedes
1	Toyota
1	Tata

4 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT CarName FROM CARDEN WHERE Capacity = 4;

Results Explain Describe Saved SQL History

CARNAME
SX4
C-Class

2 rows returned in 0.00 seconds CSV Export

QUESTION-11

TABLE 1

CREATE TABLE EMPLOYEES (ECODE NUMBER,NAME VARCHAR2(30),DESIG VARCHAR2(30),SGRADE VARCHAR2(5),DOJ DATE,DOB DATE);

INSERT INTO EMPLOYEES VALUES(101, 'Abdul Ahmad', 'EXECUTIVE', 'S03', TO_DATE('23-MAR-2003','DD-MON-YYYY'), TO_DATE('13-JAN-1980','DD-MON-YYYY'));

INSERT INTO EMPLOYEES VALUES(102, 'Ravi Chander', 'HEAD-IT', 'S02', TO_DATE('12-FEB-2010','DD-MON-YYYY'), TO_DATE('22-JUL-1987','DD-MON-YYYY'));

INSERT INTO EMPLOYEES VALUES(103, 'John Ken', 'Receptionist', 'S03', TO_DATE('24-JUN-2009','DD-MON-YYYY'), TO_DATE('24-FEB-1983','DD-MON-YYYY'));

INSERT INTO EMPLOYEES VALUES(105, 'Nazar Ameen', 'GM', 'S02', TO_DATE('11-AUG-2006','DD-MON-YYYY'), TO_DATE('03-MAR-1984','DD-MON-YYYY'));

INSERT INTO EMPLOYEES VALUES(108, 'PriyamSen', 'CEO', 'S01', TO_DATE('29-DEC-2004','DD-MON-YYYY'), TO_DATE('19-JAN-1982','DD-MON-YYYY'));

TABLE 2

CREATE TABLE SALGRADE (SGRADE VARCHAR2(5),SALARY NUMBER,HRA NUMBER);

INSERT INTO SALGRADE VALUES('S01', 56000, 18000);

INSERT INTO SALGRADE VALUES('S02', 32000, 12000);

INSERT INTO SALGRADE VALUES('S03', 24000, 8000);

QUERY 1

SELECT *FROM EMPLOYEES ORDER BY DOJ DESC;

Results Explain Describe Saved SQL History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
102	Ravi Chander	HEAD-IT	S02	12-FEB-10	22-JUL-87
103	John Ken	Receptionist	S03	24-JUN-09	24-FEB-83
105	Nazar Ameen	GM	S02	11-AUG-06	03-MAR-84
108	PriyamSen	CEO	S01	29-DEC-04	19-JAN-82
101	Abdul Ahmad	EXECUTIVE	S03	23-MAR-03	13-JAN-80

5 rows returned in 0.04 seconds CSV Export

QUERY 2

SELECT NAME, DESIG FROM EMPLOYEES WHERE SGRADE IN ('S02', 'S03');

Results Explain Describe Saved SQL History

NAME	DESIG
Abdul Ahmad	EXECUTIVE
Ravi Chander	HEAD-IT
John Ken	Receptionist
Nazar Ameen	GM

4 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT *FROM EMPLOYEES WHERE DOJ BETWEEN TO_DATE('09-FEB-2006','DD-MON-YYYY')AND TO_DATE('08-AUG-2009','DD-MON-YYYY');

Results Explain Describe Saved SQL History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
103	John Ken	Receptionist	S03	24-JUN-09	24-FEB-83
105	Nazar Ameen	GM	S02	11-AUG-06	03-MAR-84

2 rows returned in 0.02 seconds CSV Export

QUERY 4

INSERT INTO EMPLOYEES VALUES(109, 'Harish Roy', 'HEAD-IT', 'S02', TO_DATE('09-SEP-2007','DD-MON-YYYY'),TO_DATE('21-APR-1983','DD-MON-YYYY'));

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

QUERY 5

SELECT COUNT(SGRADE), SGRADE FROM EMPLOYEES GROUP BY SGRADE;

Results Explain Describe Saved SQL History

COUNT(SGRADE)	SGRADE
2	S03
3	S02
1	S01

3 rows returned in 0.02 seconds CSV Export

QUERY 6

SELECT MIN(DOB), MAX(DOJ)FROM EMPLOYEES;

Results Explain Describe Saved SQL History

MIN(DOB)	MAX(DOJ)
13-JAN-80	12-FEB-10

1 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT NAME, SALARY FROM EMPLOYEES E, SALGRADE S WHERE E.SGRADE = S.SGRADE AND E.ECODE < 103;

Results Explain Describe Saved SQL History

NAME	SALARY
Ravi Chander	32000
Abdul Ahmad	24000

2 rows returned in 0.00 seconds CSV Export

QUESTION-12

TABLE 1

CREATE TABLE WORKERS (ECODE NUMBER,NAME VARCHAR2(30),DESIGN VARCHAR2(30),PLEVEL VARCHAR2(5),DOJ DATE,DOB DATE);

INSERT INTO WORKERS VALUES(11, 'RadheShyam', 'Supervisor', 'P001',TO_DATE('13-SEP2004','DD-MON-YYYY'),TO_DATE('23-AUG-1981','DD-MON-YYYY'));

INSERT INTO WORKERS VALUES(12, 'ChanderNath', 'Operator', 'P003', TO_DATE('22-FEB-2010','DD-MON-YYYY'), TO_DATE('12-JUL-1987','DD-MON-YYYY'));

INSERT INTO WORKERS VALUES(13, 'Fizza', 'Operator', 'P003', TO_DATE('14-JUN-2009','DD-MON-YYYY'), TO_DATE('14-OCT-1983','DD-MON-YYYY'));

INSERT INTO WORKERS VALUES(15, 'Ahmeen Ahmad', 'Mechanic', 'P002', TO_DATE('21-AUG-2006','DD-MON-YYYY'), TO_DATE('13-MAR-1984','DD-MON-YYYY'));

INSERT INTO WORKERS VALUES(18, 'Sanya', 'Clerk', 'P002', TO_DATE('19-DEC-2005','DD-MON-YYYY'), TO_DATE('09-JUN-1983','DD-MON-YYYY'));

TABLE 2

CREATE TABLE PAYLEVELS (PLEVEL VARCHAR2(5),PAY NUMBER,ALLOWANCE NUMBER);

INSERT INTO PAYLEVELS VALUES('P001', 26000, 12000);

INSERT INTO PAYLEVELS VALUES('P002', 22000, 10000);

INSERT INTO PAYLEVELS VALUES('P003', 12000, 6000);

QUERY 1

SELECT *FROM WORKERS ORDER BY DOB DESC;



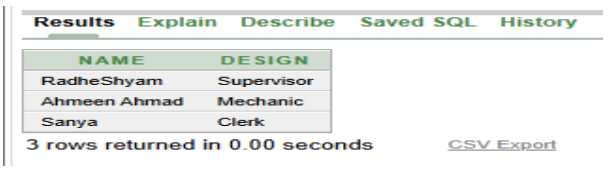
Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
12	ChanderNath	Operator	P003	22-FEB-10	12-JUL-87
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-06	13-MAR-84
13	Fizza	Operator	P003	14-JUN-09	14-OCT-83
18	Sanya	Clerk	P002	19-DEC-05	09-JUN-83
11	RadheShyam	Supervisor	P001	13-SEP-04	23-AUG-81

5 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT NAME, DESIGN FROM WORKERS WHERE PLEVEL IN ('P001', 'P002');



Results Explain Describe Saved SQL History

NAME	DESIGN
RadheShyam	Supervisor
Ahmeen Ahmad	Mechanic
Sanya	Clerk

3 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT *FROM WORKERS WHERE DOB BETWEEN TO_DATE('19-JAN-1984','DD-MON-YYYY')AND TO_DATE('18-JAN-1987','DD-MON-YYYY');

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-06	13-MAR-84

1 rows returned in 0.00 seconds CSV Export

QUERY 4

INSERT INTO WORKERS VALUES(19, 'Daya Kishore', 'Operator', 'P003',TO_DATE('19-JUN-2008','DD-MON-YYYY'),TO_DATE('11-JUL-1984','DD-MON-YYYY'));

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.01 seconds
```

QUERY 5

SELECT COUNT(PLEVEL), PLEVEL FROM WORKERS GROUP BY PLEVEL;

Results Explain Describe Saved SQL History

COUNT(PLEVEL)	PLEVEL
1	P001
3	P003
2	P002

3 rows returned in 0.00 seconds CSV Export

QUERY 6

SELECT MAX(DOB), MIN(DOJ)FROM WORKERS;

Results Explain Describe Saved SQL History

MAX(DOB)	MIN(DOJ)
12-JUL-87	13-SEP-04

1 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT W.NAME, P.PAY FROM WORKERS W, PAYLEVELS P WHERE W.PLEVEL = P.PLEVEL AND W.ECODE < 13;

Results Explain Describe Saved SQL History

NAME	PAY
RadheShyam	26000
ChanderNath	12000

2 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT PLEVEL, PAY + ALLOWANCE FROM PAYLEVELS WHERE PLEVEL = 'P003';

Results Explain Describe Saved SQL History

PLEVEL	PAY+ALLOWANCE
P003	18000

1 rows returned in 0.00 seconds CSV Export

QUESTION-13

TABLE 1

CREATE TABLE STORES (ItemNo NUMBER,Item VARCHAR2(30),Scode NUMBER,Qty NUMBER,Rate NUMBER,LastBuy DATE);

INSERT INTO STORES VALUES(2005, 'Sharpener Classic', 23, 60, 8, TO_DATE('30-JUN-2009','DD-MON-YYYY'));

INSERT INTO STORES VALUES(2003, 'Ball Pen 0.25', 22, 50, 25,TO_DATE('01-FEB-2009','DD-MON-YYYY'));

INSERT INTO STORES VALUES(2002, 'Gel Pen Premium', 21, 150, 12,TO_DATE('24-FEB-2009','DD-MON-YYYY'));

INSERT INTO STORES VALUES(2006, 'Gel Pen Classic', 21, 250, 20,TO_DATE('11-MAY-2009','DD-MON-YYYY'));

INSERT INTO STORES VALUES(2001, 'Eraser Small', 22, 220, 6,TO_DATE('19-JAN-2009','DD-MON-YYYY'));

INSERT INTO STORES VALUES(2004, 'Eraser Big', 22, 110, 8,TO_DATE('02-DEC-2009','DD-MON-YYYY'));

INSERT INTO STORES VALUES(2009, 'Ball Pen 0.5', 21, 180, 18,TO_DATE('03-NOV-2009','DD-MON-YYYY'));

TABLE 2

CREATE TABLE SUPPLIER (Scode NUMBER,Sname VARCHAR2(30));

INSERT INTO SUPPLIER VALUES(21, 'Premium Stationers');

INSERT INTO SUPPLIER VALUES(23, 'Soft Plastics');

INSERT INTO SUPPLIER VALUES(22, 'Tetra Supply');

QUERY 1

SELECT *FROM STORES ORDER BY LastBuy ASC;

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2001	Eraser Small	22	220	6	19-JAN-09
2003	Ball Pen 0.25	22	50	25	01-FEB-09
2002	Gel Pen Premium	21	150	12	24-FEB-09
2006	Gel Pen Classic	21	250	20	11-MAY-09
2005	Sharpener Classic	23	60	8	30-JUN-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09
2004	Eraser Big	22	110	8	02-DEC-09

7 rows returned in 0.00 seconds CSV Export

QUERY 2

SELECT ItemNo, Item FROM STORES WHERE Rate > 15;

Results Explain Describe Saved SQL History

ITEMNO	ITEM
2003	Ball Pen 0.25
2006	Gel Pen Classic
2009	Ball Pen 0.5

3 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT * FROM STORES WHERE Scode = 22 OR Qty > 110;

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2003	Ball Pen 0.25	22	50	25	01-FEB-09
2002	Gel Pen Premium	21	150	12	24-FEB-09
2006	Gel Pen Classic	21	250	20	11-MAY-09
2001	Eraser Small	22	220	6	19-JAN-09
2004	Eraser Big	22	110	8	02-DEC-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09

6 rows returned in 0.00 seconds CSV Export

QUERY 4

SELECT Scode, MIN(Rate)FROM STORES GROUP BY Scode;

Results Explain Describe Saved SQL History

SCODE	MIN(RATE)
22	6
21	12
23	8

3 rows returned in 0.00 seconds CSV Export

QUERY 5

SELECT COUNT(DISTINCT Scode) FROM STORES;

Results Explain Describe Saved SQL History

COUNT(DISTINCTSCODE)
3

1 rows returned in 0.02 seconds CSV Export

QUERY 6

SELECT Rate * Qty FROM STORES WHERE ItemNO = 2004;

Results Explain Describe Saved SQL History

RATE*QTY
880

1 rows returned in 0.00 seconds CSV Export

QUERY 7

SELECT S.Item, P.Sname FROM STORES S, SUPPLIER P WHERE S.Scode = P.Scode AND S.ItemNo = 2006;

Results Explain Describe Saved SQL History

ITEM	SNAME
Gel Pen Classic	Premium Stationers

1 rows returned in 0.01 seconds CSV Export

QUESTION-14

TABLE 1

CREATE TABLE GARMENT (GCODE NUMBER,DESCRIPTION VARCHAR2(30),PRICE NUMBER,FCODE VARCHAR2(5),READYDATE DATE);

INSERT INTO GARMENT VALUES(10023, 'Pencil Skirt', 1150, 'F03',TO_DATE('19-DEC-2008','DD-MON-YYYY'));

INSERT INTO GARMENT VALUES(10001, 'Formal Shirt', 1250, 'F01',TO_DATE('12-JAN-2008','DD-MON-YYYY'));

INSERT INTO GARMENT VALUES(10012, 'Formal Shirt', 1550, 'F02',TO_DATE('06-JUN-2008','DD-MON-YYYY'));

INSERT INTO GARMENT VALUES(10024, 'Baby Top', 750, 'F03',TO_DATE('07-APR-2007','DD-MON-YYYY'));

INSERT INTO GARMENT VALUES(10090, 'Tulip Skirt', 850, 'F02',TO_DATE('31-MAR-2007','DD-MON-YYYY'));

INSERT INTO GARMENT VALUES(10019, 'Evening Gown', 850, 'F03',TO_DATE('06-JUN-2008','DD-MON-YYYY'));

INSERT INTO GARMENT VALUES(10009, 'Formal Pant', 1500, 'F02',TO_DATE('20-OCT-2008','DD-MON-YYYY'));

INSERT INTO GARMENT VALUES(10007, 'Formal Pant', 1350, 'F01',TO_DATE('09-MAR-2008','DD-MON-YYYY'));

INSERT INTO GARMENT VALUES(10020, 'Frock', 850, 'F04',TO_DATE('09-SEP-2007','DD-MON-YYYY'));

INSERT INTO GARMENT VALUES(10089, 'Slacks', 750, 'F03',TO_DATE('20-OCT-2008','DD-MON-YYYY'));

TABLE 2

CREATE TABLE FABRIC (FCODE VARCHAR2(5),TYPE VARCHAR2(20));

INSERT INTO FABRIC VALUES ('F04', 'Polyester');

INSERT INTO FABRIC VALUES ('F02', 'Cotton');

INSERT INTO FABRIC VALUES ('F03', 'Silk');

INSERT INTO FABRIC VALUES ('F01', 'Terelene');

QUERY 1

SELECT GCODE, DESCRIPTION FROM GARMENT ORDER BY GCODE DESC;

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION
10090	Tulip Skirt
10089	Slacks
10024	Baby Top
10023	Pencil Skirt
10020	Frock
10019	Evening Gown
10012	Formal Shirt
10009	Formal Pant
10007	Formal Pant
10001	Formal Shirt

10 rows returned in 0.01 seconds CSV Export

QUERY 2

SELECT * FROM GARMENT WHERE READYDATE BETWEEN TO_DATE('08-DEC-2007','DD-MON-YYYY') AND TO_DATE('16-JUN-2008','DD-MON-YYYY');

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION	PRICE	FCODE	READYDATE
10001	Formal Shirt	1250	F01	12-JAN-08
10012	Formal Shirt	1550	F02	06-JUN-08
10019	Evening Gown	850	F03	06-JUN-08
10007	Formal Pant	1350	F01	09-MAR-08

4 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT AVG(PRICE) FROM GARMENT WHERE FCODE = 'F03';

Results Explain Describe Saved SQL History

AVG(PRICE)
875

1 rows returned in 0.00 seconds CSV Export

QUERY 4

SELECT FCODE, MAX(PRICE), MIN(PRICE) FROM GARMENT GROUP BY FCODE;

Results Explain Describe Saved SQL History

FCODE	MAX(PRICE)	MIN(PRICE)
F02	1550	850
F04	850	850
F03	1150	750
F01	1350	1250

4 rows returned in 0.00 seconds CSV Export

QUERY 5

SELECT SUM(PRICE)FROM GARMENT WHERE FCODE = 'F01';

Results Explain Describe Saved SQL History

SUM(PRICE)
2600

1 rows returned in 0.00 seconds CSV Export

QUERY 6

SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT.FCODE = FABRIC.FCODE AND GARMENT.PRICE >= 1260;

Results Explain Describe Saved SQL History

DESCRIPTION	TYPE
Formal Pant	Cotton
Formal Shirt	Cotton
Formal Pant	Terelene

3 rows returned in 0.01 seconds CSV Export

QUERY 7

SELECT MAX(FCODE)FROM FABRIC;

Results Explain Describe Saved SQL History

MAX(FCODE)
F04

1 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT COUNT(DISTINCT PRICE)FROM GARMENT;

Results Explain Describe Saved SQL History

COUNT(DISTINCTPRICE)
7

1 rows returned in 0.00 seconds CSV Export

QUESTION-15

TABLE 1

```
CREATE TABLE FURNITURE (No NUMBER,ItemName VARCHAR2(30),Type
VARCHAR2(30),DateOfStock DATE,Price NUMBER,Discount NUMBER);

INSERT INTO FURNITURE VALUES(1, 'White lotus','Double
Bed',TO_DATE('23/02/2002','DD/MM/YYYY'), 30000, 25);

INSERT INTO FURNITURE VALUES(2, 'Pink feather','Baby
cot',TO_DATE('20/01/2002','DD/MM/YYYY'), 7000, 20);

INSERT INTO FURNITURE VALUES(3, 'Dolphin', 'Baby
cot',TO_DATE('19/02/2002','DD/MM/YYYY'), 9500, 20);

INSERT INTO FURNITURE VALUES(4, 'Decent', 'Office
Table',TO_DATE('01/01/2002','DD/MM/YYYY'), 25000, 30);

INSERT INTO FURNITURE VALUES(5, 'Comfort zone', 'Double
Bed',TO_DATE('12/01/2002','DD/MM/YYYY'), 25000, 25);

INSERT INTO FURNITURE VALUES(6, 'Donald', 'Baby
cot',TO_DATE('24/02/2002','DD/MM/YYYY'), 6500, 15);

INSERT INTO FURNITURE VALUES(7, 'Royal Finish', 'Office

Table',TO_DATE('20/02/2002','DD/MM/YYYY'), 18000, 30);

INSERT INTO FURNITURE VALUES(8, 'Royal tiger',
'Sofa',TO_DATE('22/02/2002','DD/MM/YYYY'), 31000, 30);

INSERT INTO FURNITURE VALUES(9, 'Econ seating',
'Sofa',TO_DATE('13/01/2002','DD/MM/YYYY'), 9500, 25);

INSERT INTO FURNITURE VALUES(10, 'Eating paradise', 'Dining
Table',TO_DATE('19/02/2002','DD/MM/YYYY'), 11500, 25);
```

TABLE 1

```
CREATE TABLE ARRIVALS (No NUMBER,ItemName VARCHAR2(30),Type
VARCHAR2(30),DateOfStock DATE,Price NUMBER,Discount NUMBER);

INSERT INTO ARRIVALS VALUES(11, 'Wood Comfort', 'Double
Bed',TO_DATE('23/03/2003','DD/MM/YYYY'), 25000, 25);

INSERT INTO ARRIVALS VALUES(12, 'Old Fox',
'Sofa',TO_DATE('20/02/2003','DD/MM/YYYY'), 17000, 20);

INSERT INTO ARRIVALS VALUES(13, 'Micky', 'Baby
cot',TO_DATE('21/02/2002','DD/MM/YYYY'), 7500, 15);
```

QUERY 1

SELECT *FROM FURNITURE WHERE Type = 'Baby cot';

Results Explain Describe Saved SQL History

NO	ITEMNAME	TYPE	DATEOFSTOCK	PRICE	DISCOUNT
2	Pink feather	Baby cot	20-JAN-02	7000	20
3	Dolphin	Baby cot	19-FEB-02	9500	20
6	Donald	Baby cot	24-FEB-02	6500	15

3 rows returned in 0.01 seconds CSV Export

QUERY 2

SELECT ItemName FROM FURNITURE WHERE Price > 15000;

Results Explain Describe Saved SQL History

ITEMNAME
White lotus
Decent
Comfort zone
Royal Finish
Royal tiger

5 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT ItemName, Type FROM FURNITURE WHERE DateOfStock < TO_DATE('22/01/2002','DD/MM/YYYY')ORDER BY ItemName DESC;

Results Explain Describe Saved SQL History

ITEMNAME	TYPE
Pink feather	Baby cot
Econ seating	Sofa
Decent	Office Table
Comfort zone	Double Bed

4 rows returned in 0.00 seconds CSV Export

QUERY 4

SELECT ItemName, DateOfStock FROM FURNITURE WHERE Discount > 25;

Results Explain Describe Saved SQL History

ITEMNAME	DATEOFSTOCK
Decent	01-JAN-02
Royal Finish	20-FEB-02
Royal tiger	22-FEB-02

3 rows returned in 0.00 seconds CSV Export

QUERY 5

SELECT COUNT(*)FROM FURNITURE WHERE Type = 'Sofa';

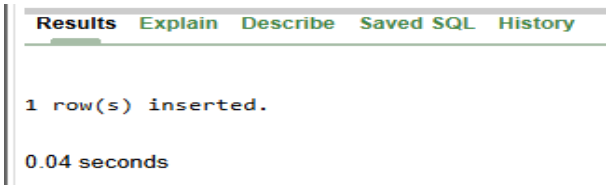
Results Explain Describe Saved SQL History

COUNT(*)
2

1 rows returned in 0.00 seconds CSV Export

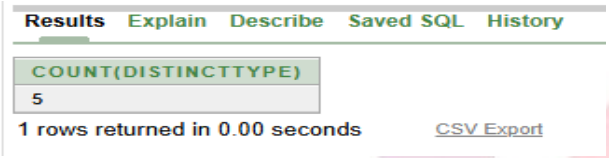
QUERY 6

INSERT INTO ARRIVALS VALUES(14, 'Velvet touch', 'Double Bed',TO_DATE('25/03/2003','DD/MM/YYYY'), 25000, 30);



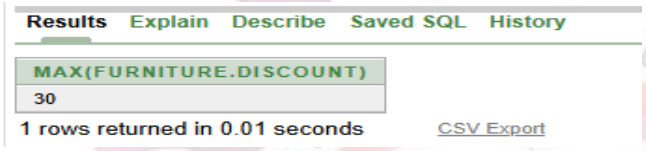
QUERY 7

SELECT COUNT(DISTINCT Type) FROM FURNITURE;



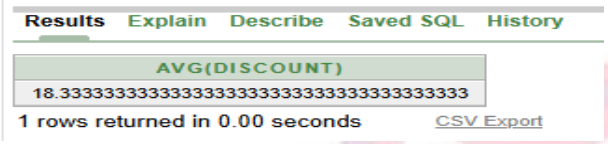
QUERY 8

SELECT MAX(FURNITURE.Discount)FROM FURNITURE, ARRIVALS;



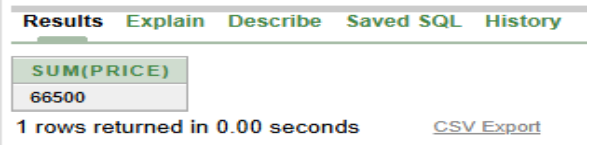
QUERY 9

SELECT AVG(Discount)FROM FURNITURE WHERE Type = 'Baby cot';



QUERY 10

SELECT SUM(Price)FROM FURNITURE WHERE DateOfStock < TO_DATE('12/02/2002','DD/MM/YYYY');



QUESTION-16

TABLE 1

CREATE TABLE COVID19 (PatientID NUMBER,PatientName VARCHAR2(30),State VARCHAR2(30),Age NUMBER,Status VARCHAR2(20),Vaccinated VARCHAR2(5));

INSERT INTO COVID19 VALUES(101, 'Arun', 'Tamil Nadu', 35, 'Recovered', 'Yes');

INSERT INTO COVID19 VALUES(102, 'Priya', 'Kerala', 42, 'Active', 'Yes');

INSERT INTO COVID19 VALUES(103, 'Karthik', 'Karnataka', 28, 'Recovered', 'No');

INSERT INTO COVID19 VALUES(104, 'Divya', 'Tamil Nadu', 31, 'Active', 'Yes');

INSERT INTO COVID19 VALUES(105, 'Rahul', 'Andhra Pradesh', 55, 'Deceased', 'No');

INSERT INTO COVID19 VALUES(106, 'Meena', 'Kerala', 24, 'Recovered', 'Yes');

INSERT INTO COVID19 VALUES(107, 'Suresh', 'Tamil Nadu', 48, 'Active', 'No');

QUERY 1

SELECT * FROM COVID19 WHERE State = 'Tamil Nadu';

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	STATE	AGE	STATUS	VACCINATED
101	Arun	Tamil Nadu	35	Recovered	Yes
104	Divya	Tamil Nadu	31	Active	Yes
107	Suresh	Tamil Nadu	48	Active	No

3 rows returned in 0.02 seconds CSV Export

QUERY 2

SELECT PatientName FROM COVID19 WHERE Status = 'Recovered';

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Karthik
Meena

3 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT PatientName, Age FROM COVID19 ORDER BY Age ASC;

Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Meena	24
Karthik	28
Divya	31
Arun	35
Priya	42
Suresh	48
Rahul	55

7 rows returned in 0.00 seconds CSV Export

QUERY 4

SELECT PatientName FROM COVID19 WHERE Vaccinated = 'Yes';

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Priya
Divya
Meena

4 rows returned in 0.00 seconds CSV Export

QUERY 5

SELECT COUNT(*) FROM COVID19 WHERE Status = 'Active';

Results Explain Describe Saved SQL History

COUNT(*)
3

1 rows returned in 0.02 seconds CSV Export

QUERY 6

INSERT INTO COVID19 VALUES(108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes');

```
1 row(s) inserted.
```

QUERY 7

SELECT COUNT(*)FROM COVID19;

Results Explain Describe Saved SQL History

COUNT(*)
8

1 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT MAX(Age)FROM COVID19;

Results Explain Describe Saved SQL History

MAX(AGE)
55

1 rows returned in 0.00 seconds CSV Export

QUERY 9

SELECT COUNT(*)FROM COVID19 WHERE Vaccinated = 'Yes';

Results Explain Describe Saved SQL History

COUNT(*)
5

1 rows returned in 0.00 seconds CSV Export

QUERY 10

SELECT MIN(Age)FROM COVID19 WHERE Status = 'Recovered';

Results Explain Describe Saved SQL History

MIN(AGE)
24

1 rows returned in 0.00 seconds CSV Export

QUESTION-17

TABLE 1

CREATE TABLE HOUSEPRICE (HouseID NUMBER,Location VARCHAR2(30),Bedrooms NUMBER,AreaSqft NUMBER,Price NUMBER,HouseType VARCHAR2(30));

INSERT INTO HOUSEPRICE VALUES(201, 'Chennai', 2, 1100, 5200000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES(202, 'Coimbatore', 3, 1450, 6800000, 'Villa');

INSERT INTO HOUSEPRICE VALUES(203, 'Madurai', 2, 980, 4100000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES(204, 'Chennai', 4, 2200, 9800000, 'Villa');

INSERT INTO HOUSEPRICE VALUES(205, 'Trichy', 3, 1600, 7200000, 'Independent');

INSERT INTO HOUSEPRICE VALUES(206, 'Salem', 2, 1200, 5000000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES(207, 'Erode', 3, 1800, 7600000, 'Independent');

QUERY 1

SELECT *FROM HOUSEPRICE WHERE Location = 'Chennai';

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1100	5200000	Apartment
204	Chennai	4	2200	9800000	Villa

2 rows returned in 0.02 seconds CSV Export

QUERY 2

SELECT Location, Price FROM HOUSEPRICE WHERE Bedrooms = 3;

Results Explain Describe Saved SQL History

LOCATION	PRICE
Coimbatore	6800000
Trichy	7200000
Erode	7600000

3 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT Location, AreaSqft, Price FROM HOUSEPRICE ORDER BY Price DESC;

Results Explain Describe Saved SQL History

LOCATION	AREASQFT	PRICE
Chennai	2200	9800000
Erode	1800	7600000
Trichy	1600	7200000
Coimbatore	1450	6800000
Chennai	1100	5200000
Salem	1200	5000000
Madurai	980	4100000

7 rows returned in 0.00 seconds CSV Export

QUERY 4

SELECT * FROM HOUSEPRICE WHERE Price > 7000000;

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
204	Chennai	4	2200	9800000	Villa
205	Trichy	3	1600	7200000	Independent
207	Erode	3	1800	7600000	Independent

3 rows returned in 0.00 seconds CSV Export

QUERY 5

SELECT COUNT(*)FROM HOUSEPRICE WHERE HouseType = 'Apartment';

Results Explain Describe Saved SQL History

COUNT(*)
3

1 rows returned in 0.01 seconds CSV Export

QUERY 6

INSERT INTO HOUSEPRICE VALUES(208, 'Vellore', 3, 1700, 7300000, 'Villa');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

QUERY 7

SELECT MAX(Price)FROM HOUSEPRICE;

Results Explain Describe Saved SQL History

MAX(PRICE)
9800000

1 rows returned in 0.00 seconds CSV Export

QUERY 8

SELECT MIN(AreaSqft) FROM HOUSEPRICE;

Results Explain Describe Saved SQL History

MIN(AREASQFT)
980

1 rows returned in 0.00 seconds CSV Export

QUERY 9

SELECT AVG(Price)FROM HOUSEPRICE WHERE HouseType = 'Apartment';

Results Explain Describe Saved SQL History

AVG(PRICE)
4766666.666666666666666666666666666667

1 rows returned in 0.02 seconds CSV Export

QUERY 10

SELECT COUNT(*)FROM HOUSEPRICE WHERE Bedrooms = 3;

Results Explain Describe Saved SQL History

COUNT(*)
4

1 rows returned in 0.00 seconds CSV Export

QUESTION-18

TABLE 1

CREATE TABLE CANCERDATA (PatientID NUMBER,PatientName VARCHAR2(30),CancerType VARCHAR2(30),Age NUMBER,Stage VARCHAR2(5),SurvivalYears NUMBER);

INSERT INTO CANCERDATA VALUES(301, 'Kavya', 'Breast', 45, 'II', 8);

INSERT INTO CANCERDATA VALUES(302, 'Arun', 'Lung', 60, 'III', 4);

INSERT INTO CANCERDATA VALUES(303, 'Divya', 'Breast', 38, 'I', 10);

INSERT INTO CANCERDATA VALUES(304, 'Hari', 'Colon', 52, 'II', 6);

INSERT INTO CANCERDATA VALUES(305, 'Meena', 'Lung', 57, 'IV', 2);

INSERT INTO CANCERDATA VALUES(306, 'Ravi', 'Prostate', 63, 'III', 5);

INSERT INTO CANCERDATA VALUES(307, 'Priya', 'Breast', 41, 'II', 9);

QUERY 1

SELECT *FROM CANCERDATA WHERE CancerType = 'Breast';

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	CANCERTYPE	AGE	STAGE	SURVIVALYEARS
301	Kavya	Breast	45	II	8
303	Divya	Breast	38	I	10
307	Priya	Breast	41	II	9

3 rows returned in 0.02 seconds CSV Export

QUERY 2

SELECT PatientName FROM CANCERDATA WHERE Stage = 'III';

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Ravi

2 rows returned in 0.01 seconds CSV Export

QUERY 3

SELECT PatientName, Age FROM CANCERDATA ORDER BY Age ASC;

Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Divya	38
Priya	41
Kavya	45
Hari	52
Meena	57
Arun	60
Ravi	63

7 rows returned in 0.01 seconds CSV Export

QUERY 4

SELECT PatientName FROM CANCERDATA WHERE SurvivalYears > 5;

Results Explain Describe Saved SQL History

PATIENTNAME
Kavya
Divya
Hari
Priya

4 rows returned in 0.02 seconds CSV Export

QUERY 5

SELECT COUNT(*)FROM CANCERDATA WHERE CancerType = 'Lung';

Results Explain Describe Saved SQL History

COUNT(*)
2

1 rows returned in 0.00 seconds CSV Export

QUERY 6

INSERT INTO CANCERDATA VALUES(308, 'Anand', 'Leukemia', 50, 'II', 7);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

QUERY 7

SELECT COUNT(DISTINCT CancerType)FROM CANCERDATA;

Results Explain Describe Saved SQL History

COUNT(DISTINCTCANCERTYPE)
5

1 rows returned in 0.01 seconds CSV Export

QUERY 8

SELECT MAX(SurvivalYears) FROM CANCERDATA;

Results Explain Describe Saved SQL History

MAX(SURVIVALYEARS)
10

1 rows returned in 0.00 seconds CSV Export

QUERY 9

SELECT AVG(Age)FROM CANCERDATA WHERE CancerType = 'Breast';

Results Explain Describe Saved SQL History

AVG(AGE)
41.333333333333333333333333333333333333

1 rows returned in 0.00 seconds CSV Export

QUERY 10

SELECT MIN(Age)FROM CANCERDATA WHERE Stage = 'II';

Results Explain Describe Saved SQL History

MIN(AGE)
41

1 rows returned in 0.00 seconds CSV Export

QUESTION-19

TABLE 1

CREATE TABLE ITEMS (CODE NUMBER,INAME VARCHAR2(30),QTY NUMBER,PRICE NUMBER,

COMPANY VARCHAR2(30),TCODE VARCHAR2(5));

INSERT INTO ITEMS VALUES(1001, 'DIGITAL PAD12', 120, 11000, 'XENITA', 'T01');

INSERT INTO ITEMS VALUES(1006, 'LED SCREEN 40', 70, 38000, 'SANTORA', 'T02');

INSERT INTO ITEMS VALUES(1004, 'CAR GPS SYSTEM', 50, 21500, 'GEOKNOW', 'T01');

INSERT INTO ITEMS VALUES(1003, 'DIGITAL CAMERA 12X', 160, 8000, 'DIGICLICK', 'T02');

INSERT INTO ITEMS VALUES(1005, 'PEN DRIVE 32 GB', 600, 1200, 'STOREHOME', 'T03');

TABLE 2

CREATE TABLE TRADERS (TCODE VARCHAR2(5),TNAME VARCHAR2(30),CITY VARCHAR2(20));

INSERT INTO TRADERS VALUES('T01', 'ELECTRONIC SALES', 'MUMBAI');

INSERT INTO TRADERS VALUES('T03', 'BUSY STORE CORP', 'DELHI');

INSERT INTO TRADERS VALUES('T02', 'DISP HOUSE INC', 'CHENNAI');

QUERY 1

SELECT *FROM ITEMS ORDER BY INAME ASC;

Results Explain Describe Saved SQL History

CODE	INAME	QTY	PRICE	COMPANY	TCODE
1004	CAR GPS SYSTEM	50	21500	GEOKNOW	T01
1003	DIGITAL CAMERA 12X	160	8000	DIGICLICK	T02
1001	DIGITAL PAD12	120	11000	XENITA	T01
1006	LED SCREEN 40	70	38000	SANTORA	T02
1005	PEN DRIVE 32 GB	600	1200	STOREHOME	T03

5 rows returned in 0.03 seconds CSV Export

QUERY 2

SELECT INAME, PRICE FROM ITEMS WHERE PRICE BETWEEN 10000 AND 22000;

Results Explain Describe Saved SQL History

INAME	PRICE
DIGITAL PAD12	11000
CAR GPS SYSTEM	21500

2 rows returned in 0.00 seconds CSV Export

QUERY 3

SELECT TCODE, COUNT(*)FROM ITEMS GROUP BY TCODE;

Results Explain Describe Saved SQL History

TCODE	COUNT(*)
T01	2
T03	1
T02	2

3 rows returned in 0.01 seconds CSV Export

QUERY 4

SELECT PRICE, INAME, QTY FROM ITEMS WHERE QTY > 150;

Results Explain Describe Saved SQL History

PRICE	INAME	QTY
8000	DIGITAL CAMERA 12X	160
1200	PEN DRIVE 32 GB	600

2 rows returned in 0.02 seconds CSV Export

QUERY 5

SELECT TNAME FROM TRADERS WHERE CITY IN ('DELHI', 'MUMBAI');

Results Explain Describe Saved SQL History

TNAME
ELECTRONIC SALES
BUSY STORE CORP

2 rows returned in 0.01 seconds CSV Export

QUERY 6

SELECT COMPANY, INAME FROM ITEMS ORDER BY COMPANY DESC;

Results Explain Describe Saved SQL History

COMPANY	INAME
XENITA	DIGITAL PAD12
STOREHOME	PEN DRIVE 32 GB
SANTORA	LED SCREEN 40
GEOKNOW	CAR GPS SYSTEM
DIGICLICK	DIGITAL CAMERA 12X

5 rows returned in 0.01 seconds CSV Export

QUERY 7

SELECT MAX(PRICE), MIN(PRICE) FROM ITEMS;

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
38000	1200

1 rows returned in 0.02 seconds CSV Export

QUERY 8

SELECT PRICE * QTY AS AMOUNT FROM ITEMS WHERE CODE = 1004;

Results Explain Describe Saved SQL History

AMOUNT
1075000

1 rows returned in 0.00 seconds CSV Export

QUERRY 9

SELECT DISTINCT TCODE FROM ITEMS;

Results Explain Describe Saved SQL History

TCODE
T01
T03
T02

3 rows returned in 0.02 seconds CSV Export

Query 10

SELECT I.INAME, T.TNAME FROM ITEMS I, TRADERS T WHERE I.TCODE = T.TCODE AND I.QTY < 100;

Results Explain Describe Saved SQL History

INAME	TNAME
CAR GPS SYSTEM	ELECTRONIC SALES
LED SCREEN 40	DISP HOUSE INC

2 rows returned in 0.02 seconds CSV Export

TRUTH-TOIL-TRIUMPH AUTONOMOUS